

CS 260B Algorithmic Machine Learning scribed notes

Yimeng(Kobe) Wang

April 2025

Abstract

This document contains Yimeng(Kobe) Wang's scribed notes when he was TAing for CS 260B Algorithmic Machine Learning at UCLA, taught by Professor Raghu Meka in Spring 2025. The link to the course website is here: [link to website](#).

Some of Raghu's lectures may cover a couple of different topics. These notes will be organized based on the topics covered. For instance, although in Lecture 5 Raghu covered both adaptive methods and the beginning of online learning, for the purpose of these notes, the online learning portion of Lecture 5 is combined with lecture 6 for a more coherent presentation.

The notes may contain typos / errors. If an error is spotted, please contact kobewang1998@gmail.com.

Contents

I	Optimization	5
1	Lecture 1. Optimization and Gradient Descent	5
1.1	Introduction	5
1.2	Learning as Optimization	5
1.3	Gradient Descent	7
2	Lecture 2. Properties of functions and Analysis of Gradient Descent	10
2.1	Lipschitzness and Smoothness	10
2.2	Monotonicity of Gradient Descent	11
2.3	Convexity	12
2.4	Analysis of Gradient Descent	14
3	Lecture 3. Accelerated Gradient Descent	16
3.1	Quadratic functions and Heavy Ball method	16
3.2	Nesterov's Accelerated Gradient Descent	18
4	Lecture 4. Stochastic Gradient Descent and adaptive methods	20
4.1	Cost of Gradient Descent	20
4.2	Stochastic Gradient Descent	21
4.3	Convergence Analysis for SGD	22
4.4	Adaptive step-sizes	24
5	Lecture 5. More on Adaptive Methods and Convex Optimization	25
5.1	AdaGrad and variants	25
5.2	ADAM	25
5.3	Extension to Constrained Optimization	27
II	Online Learning	29
6	Lecture 6. Online Learning and Perceptron	29
6.1	Online Learning of Halfspaces	29
6.2	Regret Minimization and Learning with Experts	33
6.2.1	Follow the Leader (FTL)	33
6.2.2	Follow the Majority (FTM)	34
6.2.3	Follow the Weighted Majority (FTWM)	34
7	Lecture 7. Multiplicative Weights Method	37
8	Lecture 8. Bandit Learning	40
8.1	Upper Confidence Bound (UCB) algorithm	41
8.2	Thompson Sampling	43
III	Principal Component Analysis	45
9	Lecture 9. Best-fit Subspace	45
9.1	Quick Linear Algebra Review	46
9.2	Principal Components	46
9.3	Top Principal Components and best-fit subspace	49

10 Lecture 10. Low-rank Approximation	52
10.1 Netflix Challenge	53
10.2 Singular Value Decomposition (SVD)	53
10.2.1 SVD definition and examples	53
10.2.2 Computing SVD	55
10.3 Power Iteration	56
10.3.1 Variants of Power Iterations	58
10.4 Matrix Completion	59
 IV Privacy	 62
11 Lecture 11. Privacy in Machine Learning	62
11.1 Introduction	62
11.2 Mean Estimation	63
11.3 Differential Privacy	64
12 Lecture 12. The Laplacian Mechanism	66
12.1 Laplacian Mechanism	67
12.2 Properties of Differential Privacy	69
12.2.1 Meta Processing	69
12.2.2 Group Privacy	69
12.2.3 Some example of Queries	70
13 Lecture 13. The Exponential Mechanism and other topics in DP	72
13.1 The Exponential Mechanism	72
13.2 Approximate Differential Privacy	74
13.2.1 Properties of Approximate-DP	75
13.3 Some Practical Scenarios for DP	76
13.3.1 Adaptive Data Analysis	76
13.3.2 Kaggle / ML Competitions	77
13.4 Private Empirical Risk Minimization	77
 V Distribution Learning	 79
14 Lecture 14. Bayesian Network	79
14.1 Bayesian Networks	79
14.2 Tree Learning	82
14.2.1 Finding the best D_T given T	83
14.3 Chow-Liu Algorithm	84
15 Lecture 15. Undirected Graphical Models	86
15.1 Undirected Graphical Models	86
15.2 Boltzmann Machines / Ising Models	88
15.3 Sparsitron	91
15.4 Generalized Linear Model	93
16 Lecture 16. Diffusion Model	94
16.1 Langevin Dynamics	94
16.2 Diffusion model	97
16.3 The Learning Problem	99

17 Lecture 17. Transformers	101
17.1 Introduction	101
17.2 Language Modeling	101
17.3 Attention	103
17.4 Runtime efficiency	106
 VI Reinforcement Learning	 108
18 Lecture. Markov Decision Process	108
18.1 Policy evaluation	108
18.2 Finding the optimal policy for Planning	109

Part I

Optimization

Lecture 1. Optimization and Gradient Descent

1.1 Introduction

Welcome to CS 260B. In this class, we will study and understand a handful of algorithms used in Machine Learning. Machine Learning is a big topic that covers a lot of grounds. There are quite a few different subareas one study within the field.

Just to list a few: Reinforcement Learning, Linear Regression, Supervised Learning, Clustering / Unsupervised Learning, Representation Learning, Distribution Learning, Active Learning, Online Learning and many others. We will touch upon many topics listed above in this class. To start, let's consider the most familiar setting to most people: *Supervised Learning*.

In Supervised Learning, we have some domain $\mathcal{X}$. This may be images, texts, DNA sequences, biometric data, etc., depending on your task. For each such information from the domain, we have an associated label from a set $\mathcal{L}$. For instance, labels can be categorical such as ("cats", "dogs", "birds") or labels can be numerical information. Then in a Supervised Learning problem, we are given as data

$$(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n),$$

where $x^i \in \mathcal{X}$ and $y^i \in \mathcal{L}$ for each i . This set of data $\{(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)\}$ is called *training data*. Our goal is: given a new point $x \in \mathcal{X}$, predict its associated label y .

To make this task mathematically rigorous and solvable, we want to make a couple of assumptions on the data.

Assumption 1.1. *There is a "ground truth" distribution on the training and test examples.*

Formally speaking, this means there exists a probability distribution $\mathcal{D}$ over the domain $\mathcal{X}$ (note this also defines a distribution over $\mathcal{X} \times \mathcal{L}$ since there is a ground truth function that maps $\mathcal{X}$ to $\mathcal{L}$). Our samples $x_1, x_2, \dots, x_n$ are sampled from $\mathcal{D}$. At the same time, the new data that we want to test our algorithm on are also drawn from the same distribution $\mathcal{D}$.

Since the set of functions are infinite-dimensional and almost impossible to work with, we need to impose some limitations on the choice of functions we can use to make our task hopefully a solvable one.

Assumption 1.2. *There's a restricted class of predictors we are competing against.*

1.2 Learning as Optimization

With the assumptions above, we are now ready model Supervised Learning as an Optimization problem. The setup is the following: we have an underlying distribution $\bar{\mathcal{D}}$ on $\mathcal{X} \times \mathcal{L}$ from which the training and testing samples are drawn. We also have a class of predictors that we are competing against. For now we will consider "parametrized hypothesis / predictor class," i.e., predictors from this class are determined by some parameter. Lastly, we have a loss function $l : \mathcal{L} \times \mathcal{L}$ that evaluates how good our predictions are.

In summary, given inputs $(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)$ drawn from some distribution $\bar{\mathcal{D}}$, we want to output $\theta \in \Theta$ that minimizes the *average loss*

$$\frac{1}{n} \sum_{i=1}^n l(h_{\theta}(x^i), y^i).$$

Here l is the *loss function* and h_{θ} is the *predictor with parameter θ* . The whole expression above is known as the *empirical loss* or *training loss*.

To obtain the best predictor, it would be ideal if we can set up and minimize the problem with respect to the true distribution $\mathcal{D}$, i.e.,

$$\min_{\theta \in \Theta} \mathbf{E}_{(x,y) \sim \mathcal{D}} [l(h_{\theta}(x), y)].$$

The function being minimized above is also called as *population loss*. However, this is not practical since we do not know what the actual distribution is and we can only have access to the distribution $\bar{\mathcal{D}}$ via the samples. This leads us to the task of *Empirical Risk Minimization (ERM)*:

Given inputs $(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n) \sim \bar{\mathcal{D}}$, find $\theta \in \Theta$ that minimizes

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n l(h_\theta(x_i), y_i). \quad (1)$$

Now let's see examples of some supervised learning problem stated as a ERM problem.

Example 1.3. (*Linear Prediction*) Let the domain be $\mathcal{X} = \mathbb{R}^d$ and the labels be $\mathcal{L} = \mathbb{R}$. Let the predictor class $\Theta = \mathbb{R}^d$. Let the predictor associated with $\theta \in \Theta$ be the linear predictor:

$$h_\theta(x) = \sum_{i=1}^n \theta_i x_i = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d = \langle \theta, x \rangle.$$

Pick the loss function $l : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ to be $l(a, b) = (a - b)^2$. Then the ERM problem can be stated as:

Given inputs $(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n) \sim \bar{\mathcal{D}}$, find $\theta \in \Theta$ that minimizes

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (\langle \theta, x^i \rangle - y^i)^2. \quad (2)$$

This formalization of the problem is often called Least Squares Regression.

Example 1.4. We can obtain a very different problem by picking a different loss function. Let the domain be $\mathcal{X} = \mathbb{R}^d$ and the labels be $\mathcal{L} = \mathbb{R}$. Let the predictor class $\Theta = \mathbb{R}^d$. Let the predictor associated with $\theta \in \Theta$ be the linear predictor:

$$h_\theta(x) = \sum_{i=1}^n \theta_i x_i = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d = \langle \theta, x \rangle.$$

Pick the loss function $l : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ to be $l(a, b) = |a - b|$. Then the ERM problem can be stated as:

Given inputs $(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n) \sim \bar{\mathcal{D}}$, find $\theta \in \Theta$ that minimizes

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n |\langle \theta, x^i \rangle - y^i|. \quad (3)$$

Example 1.5. The following problem looks almost identical to Least Squares Regression except for a tweak on predictor class. Let the domain be $\mathcal{X} = \mathbb{R}^d$ and the labels be $\mathcal{L} = \mathbb{R}$. Let the predictor class $\Theta = \mathbb{R}_{\geq 0}^d$, i.e., all coordinates of $\theta \in \Theta$ are non-negative. Let the predictor associated with $\theta \in \Theta$ be the linear predictor:

$$h_\theta(x) = \sum_{i=1}^n \theta_i x_i = \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_d x_d = \langle \theta, x \rangle.$$

Pick the loss function $l : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ to be $l(a, b) = (a - b)^2$. Then the ERM problem can be stated as:

Given inputs $(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n) \sim \bar{\mathcal{D}}$, find $\theta \in \mathbb{R}_{\geq 0}^d$ that minimizes

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (\langle \theta, x^i \rangle - y^i)^2. \quad (4)$$

On a first glance, one might observe these different formalizations to be very similar to each other. Depending on the specific supervised learning problem, we might want to choose one formalization over the others. However, one major differences between them is that these formalizations give very different optimization problems (L_1 loss vs. L_2 loss, constrained vs. unconstrained).

1.3 Gradient Descent

Having formalized supervised learning problems as optimization problems, we want to investigate methods that actually solve them. We will start with the simplest case, namely, *unconstrained optimization*. In this case, our goal is to simply find the parameter that minimizes the loss in the entire space. There are no extra conditions the solution need to satisfy. Given a function $L : \mathbb{R}^d \rightarrow \mathbb{R}$, the goal is to find θ that minimizes $L(\theta)$.

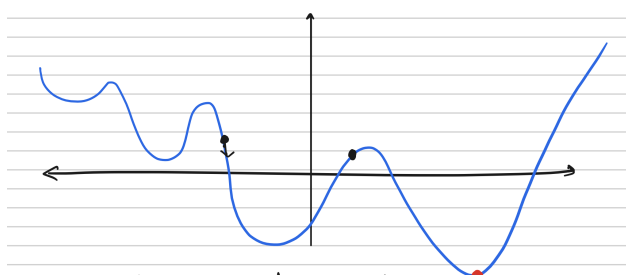


Figure 1: Example of optimization: point in red is where the minimum of the function is attained. Our goal is find or get close to this point as much as possible.

Here is an idea for how to find the minimum:

- Pick a starting point.
- Then, move in a direction that reduces the function value the most per *distance* traveled.

In mathematics, we can have the quantity that satisfies these desired conditions.

Claim 1.6. *If the distance is Euclidean, then the direction of the "steepest descent" is $-\nabla L(\theta)$.*

Let's recall what gradient is. Let $L : \mathbb{R}^d \rightarrow \mathbb{R}$ be a function. Then the gradient of L is given by

$$\nabla L(\theta) = \left(\frac{\partial L}{\partial \theta_1}, \frac{\partial L}{\partial \theta_2}, \dots, \frac{\partial L}{\partial \theta_d} \right),$$

where $\partial L / \partial \theta_i$ are the partial derivatives. Now we are ready to state the famous *Gradient Descent* algorithm.

Algorithm 1 Gradient Descent (GD)

Starting at some point $\theta_0 \in \mathbb{R}^d$

for $i = 1, 2, \dots, T$:

- $\theta_i = \theta_{i-1} - \eta \nabla L(\theta_{i-1})$
-

where $\eta > 0$ is called the *step-size* or *learning rate* and T is the stopping time determined by some stopping condition.

Example 1.7. *Recall the Least Squares Regression: given input $(x^1, y^1), (x^n, y^n), \dots, (x^n, y^n)$, find θ that minimizes*

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (\langle \theta, x^i \rangle - y^i)^2.$$

By the chain rule, we can compute the gradient as:

$$\begin{aligned} \nabla L(\theta) &= \frac{1}{n} \sum_{i=1}^n \nabla_{\theta} ((\langle \theta, x^i \rangle - y^i)^2) \\ &= \frac{1}{n} \sum_{i=1}^n 2(\langle \theta, x^i \rangle - y^i) x^i. \end{aligned}$$

Therefore, the Gradient Descent update for LSR can be written as: for $i = 1, 2, \dots, T$,

$$\theta_i = \theta_{i-1} - \eta \cdot \frac{1}{n} \sum_{i=1}^n 2(\langle \theta, x^i \rangle - y^i) x^i.$$

Example 1.8. We can do the similar thing for the L_1 loss function $L(a, b) = |a - b|$. In this case, we want to find the θ that minimizes:

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n |\langle \theta, x^i \rangle - y^i|.$$

We would like to compute the gradient and apply gradient descent. However, from your calculus class, you may recall that the absolute value $f(x) = |x|$ is not differentiable at $x = 0$. Thus, the gradient $\nabla L(\theta)$ does not exist for some values of θ .

To circumvent this problem, instead of using the exact gradient in the update, we can use subgradient as a substitute for gradient.

Definition 1.9. (subgradient) A direction g is a subgradient of f at x_0 if for all x in some radius around x_0 , the line with slope g is tangent to the curve at x_0 .

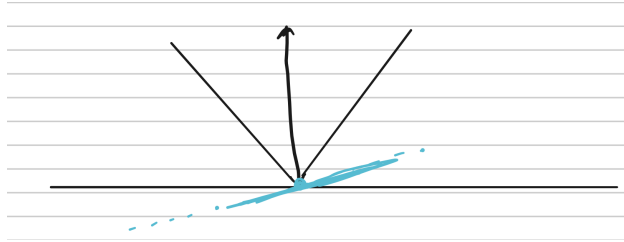


Figure 2: One subgradient (in blue) of $f(x) = |x|$.

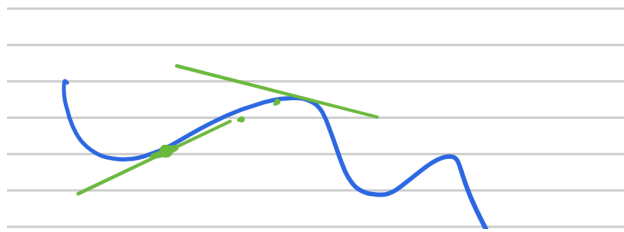


Figure 3: Subgradients (in a green) of the curve at different points.

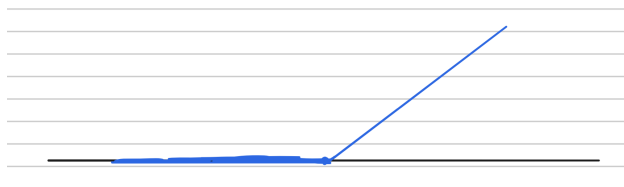


Figure 4: A subgradient (in black) of the ReLU function $f(x) = \max(0, x)$ (in blue).

It is worth noting (evident from the definition) that subgradient of a function f at x is generally not unique. Despite this, in many cases, subgradients have the essential properties of gradients. Because of this, we can now use subgradients in place of gradient for functions that are not differentiable in gradient descent.

Algorithm 2 Subgradient Descent

Starting at some point $\theta_0 \in \mathbb{R}^d$

for $i = 1, 2, \dots, T$:

- $\theta_i = \theta_{i-1} - \eta g$ $\triangleright g$ is a subgradient of L at θ_{i-1} .
-

There are of course, many other optimization algorithms out there. But the biggest advantage of Gradient / Sub-gradient descent is that in order to use it, we *only need to be able to compute* ∇L . Since we only need the gradient, gradient descent is a *first-order method*.

In summary, for a ERM problem, we want to minimize the objective

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n l(h_\theta(x^i), y^i).$$

The gradient of L is given by

$$\begin{aligned} \nabla L(\theta) &= \frac{1}{n} \sum_{i=1}^n \nabla L(h_\theta(x^i), y^i) \\ &= \frac{1}{n} \sum_{i=1}^n \frac{\partial l}{\partial a}(h_\theta(x^i), y^i) \cdot \nabla_\theta h_\theta(x^i) \end{aligned}$$

where $\frac{\partial l}{\partial a}$ denotes the partial derivative with respect to the first element. Using this, we can run Gradient Descent to optimize $L(\theta)$. In the next lecture, we will analyze the rate of convergence of gradient descent.

Lecture 2. Properties of functions and Analysis of Gradient Descent

Last time, we talked about the gradient descent method for optimization. Recall the gradient descent (GD) update:

Given a loss function $L : \mathbb{R}^d \rightarrow \mathbb{R}_{\geq 0}$ and a initialization point $x^0 \in \mathbb{R}^d$, GD updates the point in the following manner:

For $i = 1, 2, \dots, T$:

$$x^i \leftarrow x^{i-1} - \eta \nabla L(x^{i-1}), \quad (5)$$

where $\nabla L(x^{i-1})$ is the gradient of L at x^{i-1} and $\eta > 0$ is the step-size.

One of the main goals of this part of the course is to prove that gradient descent can help us find the minimizer of the loss function efficiently. There are pathological examples of functions that are badly-behaved (see for example, Figure 5). As a result of this, we need some conditions that rule out such bad functions.

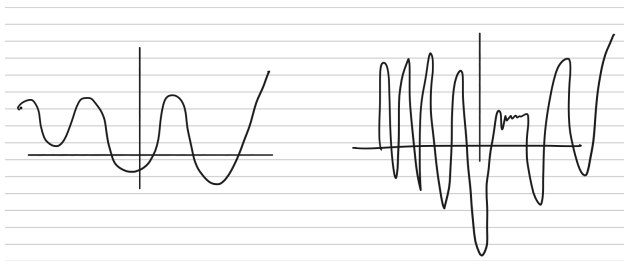


Figure 5: Examples of badly-behaved functions

2.1 Lipschitzness and Smoothness

Definition 2.1. (*Lipschitzness*) $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is L -Lipschitz if for all $x, y \in \mathbb{R}^d$,

$$|f(x) - f(y)| \leq L \cdot \|x - y\|_2,$$

where L is a positive real number and $\|x - y\|_2 = (\sum_{i=1}^n (x_i - y_i)^2)^{1/2}$ is the l_2 -distance between x and y .

Example 2.2. • Consider the function $f : \mathbb{R} \rightarrow \mathbb{R}$ defined as $f(x) = |x|$. Then for any $x, y \in \mathbb{R}$

$$|f(x) - f(y)| = ||x| - |y|| \leq 1 \cdot |x - y|.$$

Therefore, f is 1-Lipschitz.

• Now consider the function $g : \mathbb{R} \rightarrow \mathbb{R}$ defined as $g(x) = x^2$. Then for any $x, y \in \mathbb{R}$

$$|f(x) - f(y)| = |x^2 - y^2| = |x + y| \cdot |x - y|.$$

Since the term $|x + y|$ can be arbitrarily large, g is not Lipschitz.

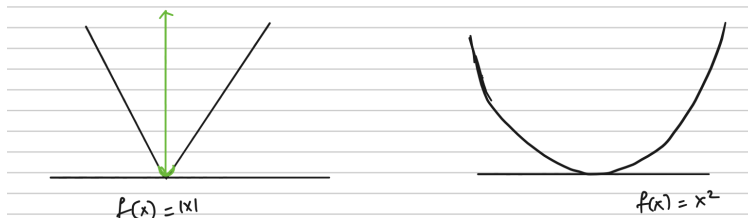


Figure 6: Plot of $f(x) = |x|$ (left) and $g(x) = x^2$ (right)

Definition 2.3. (Smoothness) $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is β -smooth if for all x, y

$$\|\nabla f(x) - \nabla f(y)\|_2 \leq \beta \cdot \|x - y\|_2, \quad (6)$$

where β is a positive real number.

Example 2.4. • The function $f(x) = |x|$ is not smooth. Notice that for any $x > 0$, $\nabla f(x) = 1$ and for any $y < 0$, $\nabla f(y) = -1$. Therefore, the left-hand-side of equation 6 is fixed to be 2 for $x > 0$ and $y < 0$. But on the right-hand-side, $\|x - y\|_2$ can be arbitrarily small for $x > 0, y < 0$. So there cannot be a fixed positive β that satisfies equation 6 for all x, y .

- The function $g(x) = x^2$ is 2-smooth. Since $\nabla g(x) = 2x$, we thus have

$$|\nabla g(x) - \nabla g(y)| = 2 \cdot |x - y|.$$

Hence, by definition, g is 2-smooth.

2.2 Monotonicity of Gradient Descent

Our goal now is to prove that under certain conditions, gradient descent updates bring us closer to the optimizer. To this end, we will crucially use the following consequence of smoothness.

Lemma 2.5. (Smoothness upper bound over $\mathbb{R}$) Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a β -smooth function. Then for all $a, h \in \mathbb{R}$, we have

$$f(a + h) \leq f(a) + f'(a) \cdot h + \frac{\beta h^2}{2}.$$

Now we are ready to prove our first positive result about the Gradient Descent method. We will first prove this result for functions over $\mathbb{R}$. Then we will state an analogous version of Lemma 2.5 over $\mathbb{R}^d$ and use that to prove the analogous result over $\mathbb{R}^d$.

Lemma 2.6. (Monotonicity of Gradient Descent over $\mathbb{R}$) Let $f : \mathbb{R} \rightarrow \mathbb{R}$ be a β -smooth function. Then for $\eta \leq 1/\beta$, we have

$$f(x_{i+1}) \leq f(x_i) - \frac{\eta}{2} (f'(x_i))^2.$$

Proof. Recall for the gradient update over $\mathbb{R}$, we have

$$x_{i+1} = x_i - \eta \nabla f(x_i) = x_i - \eta f'(x_i).$$

Now we apply Lemma 2.5 with $a = x_i$ and $h = -\eta f'(x_i)$ and obtain

$$\begin{aligned} f(x_{i+1}) &= f(x_i - \eta f'(x_i)) \leq f(x_i) + f'(x_i) \cdot (-\eta f'(x_i)) + \frac{\beta(-\eta f'(x_i))^2}{2} \\ &= f(x_i) - \eta f'(x_i)^2 + \frac{\beta \eta^2 f'(x_i)^2}{2} \\ &= f(x_i) - \eta \left(1 - \frac{\beta \eta}{2}\right) \cdot f'(x_i)^2 \\ &\leq f(x_i) - \frac{\eta}{2} f'(x_i)^2 \end{aligned}$$

where in the last line we used the assumption that $\eta \leq 1/\beta$. □

Now we prove the monotonicity of gradient descent over $\mathbb{R}^d$. Before we do this, we need the following smoothness upper bound over $\mathbb{R}^d$.

Lemma 2.7. (Smoothness upper bound over $\mathbb{R}^d$) Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a β -smooth function. Then for all $a, h \in \mathbb{R}^d$, we have

$$f(a + h) \leq f(a) + \langle \nabla f(a), h \rangle + \frac{\beta}{2} \|h\|^2.$$

Lemma 2.8. (*Monotonicity of Gradient Descent over $\mathbb{R}^d$*) If $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is β -smooth and $\eta \leq 1/\beta$. Then

$$f(x_{i+1}) \leq f(x_i) - \frac{\eta}{2} \|\nabla f(x_i)\|^2.$$

Proof. Recall the gradient update $x_{i+1} = x_i - \eta \nabla f(x_i)$. Applying Lemma 2.7 with $a = x_i$ and $h = -\eta \nabla f(x_i)$, we get

$$\begin{aligned} f(x_{i+1}) &= f(x_i - \eta \nabla f(x_i)) \leq f(x_i) + \langle \nabla f(x_i), h \rangle + \frac{\beta}{2} \|h\|^2 \\ &= f(x_i) + \langle \nabla f(x_i), -\eta \nabla f(x_i) \rangle + \frac{\beta}{2} \|\eta \nabla f(x_i)\|^2 \\ &= f(x_i) - \eta \|\nabla f(x_i)\|^2 + \frac{\beta \eta^2}{2} \|\nabla f(x_i)\|^2 \\ &= f(x_i) - \eta \|\nabla f(x_i)\|^2 \cdot \left(1 - \frac{\beta \eta}{2}\right) \\ &\leq f(x_i) - \frac{\eta}{2} \|\nabla f(x_i)\|^2 \end{aligned}$$

where from the 2nd to 3rd line we used the definition: $\langle v, v \rangle = \|v\|^2$. The last line follows since by assumption $\eta \leq 1/\beta$. $\square$

Based on the lemmas we just proved, we know that for β -smooth functions $f : \mathbb{R}^d \rightarrow \mathbb{R}$, gradient descent can make progress as long as the step size η is appropriately chosen such that $\eta \leq \frac{1}{\beta}$. Notice that the amount of progress we make in each iteration depends on how large η is. So for gradient descent to make as much progress as it can, we want to choose η as large as possible while holding $\eta \leq 1/\beta$.

Hence, one thing we can do is to start with a step-size η that satisfies $\eta \leq 1/\beta$ and greedily increase the η until the condition $\eta \leq 1/\beta$ is violated. If so, we "backtrack" to one of the previous value of η we tried and repeat the process with a smaller increment of η . This process is called *backtracking line search*. We won't go into details on this.

2.3 Convexity

The next nice property of a function is *convexity*. Convexity is perhaps one of the most important concept in optimization theory and engineering mathematics. It has numerous applications in, for instance, optimization, learning theory, algorithms, geometry and many other domains of mathematics and engineering.

Perhaps the most intuitive definition of convexity is through geometry: we say a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is *convex* if for any point x , the tangent plane at x is always below f (see Figure 7).

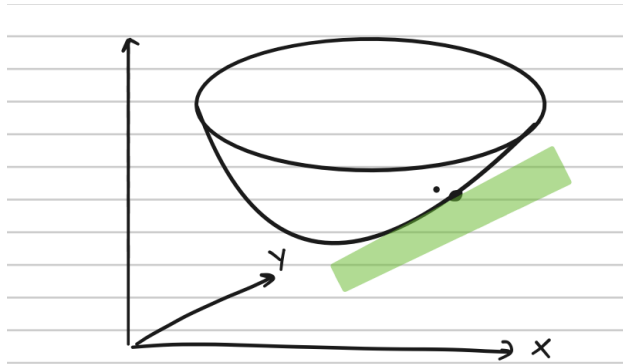


Figure 7: The tangent plane at x is below f

Another graphic intuition of convexity is if you plot your function f on a graph, then the line connecting any two point x, y lie above the graph of the function (See Figure 8).

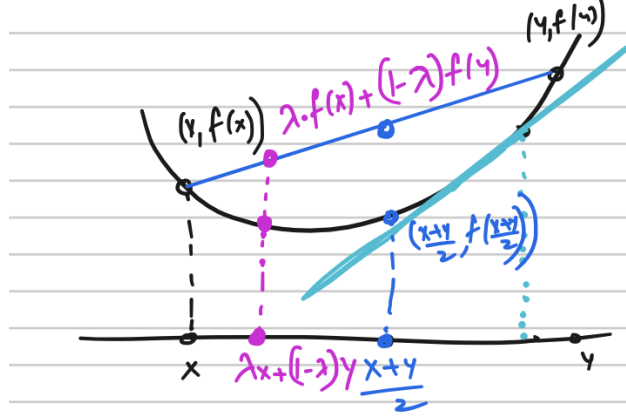


Figure 8: Line connecting any two point on the plot of f lies above f

Formally speaking we have the following equivalent definitions of convexity.

Definition 2.9. (Convexity) A function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is convex if any of the following equivalent conditions holds:

1. $\forall x, y \in \mathbb{R}^d, f(\frac{x+y}{2}) \leq \frac{1}{2}(f(x) + f(y))$
2. $\forall x, y \in \mathbb{R}^d$ and $0 \leq \lambda \leq 1$,

$$f(\lambda x + (1 - \lambda)y) \leq \lambda f(x) + (1 - \lambda)f(y)$$
3. $\forall x, h \in \mathbb{R}^d$,

$$f(x) + \langle h, \nabla f(x) \rangle \leq f(x + h).$$

Another equivalent condition is $\nabla^2 f(x) \succeq 0$, i.e., the Hessian of f at x is positive semi-definite (if you are not familiar with these terms, do not worry. We will not be using this definition much, if at all).

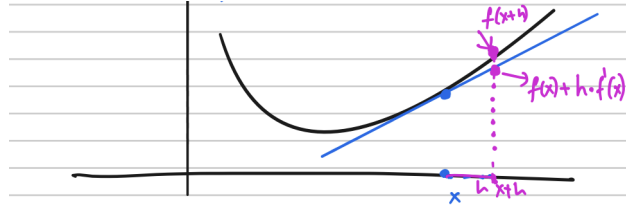


Figure 9: An illustration of condition 3 for $f : \mathbb{R} \rightarrow \mathbb{R}$

As mentioned earlier, convexity shows up naturally in many applications. It is an extremely useful concept for understanding and developing algorithms such as Linear Programming and Semi-definite programming.

Convex functions have some nice closure properties that we will be using later:

Lemma 2.10. (Closure properties of convex functions)

1. If $f, g : \mathbb{R}^d \rightarrow \mathbb{R}$ are both convex, then $f + g$ is also convex.
2. If $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is convex, then $a \cdot f$ is also convex where $a \in \mathbb{R}$ and $a > 0$.
3. Suppose $g : \mathbb{R} \rightarrow \mathbb{R}$ is convex. For $\omega \in \mathbb{R}^d$, define the function $g_\omega : \mathbb{R}^d \rightarrow \mathbb{R}$ as $g_\omega(x) = g(\langle \omega, x \rangle)$. Then g_ω is also convex.

Example 2.11. • The functions $x^2, |x|, e^x, -\log x$ are all convex.

- For $x \in \mathbb{R}^d$, $y \in \mathbb{R}$, the function

$$h(\theta) = (\langle \theta, x \rangle - y)^2$$

is convex.

- Many common loss functions in machine learning are convex: the L2-loss function

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n (\langle \theta, x_i \rangle - y_i)^2$$

is convex. The L1-loss function

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n |\langle \theta, x_i \rangle - y_i|$$

is convex. The LASSO loss function

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n |\langle \theta, x_i \rangle - y_i| + |\theta_1| + |\theta_2| + \dots + |\theta_d|$$

is also convex.

Convex functions are nice to work with. As we will see in the next theorem, for convex functions, gradient descent makes progress toward the optimal solution.

2.4 Analysis of Gradient Descent

Theorem 2.12. (Fundamental Theorem of Optimization) Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be β -smooth and convex. Then for $\eta \leq 1/\beta$, we have

$$f(x_k) \leq f(x_*) + \frac{\|x_0 - x_*\|^2}{2\eta k}$$

where x_* is the vector that achieves the minimum function value $f(x_*)$, i.e., $f(x_*) = \min_{x \in \mathbb{R}^d} f(x)$.

In short, convex optimization is easy!

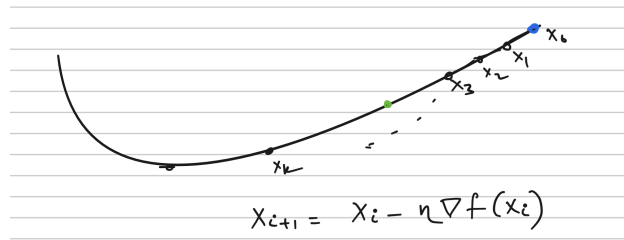


Figure 10: Gradient Descent steps for convex function f

We need the following lemma in the proof of Theorem 2.12

Lemma 2.13. Let $u, v \in \mathbb{R}^d$. Then

$$2\langle u - v, u \rangle - \|u - v\|^2 = \|u\|^2 - \|v\|^2.$$

Proof. We know

$$\|u - v\|^2 = \langle u - v, u - v \rangle = \|u\|^2 - 2\langle u, v \rangle + \|v\|^2.$$

Thus,

$$\begin{aligned} 2\langle u - v, u \rangle - \|u - v\|^2 &= 2(\|u\|^2 - \langle u, v \rangle) - (\|u\|^2 - 2\langle u, v \rangle + \|v\|^2) \\ &= \|u\|^2 - \|v\|^2. \end{aligned}$$

□

We are now ready to prove the main theorem.

Proof. (of Theorem 2.12) Since f is β -smooth and $\eta \leq 1/\beta$, we can apply Lemma 2.8 (monotonicity of GD) and get

$$f(x_{i+1}) \leq f(x_i) - \frac{\eta \|\nabla f(x_i)\|^2}{2}. \quad (7)$$

On the other hand, since f is convex, applying 3 of Definition 2.9 with $x = x_i$ and $h = x_* - x_i$, we have

$$f(x_i) + \langle \nabla f(x_i), x_* - x_i \rangle \leq f(x_*). \quad (8)$$

Summing equations (7) (8) and subtracting $f(x_i)$ from both sides, we get

$$f(x_{i+1}) + \langle \nabla f(x_i), x_* - x_i \rangle \leq f(x_*) - \frac{\eta \|\nabla f(x_i)\|^2}{2}.$$

Rearranging, we can write the above as

$$f(x_{i+1}) - f(x_*) \leq \langle \nabla f(x_i), x_* - x_i \rangle - \frac{\eta \|\nabla f(x_i)\|^2}{2}. \quad (9)$$

Now by the gradient update, we know $x_{i+1} = x_i - \eta \nabla f(x_i)$. Therefore, $\nabla f(x_i) = \frac{x_i - x_{i+1}}{\eta}$. Substituting this into (9), we obtain

$$f(x_{i+1}) - f(x_*) \leq \left\langle \frac{x_i - x_{i+1}}{\eta}, x_i - x_* \right\rangle - \frac{\eta}{2} \left\| \frac{x_i - x_{i+1}}{\eta} \right\|^2$$

and thus

$$f(x_{i+1}) - f(x_*) \leq \frac{1}{2\eta} [2\langle x_i - x_{i+1}, x_i - x_* \rangle - \|x_i - x_{i+1}\|^2].$$

Now applying Lemma 2.13 with $u = x_i - x_*$ and $v = x_{i+1} - x_*$, we have

$$f(x_{i+1}) - f(x_*) \leq \frac{1}{2\eta} [\|x_i - x_*\|^2 - \|x_{i+1} - x_*\|^2]. \quad (10)$$

Since equation (10) holds for all i , we have the following system of inequalities:

$$\begin{aligned} f(x_1) - f(x_*) &\leq \frac{1}{2\eta} [\|x_0 - x_*\|^2 - \|x_1 - x_*\|^2] \\ f(x_2) - f(x_*) &\leq \frac{1}{2\eta} [\|x_1 - x_*\|^2 - \|x_2 - x_*\|^2] \\ &\dots \\ f(x_k) - f(x_*) &\leq \frac{1}{2\eta} [\|x_{k-1} - x_*\|^2 - \|x_k - x_*\|^2]. \end{aligned}$$

Now we add these k inequalities up. Notice that on the right hand side, all intermediate terms would cancel except for $\|x_0 - x_*\|^2$ and $-\|x_k - x_*\|^2$. We thus have

$$f(x_1) + f(x_2) + \dots + f(x_k) - kf(x_*) \leq \frac{1}{2\eta} [\|x_0 - x_*\|^2 - \|x_k - x_*\|^2]. \quad (11)$$

Now by monotonicity of Gradient Descent (Lemma 2.8), we know

$$f(x_k) \leq f(x_{k-1}) \leq \dots \leq f(x_2) \leq f(x_1),$$

which suggests $kf(x_k) \leq f(x_1) + f(x_2) + \dots + f(x_k)$. Substituting this into equation (11), we thus have

$$kf(x_k) - kf(x_*) \leq \frac{1}{2\eta} [\|x_0 - x_*\|^2 - \|x_k - x_*\|^2] \leq \frac{1}{2\eta} \|x_0 - x_*\|^2.$$

Dividing both sides by $k + 1$ and rearranging, we obtain

$$f(x_k) \leq f(x_*) + \frac{\|x_0 - x_*\|^2}{2\eta k}.$$

□

Lecture 3. Accelerated Gradient Descent

Last time we discussed several nice properties of functions and rigorously show the convergence of gradient descent. Recall the Fundamental Theorem of Optimization we showed last time.

Theorem 3.1. (*Fundamental Theorem of Optimization*) Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be β -smooth and convex. Then for $\eta \leq 1/\beta$, we have

$$f(x_k) \leq f(x_*) + \frac{\|x_0 - x_*\|^2}{2\eta k}$$

where x_* is the vector that achieves the minimum function value $f(x_*)$, i.e., $f(x_*) = \min_{x \in \mathbb{R}^d} f(x)$.

In particular, if we want to get within ϵ of the optimal value, i.e., $|f(x_k) - f(x_*)|$, then the theorem tells us we need to run Gradient Descent for $k \geq \frac{2\beta\|x_0 - x_*\|_2^2}{\epsilon} = O(1/\epsilon)$ iterations.

Gradient descent is one of the *first-order* methods in optimization. By "first-order," we mean that at any time of the algorithm, all you need is a subroutine that calculates the gradient ∇f to run the algorithm. One natural question to ask is: Is Gradient Descent the best first-order method? Are there other first-order methods that outperform Gradient Descent? The answer is yes. We will talk about this accelerated Gradient Descent.

Theorem 3.2. (*Nesterov's Accelerated Gradient Descent [NAGD 1983]*) Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a β -smooth and convex function. Then, Nesterov's Accelerated Gradient Descent has the following guarantee:

$$f(x_k) \leq f(x_*) + \frac{\beta \cdot \|x_0 - x_*\|^2}{k^2}.$$

To compare with the original Gradient Descent, let's set $\beta = 1$ and $\|x_0 - x_*\| = 1$. Then after 100 iterations, by Theorem 3.1, we would have

$$f(x_{100}) \leq f(x_*) + 0.01.$$

On the other hand, however, by Theorem 3.2, Nesterov's Accelerated Gradient Descent can achieve the same error 0.01 after only 10 iterations! In general, as evidenced by Theorem 3.2, Accelerated Gradient Descent has a quadratic speed-up over the original Gradient Descent: to get error ϵ , NAGD needs only $O(\frac{1}{\sqrt{\epsilon}})$ iterations (gradient evaluations).

As the name suggests, the main idea is to use some kind of "acceleration / momentum" in addition to the original gradient descent. Before we delve into the exact NAGD algorithm, let's get some intuition about the idea by looking at a simple quadratic function as an example.

3.1 Quadratic functions and Heavy Ball method

Consider the following quadratic function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ defined as

$$f(x) = x^T A x + \langle b, x \rangle,$$

where $b \in \mathbb{R}^d$ is a vector and A is a $d \times d$ matrix. One can show that when all of A 's eigenvalues are non-negative, f is convex. It might be unclear to one why a function of the above form is actually "quadratic." We will see why this is the case through the following examples.

Example 3.3. 1. The quadratic function $f(x) = x_1^2 + x_2^2$ can be written as

$$f(x) = x_1^2 + x_2^2 = \begin{pmatrix} x_1 & x_2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x^T A x.$$

The eigenvalues of A are 1 and 1, both with multiplicity 2. One can check $f(x)$ is convex.

2. The quadratic function $f(x) = x_1^2 - 0.1x_2^2$ can be written as

$$f(x) = x_1^2 - 0.1x_2^2 = \begin{pmatrix} x_1 & x_2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 0 & -0.1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = x^T A x.$$

The eigenvalues of A are 1 and -0.1, not all non-negative. One can check indeed that f is not convex.

Let's consider the quadratic function $f(x) = x_1^2 + 0.01x_2^2 = x_1^2 + (0.1x_2)^2$. Specifically, $\nabla f(x) = (2x_1, 2 \cdot (0.01)x_2)$. If we start with, say $x_0 = (1, 0)$, and run gradient descent then we have $f(x_0) = 1$ and roughly $f(x_k) \approx \frac{1}{k}$.

As it turns out, we can do better than Gradient Descent via the Heavy Ball Method (Polyak 1964).

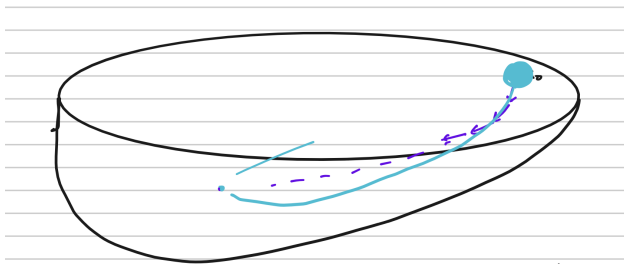


Figure 11: Gradient Descent (purple) vs Heavy Ball (blue) on a quadratic function

If we visualize a convex quadratic function f as a "bowl," then our goal is to start at any point and arrives at the bottom at the bowl as soon as possible (see Figure 11). Borrowing terms from Physics, the Gradient Descent algorithm is essentially making the ball move along the curvatures of the bowl under the influence of gravity with an initial velocity x_0 . This is indeed a good way of achieving our goal but we might be wasting a lot of iterations by spending extra than what is actually needed along the trajectory. It would be better if we go straight down the curvature to the bottom without wandering horizontally along the curvature. See Figure 12.

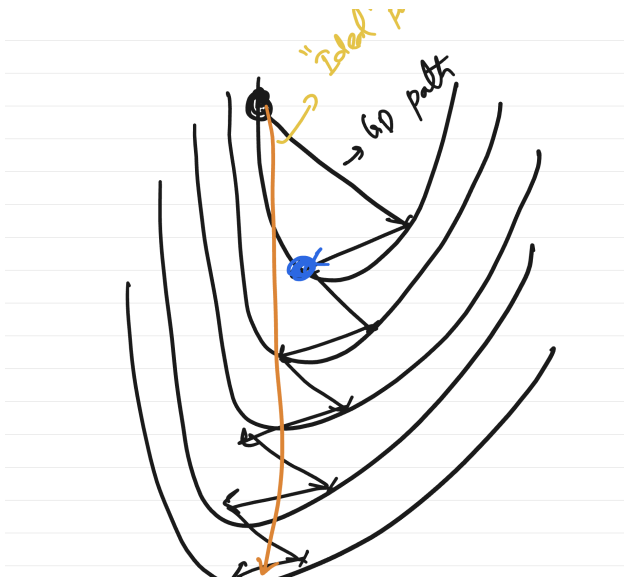


Figure 12: GD path (black) vs the ideal path (orange) on a quadratic function

To get to the "ideal" path, the idea is to use information about past gradients and use momentum. Recall the gradient descent update: $x_{k+1} = x_k - \eta \cdot \nabla f(x_k)$. Polyak's Heavy Ball method (momentum method) instead has the following update:

$$x_{k+1} = x_k - \alpha_{HB} \nabla f(x_k) + \beta_{mom}(x_k - x_{k-1})$$

where the last term $\beta_{mom}(x_k - x_{k-1})$ is the extra momentum term. For functions f that are convex and quadratic, one can show the following improved convergence guarantee.

Theorem 3.4. (Heavy ball / Polyak / Momentum) If $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is quadratic and convex, then for a suitable choice of α_{HB}, β_{mom} , we have

$$f(x_{k+1}) \leq f(x_*) + \frac{O(1)\|x_0 - x_*\|^2}{k^2}.$$

Remark: Although the Heavy Ball method gives an improved guarantee over the original Gradient Descent method, it **fails to** get acceleration for non-quadratic functions even if the function is convex.

3.2 Nesterov's Accelerated Gradient Descent

We have seen an improvement over the gradient descent for quadratic convex functions via the Heavy Ball method. While Heavy ball method does not translate directly to more general convex functions, ideas of using past points/gradient as momentum are extremely useful. Using the similar ideas, Nesterov's Accelerated Gradient Descent (NAGD) outperform the original Gradient Descent over general convex functions.

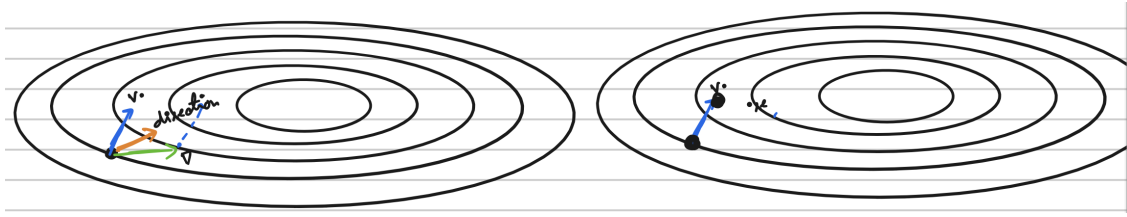


Figure 13: Illustrations of NAGD: on the right is the gradient descent update. On the left is the NAGD update. In blue is the direction of the original GD update. In green is the direction computed using the current point and the previous update. In orange is the actual direction in which an NAGD update is moving.

Nesterov's Accelerated Gradient Descent (NAGD) works in the following way: for $k = 1, 2, \dots, T$:

1. Compute "lookahead point"

$$y_k = x_k + \beta_{NAG}(x_k - x_{k-1}).$$

2. Compute the gradient of f at y_k : $\nabla f(y_k)$.

3. Make the update:

$$x_{k+1} = y_k - \alpha_{NAG} \nabla f(y_k).$$

Putting these together, the NAGD update is given as follows:

$$x_{k+1} = x_k - \alpha_{NAG} \nabla f(x_k + \beta_{NAG}(x_k - x_{k-1})) + \beta_{NAG}(x_k - x_{k-1}).$$

Theorem 3.5. Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be some convex and β -smooth function. Then NAGD with the appropriate $\alpha_{NAGD}(k), \beta_{NAGD}(k)$ satisfies

$$f(x_k) \leq f(x_*) + \frac{\beta \cdot \|x_0 - x_*\|^2}{k^2}.$$

For numerical stability in practice, we need to choose $\alpha_{NAGD}(k), \beta_{NAGD}(k)$ properly. One common choice that works well is $\alpha_{step} = 1/\beta$ where β is the smoothness parameter and $\beta_{NAGD}(k) = \frac{t_{k-1}-1}{t_k}$ where $t_k = \frac{1 + \sqrt{1 + 4t_{k-1}^2}}{2}$.

In practice, one breaks down the update into three steps with extra parameters that one can adjust and runs the following version of NAGD for better stability: for $k = 1, 2, \dots, T$:

1. $x_{k+1} = y_k - \eta \nabla f(y_k)$
2. $z_{k+1} = z_k - \eta_k \nabla f(y_k)$
3. $y_{k+1} = \alpha_i z_k + (1 - \alpha) x_k$.

Theorem 3.6. *Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be some convex and β -smooth function. Then the above Stable NAGD algorithm has the following guarantee:*

$$f(x_k) \leq f(x_*) + \frac{2\beta\|x_0 - x_*\|^2}{k^2}$$

for $\eta \leq 1/\beta$, $\eta_k = (k+1)\eta/3$ and $\alpha_k = 2/(k+3)$.

Here is a brief summary of what we covered so far:

1. Convex functions are good testing grounds for finding efficient algorithms.
2. Gradient Descent gets $O(1/\epsilon)$ convergence rate for convex functions.
3. Polyak's Heavy Ball Method can provide acceleration for "well-conditioned" quadratic functions.
4. Momentum + looking ahead can give us acceleration for general smooth / convex functions as in NAGD.

All of the algorithms we mentioned so far are first-order methods, i.e., they only need to look at the gradient ∇f .

One may ask is NAGD the best we can do? The answer is yes. NAGD gives the best guarantee one can hope for in general among all the first-order methods. There are convex, smooth functions f where you need $1/\sqrt{\epsilon}$ gradient evaluations to get ϵ -error.

Lecture 4. Stochastic Gradient Descent and adaptive methods

4.1 Cost of Gradient Descent

To motivate our discussion of Stochastic Gradient Descent, let's consider a concrete learning problem: Least Squares Regression (LSR): given data $(x^1, y^1), (x^2, y^2), \dots, (x^n, y^n)$, the goal is to find the best linear predictor θ such that the linear function $h_\theta(x) = \langle \theta, x \rangle$ minimizes the L_2 -loss $l(a, b) = (a - b)^2$. More specifically, the empirical risk minimization function can be written as

$$\text{Find the } \theta \text{ that minimizes } L(\theta) = \frac{1}{n} \sum_{i=1}^n (\langle \theta, x^i \rangle - y^i)^2.$$

Using the chain rule, one can calculate the gradient $L(\theta)$ as

$$\nabla L(\theta) = \frac{1}{n} \sum_{i=1}^n 2(\langle \theta, x^i \rangle - y^i) \cdot x^i.$$

One can also write the LSR objective in a matrix form:

$$\begin{aligned} L(\theta) &= \frac{1}{n} \sum_{i=1}^n (\langle \theta, x^i \rangle - y^i)^2 \\ &= \frac{1}{n} \left\| \begin{pmatrix} - & - & - & - & x^1 & - & - & - & - \\ - & - & - & - & x^2 & - & - & - & - \\ & & & & \cdot & & & & \\ & & & & \cdot & & & & \\ & & & & \cdot & & & & \\ - & - & - & - & x^n & - & - & - & - \end{pmatrix} \theta - \begin{pmatrix} y_1 \\ y_2 \\ \cdot \\ \cdot \\ \cdot \\ y_n \end{pmatrix} \right\|^2 \\ &= \frac{1}{n} \|X\theta - y\|^2. \end{aligned}$$

Similarly, for the gradient we have

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n 2(\langle \theta, x^i \rangle - y^i) \cdot x^i = \frac{2}{n} X^T (X\theta - y).$$

To evaluate each inner product $\langle \theta, x^i \rangle$, $O(d)$ time (operations) are needed. Since we need to evaluate n such inner products, each iteration of Gradient Descent will cost us $O(nd)$ time (operations).

Now for a general Empirical Risk Minimization problem of the form

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n l(h_\theta(x^i), y^i)$$

with gradient

$$\nabla L(\theta) = \frac{1}{n} \sum_{i=1}^n l'(h_\theta(x^i), y^i) \cdot \nabla_\theta h_\theta(x^i),$$

if we have d parameters, i.e., $\theta \in \mathbb{R}^d$, then each gradient computation needs time $O(nd)$. Suppose we have a database of a million images, i.e., $n = 10^6$ and each image contains $d = 250 * 250$ points. The cost per iteration can thus be as high as $10^6 * 250 * 250$ operations!

As a result, one issue of the original gradient descent is that the cost per iteration can be too expensive when you have a large data set. In such scenarios we would like an algorithm that is "zeroth-order" in a sense that it doesn't need to evaluate the gradient of the loss function at all data points.

If there's way to do this, then this can reduce the cost per iteration. However, this does not come for free. We should expect the algorithm to take many more iterations to converge. So there's really a tradeoff between numbers of iterations and cost per iteration.

4.2 Stochastic Gradient Descent

To elaborate on this idea, suppose $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is the function that we want to minimize. The original gradient descent would need us to compute ∇f at all points. Now, instead of computing ∇f exactly, we have an "estimator" g_x for ∇f that is easy to compute and satisfies:

$$\nabla f(x) = \mathbf{E}[g_x].$$

We would then use g_x instead of ∇f when running our gradient descent algorithm since g_x is much cheaper to evaluate than $\nabla f(x)$. This is the idea of *Stochastic Gradient Descent*. For $i = 1, 2, \dots$, Stochastic Gradient Descent has the following update:

$$x_i = x_{i-1} - \eta g_{x_{i-1}},$$

where $\eta > 0$ is the step-size.

Before analyzing the algorithm, since g_x is a random variable, let's review some concepts from probability theory. As one is probably familiar with, random variables that are very different may have the same expected value.

Example 4.1. Let Y, Z be scalar-valued random variables such that

$$Y = \begin{cases} 2 & \text{with probability } \frac{1}{2} \\ 0 & \text{with probability } \frac{1}{2} \end{cases}$$

and

$$Z = \begin{cases} 1000 & \text{with probability } \frac{1}{1000} \\ 0 & \text{with probability } \frac{999}{1000}. \end{cases}$$

Then it is easy to check $\mathbf{E}[Y] = \mathbf{E}[Z]$ although Y, Z are very different random variables.

Hence, expected value by itself can be deceiving. To get a better understanding of random variables, recall the notion of variance that measures on average, how far a random variable is from its expected value:

$$\mathbf{Var}(Y) = \mathbf{E}[(Y - \mathbf{E}[Y])^2] = \mathbf{E}[Y^2] - \mathbf{E}[Y]^2.$$

Similarly, for vector-valued random variable X , we have

$$\mathbf{Var}(X) = \mathbf{E}[\|X - \mathbf{E}[X]\|^2].$$

With this notion in mind, now let's revisit the Empirical Risk Minimization (ERM) problem. Recall in ERM, we want to minimize

$$L(\theta) = \frac{1}{n} \sum_{i=1}^n l(h_\theta(x^i), y^i)$$

with gradient

$$\nabla L(\theta) = \frac{1}{n} \sum_{i=1}^n l'(h_\theta(x^i), y^i) \cdot \nabla h_\theta(x^i).$$

As mentioned above, evaluating this can be very costly. So what we can do instead is to use an estimator of this gradient. Consider the following *Stochastic Gradient Descent* estimator for the gradient:

- Choose a data point (x_i, y_i) uniformly at random from $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$.
- Evaluate the gradient at (x_i, y_i) :

$$g_\theta = l'(h_\theta(x^i), y^i) \nabla h_\theta(x^i).$$

In words, we are picking a random data point and evaluate the gradient only on that randomly picked point. The cost of computing this estimator is only $O(d)$! Note this estimator indeed satisfies the condition $\mathbf{E}[g_\theta] = \nabla L(\theta)$:

$$\mathbf{E}[g_\theta] = \sum_{i=1}^n \frac{1}{n} l'(h_\theta(x^i), y^i) \cdot \nabla h_\theta(x^i) = \nabla L(\theta).$$

On the other hand, instead of evaluating the gradient on only one randomly picked point, we can evaluate on a few randomly selected points. This estimator is called a *Mini-Batch Estimator*:

- Pick k random indices $i_1, i_2, \dots, i_k \in \{1, \dots, n\}$
- Evaluate the gradient at these points:

$$g_\theta = \frac{1}{k} \sum_{j=1}^k l'(h_\theta(x^{i_j}), y^{i_j}) \cdot \nabla h_\theta(x^{i_j}).$$

The cost of computing the mini-batch estimator is $O(kd)$.

Remark: Instead of sampling a random point in each iteration, we can order all of the points in a random order and go sequentially through this order. For instance, we can have the data points being randomly ordered as

$$(x^{100}, y^{100}), (x^{17}, y^{17}), (x^{51}, y^{51}), \dots$$

One pass of the entire ordered data points is called an epoch.

The following theorem tells us the convergence guarantees of Stochastic Gradient Descent.

Theorem 4.2. (Convergence of Stochastic Gradient Descent) Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be β -smooth and convex. If we have an estimator g for the gradient ∇f with variance $\leq \sigma^2$. Then for $\eta \leq 1/\beta$, running Stochastic Gradient Descent on f with estimator g satisfies:

$$\mathbf{E}[f(\bar{x})] \leq f(x_*) + \frac{\|x_* - x_0\|^2}{2\eta k} + \frac{\eta\sigma^2}{2},$$

where $\bar{x}$ is either $\frac{x_1 + x_2 + \dots + x_k}{k}$ or $\arg \min_i f(x_i)$.

We will prove this theorem shortly. Assuming the theorem, by setting $\eta = \frac{\|x_0 - x_*\|}{\sqrt{k} \cdot \sigma}$ (this choice of η minimizes the term $\frac{\|x_0 - x_*\|^2}{2\eta k} + \frac{\eta\sigma^2}{2}$), we obtain the following corollary.

Corollary 4.3. We can find a $\bar{x}$ such that

$$\mathbf{E}[f(\bar{x})] \leq f(x_*) + \frac{\|x_0 - x_*\|^2 \cdot \sigma}{2\sqrt{k}}.$$

Now let's compare SGD with GD and NAGD. If $f : \mathbb{R}^d \rightarrow \mathbb{R}$ is β -smooth and convex, then to get within error ϵ :

- Stochastic Gradient Descent needs $O(\frac{1}{\epsilon^2})$ iterations. If $\epsilon = 0.01$, then SGD needs ≈ 10000 iterations.
- Gradient Descent needs $O(\frac{1}{\epsilon})$ iterations. If $\epsilon = 0.01$, then GD needs ≈ 100 iterations.
- Nesterov's Accelerated Gradient Descent needs $O(\frac{1}{\sqrt{\epsilon}})$ iterations. If $\epsilon = 0.01$, then NAGD needs ≈ 10 iterations.

Remark: there exists functions where $1/\epsilon^2$ iterations is the best we can do if we only have access to the estimators.

4.3 Convergence Analysis for SGD

We now prove Theorem 4.2. Just like the proof for the convergence of Gradient Descent, we start with the following claim about the monotonicity of SGD.

Claim 4.4. (Approximate monotonicity of SGD) Let $f : \mathbb{R}^d \rightarrow \mathbb{R}$ be a function that is β -smooth and convex. For $\eta \leq \frac{1}{\beta}$, SGD updates satisfies:

$$\mathbf{E}[f(x_{i+1})] \leq f(x_i) - \frac{\eta}{2} \|\nabla f(x_i)\|^2 + \frac{\eta\sigma^2}{2}.$$

Proof. By Lemma 2.7 (the smoothness upper bound), we know

$$f(x_{i+1}) \leq f(x_i) + \langle \nabla f(x_i), x_{i+1} - x_i \rangle + \frac{\beta}{2} \|x_{i+1} - x_i\|^2.$$

Plugging in the SGD update $x_{i+1} = x_i - \eta g$ into the inequality above, we have

$$f(x_{i+1}) = f(x_i) + \eta \langle \nabla f(x_i), -\eta g \rangle + \frac{\beta}{2} \|\eta \cdot g\|^2.$$

Taking the expectation over g on both sides, we get

$$\begin{aligned} \mathbf{E}[f(x_{i+1})] &= f(x_i) - \eta \langle \nabla f(x_i), \mathbf{E}[g] \rangle + \frac{\beta \eta^2}{2} \cdot \mathbf{E}[\|g\|^2] \\ &= f(x_i) - \eta \langle \nabla f(x_i), \nabla f(x_i) \rangle + \frac{\beta \eta^2}{2} \mathbf{E}[\|g\|^2], \end{aligned} \tag{12}$$

where we used the fact $\mathbf{E}[g_x] = \nabla f(x)$. Now recall that for a scalar-valued random variable Z , we have $\mathbf{Var}(Z) = \mathbf{E}[Z^2] - (\mathbf{E}[Z])^2$. Similarly, for a vector-valued random variable Y , we have

$$\mathbf{Var}(Y) = \mathbf{E}[\|Y\|^2] - \|\mathbf{E}[Y]\|^2.$$

As a result, we have

$$\mathbf{E}[\|g\|^2] = \mathbf{Var}(g) + \|\mathbf{E}[g]\|^2 \leq \sigma^2 + \|\nabla f(x_i)\|^2.$$

Plugging this into (12), we know

$$\begin{aligned} \mathbf{E}[f(x_{i+1})] &\leq f(x_i) - \eta \|\nabla f(x_i)\|^2 + \frac{\beta \eta^2}{2} (\|f(x_i)\|^2 + \sigma^2) \\ &\leq f(x_i) - \frac{\eta}{2} \|\nabla f(x_i)\|^2 + \frac{\eta}{2} \sigma^2 \end{aligned}$$

where the last line holds since $\eta \leq \frac{1}{\beta}$. □

Now we are ready to prove Theorem 4.2.

Proof. (of Theorem 4.2) Using exactly the same idea as in the proof of Gradient Descent convergence, we can obtain the following statement:

$$\mathbf{E}[f(x_{i+1})] - f(x_*) \leq \frac{1}{2\eta} \mathbf{E}[\|x_i - x_*\|^2 - \|x_{i+1} - x_*\|^2] + \frac{\eta \sigma^2}{2}.$$

It is a good exercise for you to try to arrive at the statement above following the same idea in the proof of Theorem 2.12 using Claim 4.4. Now since the above inequality holds for all i , we have

$$\begin{aligned} \mathbf{E}[f(x_1)] - f(x_*) &\leq \frac{1}{2\eta} \mathbf{E}[\|x_0 - x_*\|^2 - \|x_1 - x_*\|^2] + \frac{\eta \sigma^2}{2}. \\ \mathbf{E}[f(x_2)] - f(x_*) &\leq \frac{1}{2\eta} \mathbf{E}[\|x_1 - x_*\|^2 - \|x_2 - x_*\|^2] + \frac{\eta \sigma^2}{2}. \\ &\dots \\ \mathbf{E}[f(x_{k+1})] - f(x_*) &\leq \frac{1}{2\eta} \mathbf{E}[\|x_k - x_*\|^2 - \|x_{k+1} - x_*\|^2] + \frac{\eta \sigma^2}{2}. \end{aligned}$$

Adding these inequalities up, we get

$$\mathbf{E}[f(x_1) + f(x_2) + \dots + f(x_{k+1})] - (k+1)f(x_*) \leq \frac{1}{2\eta} \mathbf{E}[\|x_0 - x_*\|^2 - \|x_{k+1} - x_*\|^2] + \frac{(k+1)\eta \sigma^2}{2}$$

Dividing both sides by $(k+1)$ and using the fact $\|x_{k+1} - x_*\|^2 \geq 0$, we have

$$\mathbf{E} \left[\frac{f(x_1) + f(x_2) + \dots + f(x_{k+1})}{k+1} \right] \leq f(x_*) + \frac{\|x_0 - x_*\|^2}{2\eta(k+1)} + \frac{\eta\sigma^2}{2}.$$

Lastly, since f is a convex function, we have the following:

$$f\left(\frac{x_1 + \dots + x_{k+1}}{k+1}\right) \leq \frac{f(x_1) + \dots + f(x_{k+1})}{k+1}.$$

This can be proved using (2) in Definition 2.9.

As a result, we have thus shown

$$\mathbf{E}[f(\bar{x})] \leq f(x_*) + \frac{\|x_0 - x_*\|^2}{2\eta(k+1)} + \frac{\eta\sigma^2}{2}$$

for $\bar{x} = \frac{x_1 + \dots + x_{k+1}}{k+1}$ as desired. Since $\min_i f(x_i) \leq f(\bar{x})$, the statement still hold by replacing $\bar{x}$ with $\arg \min_i f(x_i)$. $\square$

4.4 Adaptive step-sizes

At this point, one may wonder: can we accelerate SGD just as we did for GD? Unfortunately, in general, the answer is no. We can have speed-up in some special cases by using the idea of momentum again. For instance, one can use the following stochastic version of Heavy Ball Method:

$$x_{i+1} = x_i - \eta g + \beta \cdot (x_i - x_{i-1}).$$

This can give us some speed-up for convex quadratic functions.

So far we have been running our algorithms with a fixed step-size η . One thing we can do is to adapt "step-size" or "learning rate" for each parameters separately. This is known as ADAGRAD, i.e., "adaptive GD."

AdaGrad

- Start at some initialization point x_0 ; Set $G_0 = 0$; Let α be some small constant, say $\alpha = 10^{-3}$.
- For each step i :
 - let g be the estimate for $\nabla f(x_{i-1})$.
 - For each coordinate j :
 - * Set $(G_i)_j = (G_{i-1})_j + (g_j)^2$
 - * Set $(X_i)_j = (X_{i-1})_j - \frac{\eta}{\sqrt{(G_i)_j + \alpha}} g_j$.

The key idea of these adaptively chosen step-sizes is that we can calculate parameter-wise learning rates based on the sum of squares of all past gradients.

Lecture 5. More on Adaptive Methods and Convex Optimization

5.1 AdaGrad and variants

Last time we talked about adaptive step sizes for first-order methods. Today we will continue this discussion. Recall the AdaGrad algorithm from last time.

AdaGrad

- Start at some initialization point x_0 ; Set $G_0 = 0$; Let α be some small constant, say $\alpha = 10^{-3}$.
- For each step i :
 - let g be the estimate for $\nabla f(x_{i-1})$.
 - For each coordinate j :
 - * Set $(G_i)_j = (G_{i-1})_j + (g_j)^2$
 - * Set $(X_i)_j = (X_{i-1})_j - \frac{\eta}{\sqrt{(G_i)_j + \alpha}} g_j$.

Intuitively, AdaGrad keeps track of how each coordinate from one iteration to another and adjust the step sizes accordingly. However, one issue with this algorithm is that over time, the step-sizes can become very small as we are weighing the effects of all past updates equally. One fix to this problem is to weigh the effects of earlier iteration smaller than the later iterations. This bring us to the following version of AdaGrad with exponentially weighted averages.

AdaGrad (with exponentially weighted averages)

- Start at some initialization point x_0 ; Set $G_0 = 0$; Let α be some small constant, say $\alpha = 10^{-3}$.
- Choose some $\delta > 0$, say $\delta = 0.9$
- For each step i :
 - let g be the estimate for $\nabla f(x_{i-1})$.
 - For each coordinate j :
 - * Set $(G_i)_j = \delta(G_{i-1})_j + (1 - \delta)(g_j)^2$
 - * Set $(X_i)_j = (X_{i-1})_j - \frac{\eta}{\sqrt{(G_i)_j + \alpha}} g_j$.

Notice that this exponential weighing scheme allows us to have exponentially smaller weights on the first iterations than the later iterations and therefore prevent the step-sizes to be too small in later iterations.

5.2 ADAM

Now that we have "upgraded" gradient descent with adaptive step-sizes, we can do the same thing with accelerated versions of gradient descents. Recall that we can accelerate gradient descent by adding a "momentum" term and obtain *the Heavy Ball* method with update:

$$x_{k+1} = x_k - \alpha_{HB} \nabla f(x_k) + \beta_{HB} m_k, \quad (13)$$

where $m_k := x_k - x_{k-1}$ is the momentum term. We can rewrite this update in the following manner:

1. Start with $m_0 = 0$.
2. For $k = 0, 1, \dots, T$:
 - Update momentum: $m_k = \beta m_{k-1} + g_k$, where g_k is the gradient estimate.
 - Set $x_{k+1} = x_k - \eta m_k$.

To check this is indeed equivalent to the Heavy-Ball update, note

$$x_{k+1} = x_k - \eta m_k = x_k - \eta(\beta m_{k-1} + g_k) = x_k - \eta(\beta(x_{k-1} - x_k)/\eta + g_k),$$

which has exactly the same form as (13) after rearranging and renaming parameters. Adding adaptive step-sizes to this version of Heavy Ball method, we obtain ADAM, a widely used optimization algorithm in practice.

ADAM (without bias correction)

1. Set $m_0 = 0$ and $G_0 = 0$. Let $\beta_1, \beta_2, \eta, \epsilon > 0$.
2. For $k = 1, 2, \dots$:
 - Let g be the estimate for $\nabla f(x_k)$.
 - Set $m_k = \beta_1 m_{k-1} + (1 - \beta_1)g_k$
 - For each coordinate j :
 - Set $(G_k)_j = \beta_2(G_{k-1})_j + (1 - \beta_2)(g_k)_j^2$
 - Set $(x_{k+1})_j = (x_k)_j - \frac{\eta}{\sqrt{(G_k)_j + \epsilon}}(m_k)_j$.

One can also add bias correction to make the starting iterations more robust. The overall performance with or without the bias correction are roughly the same.

ADAM (with bias correction)

1. Set $m_0 = 0$ and $G_0 = 0$. Let $\beta_1, \beta_2, \eta, \epsilon > 0$.
2. For $k = 1, 2, \dots$:
 - Let g be the estimate for $\nabla f(x_k)$.
 - Set $m_k = \beta_1 m_{k-1} + (1 - \beta_1)g_k$
 - For each coordinate j :
 - Set $(G_k)_j = \beta_2(G_{k-1})_j + (1 - \beta_2)(g_k)_j^2$
 - Set $(x_{k+1})_j = (x_k)_j - \frac{\eta}{\sqrt{(\hat{G}_k)_j + \epsilon}}(\hat{m}_k)_j$.

where $\hat{m}_k = \frac{m_k}{1 - \beta_1^k}$ and $\hat{G}_k = \frac{G_k}{1 - \beta_2^k}$.

So far we have been talking about adaptive step sizes for the individual coordinate. This works really well when the coordinates are not correlated. For instance, consider the following function:

$$f(x) = 0.1x^2 + 10x_2^2.$$

Because f depends on x_1, x_2 at different rate (different partial derivatives), intuitively it is a good idea to make the update to the first coordinate take larger steps than the update to the second coordinate, which is what we have done in AdaGrad. However, if there are some correlations, then we need to be a little more cautious. Consider the following function:

$$g(x) = 0.1(x_1 - x_2)^2 + 10(x_1 + x_2)^2.$$

If you compute the partial derivatives with respect to x_1, x_2 , then you would notice they are pretty similar in magnitude and thus adjust step-sizes based on coordinates does not make much sense. But if you perform a change of variable and let $y_1 = x_1 - x_2$ and $y_2 = x_1 + x_2$, then the derivative of g with respect to y_1, y_2 will again have different magnitude and adaptive step sizes for each coordinates individually may be beneficial. The version of ADAM and AdaGrad we studied above cannot really do this change of basis, but we can enforce a change-of-basis via the following algorithm, which is an approximate version of Newton's iteration (do not worry if you are not familiar with term).

SHAMPOO

1. Initialize x_0
2. For $k = 0, 1, \dots$,
 - Let g_k be an estimator for gradient $\nabla f(x_k)$.
 - $g_{\text{precond},k} = L_k^{-1} g_k R_k^{-1}$
 - $x_{k+1} = x_k - \eta g_{\text{precond},k}$

where L_k, R_k are left / right preconditioners respectively. There are heuristics / techniques one can use to update L_k, R_k . As an example, notice that if we set $L_k = \text{diag}(\frac{1}{\sqrt{(G_k)_1 + \alpha}}, \frac{1}{\sqrt{(G_k)_2 + \alpha}}, \dots, \frac{1}{\sqrt{(G_k)_n + \alpha}})$, the matrix with the only non-zero elements being the diagonal and set $R_k^{-1} = I$, then the above procedure is equivalent to Adagrad.

By now we have talked about various algorithms for the problem of unconstrained optimization. In summary, we studied

1. Gradient Descent (GD) as a "meta" algorithm that can be adapted / adjusted to accelerate and accommodate different problem formulations
2. Stochastic Gradient Descent (SGD) which uses estimators for gradients to reduce the computational cost per iteration.
3. Momentum as a way to accelerate Gradient Descent by using history of gradients.
4. Adaptivity as a method to adjust the learning rates to use for each parameters.
5. Preconditioning the gradients to perform a "change of variable."

Using these techniques, we can obtain the optimal algorithm among all first-order methods. It is worth noticing that the gradient-based algorithms we discussed up to this point are based on the idea of "steepest descent" with respect to the normal Euclidean distance.

However, we don't have to be restricted to this choice of distance. Sometimes it is better if we alternate the "gradient step" in spaces with different notions of distance. This is the idea behind *Mirror Descent*.

There are also some other heuristics such as gradient-dipping, renormalization that one can use to obtain some speed up. We will not be discussing these in this class but you can look it up if interested.

5.3 Extension to Constrained Optimization

So far we have been talking about unconstrained optimization where we do not place any constraints on the solution. In many problems, we do have some conditions or desired properties of the solutions. For instance, we might want the solutions to have only positive coordinates. We might want the solutions to have a small length (norm). It is natural to ask whether we can extend the algorithms we discussed earlier to this constrained version of optimization problems.

Mathematically speaking, we would like our solution to be contained in some nice set C . One common kind of "nice" set people work with is a convex sets.

Definition 5.1. A set $C \subseteq \mathbb{R}^d$ is convex if for all $x_1, x_2 \in C$, we have $\frac{x_1 + x_2}{2} \in C$.

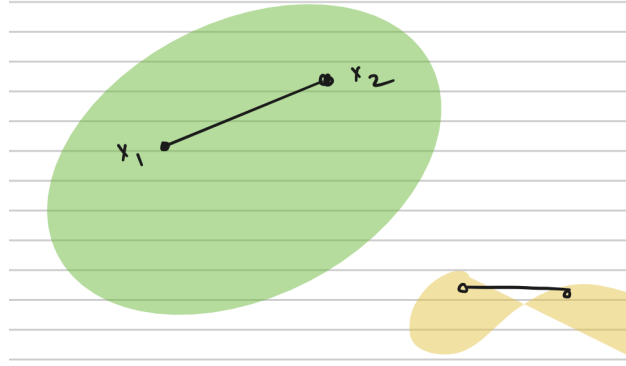


Figure 14: The green set is convex; the yellow set is not convex

See Figure 14 for an example. Given this notion, we can phrase our constrained optimization problem as the following:

$$\min f(x) \quad \text{s.t. } x \in C,$$

where C is a convex set. How do we solve this problem? It turns out we can just use gradient descent by projecting the updates into the set C . This is the Projected Gradient Descent algorithm.

Projected Gradient Descent

1. Initialize x_0
2. For $k = 1, 2, \dots$:
 - Set $y_{k+1} = x_k - \eta \nabla f(x_k)$
 - Set $x_{k+1} = \arg \min_{x \in C} \|x - y_{k+1}\|$.

Notice that in the last step, we are setting x_{k+1} to be the point x in C that is the closest to y_{k+1} . This is the "projection" of y onto C .

It turns out that this Projected Gradient Descent works really well as long as C is a convex set. We can also run the stochastic / accelerated versions of projected gradient. In particular, we can recover all guarantees for GD, SGD, NAGD for their respective projected versions as long as C is convex!

The optimization problem in which both the objective f and the constrained set C are convex. Many common formulations such as Linear Programming, Semi-definite Programming all fall in this category.

This wraps up the Optimization part of the class. Next time we will be talking about Online Learning.

Part II

Online Learning

Lecture 6. Online Learning and Perceptron

In this lecture, we start our discussion of a new topic: online learning. In the previous lectures, we talked about how to rewrite a supervised learning problem as an Empirical Risk Minimization (ERM) problem. One nice thing about that setup is that we have the storage and access to all the training data at once and learn the model accordingly. However, in many cases, we do not get to see all of the data at once and we are required to make a decision based on the limited data we have in the past. This happens in, for instance:

- Stock Market Predictions;
- Recommending system;
- Online games (pokers);
- Control system / autonomous cars and many others.

Now let's model online learning. Suppose for a particular task, on day 1 you receive the data (x^1, y^1) where x^1 belongs to the domain $\mathcal{X}$ and y^1 is the associated label. Based on these data, you make a prediction $\hat{y}^1$. Similarly, on day 2, you receive the data (x^2, y^2) and make a prediction $\hat{y}^2$ and repeat...

To compare between different models, we can use a loss function $l : \mathcal{Y} \times \mathcal{Y} \rightarrow \mathbb{R}$, so that the loss on first day is $l(y^1, \hat{y}^1)$, on day two is $l(y^2, \hat{y}^2)$ and so on. One natural thing to try is to minimize the weighted sum of losses across time. We will cover this a bit but as we will see later, in some cases it is hard to model a problem this way.

Another way to model this is to say that one can be arbitrarily wrong in the first iterations but after a certain point, my prediction will be pretty accurate. In other words, suppose the ground truth is $y = f_*(x)$ where $f_* \in \mathcal{H}$, how many days will we have till we stop making any mistakes?

This has some potential issues: first, in many cases we would like to avoid bad mistakes even in the first steps; second, we would like our learning algorithm to be good regardless of specific inputs. For instance, suppose at day 1 we obtain as data (x^1, y^1) . Consider the algorithm that outputs y_1 for all future days. If the data one in the following days are all (x_1, y_1) , then this naive constant algorithm never makes mistakes. However, we know this is not desired since we need our algorithm to be "robust" to the input distribution instead of achieving small error some particular choice of data streams.

Hence, we will first discuss Online Learning in the *Mistake Bound model*. Let $\mathcal{H}$ be the true hypothesis class. Let $f_* \in \mathcal{H}$ be the ground-truth predictor. We see samples / inputs of the form $(x^1, y^1), (x^2, y^2), \dots$, where $y^i = f_*(x^i)$. Our goal is to design an online algorithm, i.e., making decisions as we receive new data that will only a fixed (and small) number of mistakes no matter how long you play. We will discuss the celebrated Perceptron algorithm in this framework.

6.1 Online Learning of Halfspaces

Consider the following problem of learning halfspaces: let $\mathcal{H}$ be the set of halfspaces and $x \in \mathbb{R}^d$. Recall a subspace H is defined by a single vector $w \in \mathbb{R}^d$ and $H = \{x : \langle w, x \rangle \geq 0\}$. Let the true function be

$f_*(x) = \begin{cases} 1 & \text{if } \langle w_*, x \rangle \geq 0 \\ -1 & \text{else} \end{cases}$. On day i , we receive a new example of the form (x^i, y^i) where $y^i = f_*(x^i)$.

Our goal is to make predictions $\hat{y}^i$ for each day such that the total number of mistakes made is bounded by a small number.

To this end, we need a notion of "margin."

Definition 6.1. (Margin) Let $\mathcal{D}$ be a distribution over $\mathbb{R}^d$ and $H_* = \{x \in \mathbb{R}^d : \langle w_*, x \rangle \geq 0\}$. Define the margin of H_* with respect to $\mathcal{D}$ as

$$\delta = \min_{x \in \mathcal{D}} \frac{|\langle w_*, x \rangle|}{\|w_*\|_2 \cdot \|x\|_2}.$$

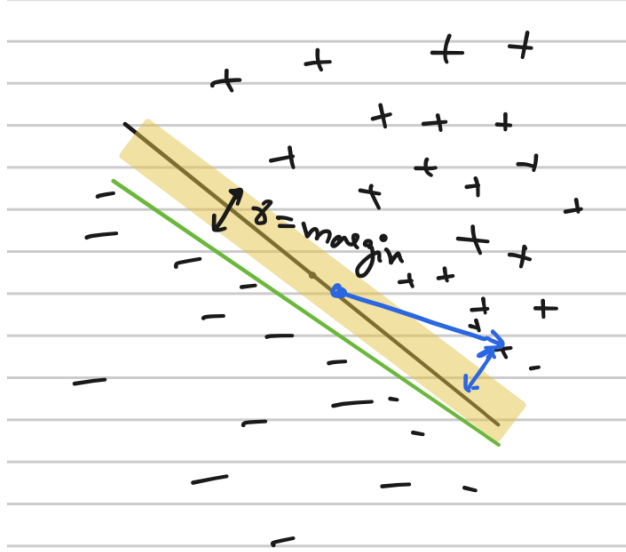


Figure 15: Margin of a halfspace

For simplicity, from this point onward, we will assume that $\|x\|_2 = 1$ and $\|w_*\|_2 = 1$.

The following algorithm by Rosenblatt 1958, solves the problem with only a small bounded error.

Perceptron Algorithm (1958, Rosenblatt)

- Let w^0 be a randomly chosen starting point
- For $i = 1, 2, \dots$:
 - Set

$$w^i = \begin{cases} w^{i-1} & \text{if no mistake is made on day } i \\ w^{i-1} + y^i x^i & \text{if a mistake is made on day } i \end{cases}$$

In other words, if on day i , the true label y^i is negative but the prediction $\langle w^i, x^i \rangle$ is positive, then

$$w^i = w^{i-1} - x^i.$$

Similarly, if the true label is positive but the prediction is negative, then

$$w^i = w^{i-1} + x^i.$$

Here we implicitly assume $\langle w^0, w_* \rangle \geq 0$. This is not a problem since for a randomly chosen w^0 with norm 1, this happens with probability $\geq 1/2$. On the other hand, by Claim 6.3, we have

$$\|w^N\|^2 \leq \|w^0\|^2 + M = M + 1. \quad (16)$$

Recall the Cauchy-Schwartz inequality: for $u, v \in \mathbb{R}^d$, we always have $|\langle u, v \rangle| \leq \|u\| \cdot \|v\|$. Combining (15) (16) with Cauchy-Schwartz, we obtain

$$\delta M \leq \langle w^N, w_* \rangle \leq \|w^N\| \cdot \|w_*\| \leq \sqrt{M+1}$$

where we again use the assumption $\|w^*\| = 1$. We thus have $\delta M \leq \sqrt{M+1}$ which is equivalent to

$$M \leq \frac{1}{\delta^2} + 1$$

as desired. $\square$

We have thus shown that the Perceptron algorithm only makes a small number of mistakes. It turns out if we take an alternative view of the algorithm, it is actually a Stochastic Gradient Descent algorithm in disguise. Recall the Perceptron Algorithm:

Perceptron Algorithm

- Let w^0 be a randomly chosen starting point
- For $i = 1, 2, \dots$:
 - Set

$$w^i = \begin{cases} w^{i-1} & \text{if no mistake} \\ w^{i-1} + y^i x^i & \text{if mistake} \end{cases}$$

We can set up an Empirical Risk Minimization(ERM) problem for learning halfspaces with an appropriate choice of $\mathcal{H}$ and loss functions. In this case, let $\mathcal{H}$ be the set of linear functions $h_w(x) = \langle w, x \rangle$. Choose the loss function to be the Hinge loss $l(a, b) = \max(0, -a \cdot b)$.

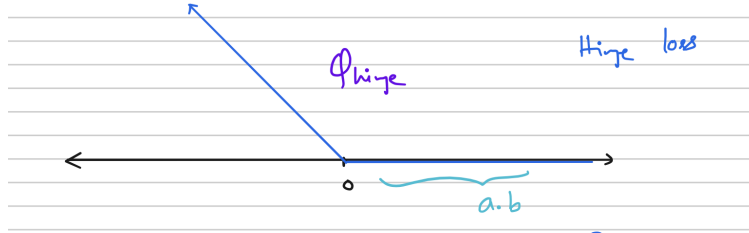


Figure 17: Graph of hinge loss, the x -axis denotes the value of the product ab .

Note one may find that for some technical reasons the hinge loss used in practice to be $l(a, b) = \max(0, 1 - a \cdot b)$ which is just the shift of the entire graph to the right by 1. Now we can set up the EMR objective as

$$\begin{aligned} L(w) &= \frac{1}{n} \sum_{i=1}^n l(h_w(x^i), y^i) \\ &= \frac{1}{n} \sum_{i=1}^n \psi_{hinge}(h_w(x^i) \cdot y^i). \end{aligned}$$

The gradient of ψ_{hinge} is given by

$$\nabla_w \psi_{hinge}(h_w(x^i), y^i) = \begin{cases} 0 & \text{if } \text{sgn}(h_w(x^i)) = y^i \\ -y^i \cdot x^i & \text{else.} \end{cases}$$

Notice this is exactly the update of the Perceptron Algorithm. Hence, the Perceptron algorithm is really Stochastic Gradient Descent with appropriately chosen loss functions and predictor class.

6.2 Regret Minimization and Learning with Experts

So far we have been discussing online learning in the mistake bound model. This is a good model for cases in which the decision is binary or categorical. However, in many scenarios, we are posed with a more complicated problem and the predictions we need to make are numerical. In this case, it does not make too much sense to keep thinking in terms the mistake bound model. In other scenarios, a mistake bound is not really realistic. Regardless of the strategy we deploy, we will always be making mistakes. Consequently, we will think of our problems from the perspective of Regret Minimization. To start, let's consider the problem of Learning with Experts

Let's say we want to predict whether the stock market will go up or down for the next couple of days. We have a set of d experts $E_1, \dots, E_d$ that we can consult. On each day, they will make their predictions and we get to make our final prediction of the day after consulting their opinions. On each day, an expert may be right or wrong and thus, each of them incur a loss.

Day	E_1	E_2	E_3	...	E_{d-1}	E_d	Your prediction	
Day1	Up 1	Down 0	Up 1	- - -	Up 1	Up 1	Up 1	Down
Day2	Down 0	Up 1	Down 0	- - -			Down 0	Down
...								
...								
$\downarrow$ Loss of E_1 $\downarrow$ Loss of E_2 $\downarrow$ Our loss								

Figure 18: Example of Learning with experts. On each day, every expert makes their prediction and we make our prediction based on their advices. The last column denote the true label. On day 1, expert E_1 incurs a loss of 1 since their prediction is different from the true label.

Let $l(t, i)$ denote the loss incurred on day t by expert i . Let $l(t)$ denote the loss incurred by our prediction on day t . Our goal is to, in hindsight, match the performance of the best expert. In other words, our goal is to minimize the regret at time T , defined as:

$$Regret(T) = \sum_{t=1}^T l(t) - \min_{i=1, \dots, d} \sum_{t=1}^T l(t, i),$$

where the term $\min_{i=1, \dots, d} \sum_{t=1}^T l(t, i)$ is the loss incurred by the best expert over that period.

6.2.1 Follow the Leader (FTL)

Let's by considering the simplest strategy: follow the leader (FTL). In this case, on each day i , our prediction will be the prediction of whoever has the best prediction record up to day i . If multiple experts have the same record up to day i , we randomly select one of them and follow their prediction. However, this strategy can be horribly wrong and we may never get a single right prediction! (for instance, whenever we pick some expert, the expert immediately makes a mistake on that day).

Suppose we have the extra assumption of an "infallible expert," i.e., there is an expert who never makes a single mistake. Then if we use the FTL strategy, it may takes us up to $d - 1$ mistakes to locate this expert. Therefore, even with an infallible expert we may still have a regret of $d - 1$ for FTL.

6.2.2 Follow the Majority (FTM)

Now instead of making our prediction based on a single expert, let's follow the majority vote of the experts. In each round, we keep a set of eligible experts and we predict based on the majority.

Follow the Majority (FTM)

- Let $A[0] = \{1, 2, \dots, d\}$
- On day $t = 1, 2, \dots, T$:
 - Predict according to the majority of the experts in $A[t-1]$.
 - Throw away any experts who made a mistake.

We have the following theorem.

Theorem 6.4. (*Regret of FTM with an infallible expert*) *If there is an infallible expert,*

$$\text{Regret}_{FTM}(T) \leq \log_2 d.$$

The punchline is that whenever we make a mistake, we are throwing out at least a half of the remaining experts. Note that by taking the majority, FTM gives us an exponentially better upper bound on the number of mistakes made.

Proof. Let $\phi(t) := |A[t]|$ denote the number of experts still in consideration after day t . Then, $\phi(0) = d$. If we (the final output) make a mistake on day t , then it means the majority of the remaining experts are wrong and hence,

$$\phi(t) \leq \frac{\phi(t-1)}{2}.$$

Therefore, we have

$$\phi(t) \leq \left(\frac{1}{2}\right)^{L(t)} \cdot \phi(0)$$

where $L(t)$ is the number of mistakes our algorithm (FTM) makes by day t . On the other hand, since by assumption, there exists an infallible expert, we thus have $\phi(t) \geq 1$ for all t . Combining the above, we thus get

$$1 \leq \phi(t) \leq \left(\frac{1}{2}\right)^{L(t)} \cdot \phi(0) = \left(\frac{1}{2}\right)^{L(t)} d,$$

which implies $L(t) \leq \log_2 d$. □

6.2.3 Follow the Weighted Majority (FTWM)

As shown above, FTM works fairly well assuming there exists an infallible expert. However, this is a very strong assumption as we generally do not have such a powerful expert in practice. However, we can still use the idea of "penalizing" experts that made mistakes:

- Assign "weights" to each expert.
- For each round, we make our final prediction based on the Weighted Majority.
- If an expert makes mistakes, we decrease their weights, for instance, by half.

Intuitively, we start out by trusting each expert equally. As time goes on, based on experts' performance, we adjust how much we trust them. This is the FTWM strategy.

Follow the Weighted Majority (FTWM) Let $w(t, i)$ denote the weight of the i th expert on day t .

- Set $w(0, i) = 1$ for $i = 1, 2, \dots, d$.
- for $t = 1, 2, \dots, T$:

- Make a prediction based on the weighted majority.
- For each expert $i \in \{1, \dots, d\}$:
 - * If E_i made a mistake, set

$$w(t, i) = \frac{w(t-1, i)}{2}$$

- * else: $w(t, i) = w(t-1, i)$.

In other words, if the sum of the weights of experts predicting UP is larger than the sum of the weights of experts prediction DOWN, we will make the prediction UP. Otherwise, we make the prediction DOWN. FTWM has the following guarantee.

Theorem 6.5. (*Regret bound of FTWM*) Let $L(T)$ denote the loss of FTWM at time T . Let $L_*(T)$ denote the loss of the best expert at time T . Then we have

$$L(T) \leq 2.4L_*(T) + 2.4 \log_2 d.$$

Here by loss we mean the number of mistakes.

Proof. Let $\phi(t) = \sum_{i=1}^d w(t, i)$, i.e., the sum of all weights at time t . Then $\phi(0) = d$. Then we have the following claim.

Claim 6.6. *If we make a mistake on day t , then*

$$\phi(t) \leq \frac{3}{4}\phi(t-1).$$

Proof. (of Claim) We have

$$\begin{aligned} \phi(t) &= \sum_{i=1}^d w(t, i) \\ &= \sum_{i: \text{expert } i \text{ was correct on day } t} w(t, i) + \sum_{i: \text{expert } i \text{ was wrong on day } t} w(t, i) \\ &= \sum_{i: \text{expert } i \text{ was correct on day } t} w(t-1, i) + \frac{1}{2} \sum_{i: \text{expert } i \text{ was wrong on day } t} w(t-1, i) \\ &= \frac{1}{2} \sum_{i: E_i \text{ correct}} w(t-1, i) + \frac{1}{2} \left(\sum_{i: E_i \text{ correct}} w(t-1, i) + \sum_{i: E_i \text{ wrong}} w(t-1, i) \right) \\ &\leq \frac{1}{2} \frac{\phi(t-1)}{2} + \frac{1}{2} \phi(t-1) \\ &= \frac{3}{4} \phi(t-1) \end{aligned}$$

where the inequality holds since FTWM was wrong on day t , implying the sum of the weights of the correct experts on that day is at most a half of the total weights. $\square$

Now by Claim 6.6, we know

$$\phi(t) \leq \left(\frac{3}{4}\right)^{L(t)} \cdot \phi(0) = \left(\frac{3}{4}\right)^{L(t)} \cdot d. \quad (17)$$

On the other hand, we have

$$\left(\frac{1}{2}\right)^{L_*(t)} \leq \phi(t). \quad (18)$$

This holds since all weights are non-negative and $w(t, \text{best expert}) = \left(\frac{1}{2}\right)^{L_*(t)}$. Consequently, combining (17) (18), we obtain

$$\left(\frac{1}{2}\right)^{L_*(t)} \leq \left(\frac{3}{4}\right)^{L(t)} \cdot d.$$

Simplifying, we know

$$L(t) \left(\log_2 \left(\frac{4}{3} \right) \right) \leq L_*(t) + \log_2 d$$

which implies

$$L(t) \leq \frac{1}{\left(\log_2 \left(\frac{4}{3} \right) \right)} (L_*(t) + \log_2 d) \approx 2.4(L_*(t) + \log_2 d)$$

as desired. □

By Theorem 6.5, we have the following corollary.

Corollary 6.7.

$$\text{Regret}_{FTWM}(T) = L(T) - L_*(T) \leq 1.4L_*(T) + 2.4 \log_2 d.$$

We say a algorithm is a no-regret algorithm if

$$\frac{\text{Regret}(T)}{T} \rightarrow 0$$

as $T \rightarrow \infty$. Now suppose there exists an expert that only makes mistakes on a tiny fraction of the times, say $L_*(T) = 0.01T$. Then by the corollary,

$$\frac{\text{Regret}_{FTWM}(T)}{T} = \frac{1.4 \cdot 0.01T + 2.4 \log_2 d}{T} = 0.014.$$

Hence, although FTWM is a good algorithm, it is not really a no-regret algorithm provided the best experts makes $\Omega(T)$ mistakes.

Notice that FTWM is a deterministic algorithm as no randomness is used in the algorithm. As it turns out, there is NO no-regret deterministic algorithm. Moreover, no deterministic algorithm can have loss

$$L(T) < 2L_*(T).$$

In the next lecture, we will see how we can make FTWM better by using randomness. Namely, we will study the Multiplicative Weight Method (MWM).

Lecture 7. Multiplicative Weights Method

Last time we discussed the problem of Learning With Experts. We talked about Follow the Leader (FTL), Follow the Majority (FTM), Follow the Weighted Majority (FTWM) algorithms. Although FTWM is a deterministic algorithm that works well when we have a reliable expert, FTWM is still not a no-regret algorithm. In fact, as we mentioned last time, FTWM has almost the best guarantee among all deterministic algorithms. Therefore, the hope to extend beyond the regret bound is to employ randomness in our algorithm, leading to the following Multiplicative Weights Method (MWM).

Multiplicative Weights Method

1. Set $w(0, i) = 1$ for $i = 1, \dots, d$.
2. On day $t = 1, 2, \dots, T$:
 - Pick an expert i at random with probability proportional to $w(t-1, i)$, i.e.,

$$\Pr[E_i \text{ is picked}] = \frac{w(t-1, i)}{\sum_{j=1}^d w(t-1, j)}$$

- Follow the prediction of the picked expert.
- Update the weights of each expert:
 - If expert i made a mistake, set

$$w(t, i) = (1 - \eta)w(t-1, i)$$

- else, set

$$w(t, i) = w(t-1, i).$$

Notice the above method is for binary decisions. But we can easily adapt the following for numerical losses by adjusting the update method: suppose the loss function $l_i(t) \in [0, 1]$, update the weight as

$$w(t, i) = e^{-\eta l_i(t)} w(t-1, i).$$

Another variant is to set

$$w(t, i) = (1 - \eta l_i(t)) w(t-1, i). \tag{19}$$

We will prove our results for this variant. We have the following guarantee.

Theorem 7.1. (*MWM regret upper bound*)

$$\mathbf{E}[L(T)] \leq (1 + \eta)L_*(T) + \frac{\log d}{\eta},$$

where $L(T)$ is our final loss till day T and $L_*(T)$ is the loss of the best expert till day T .

Proof. Let $\phi(t) = \sum_{i=1}^d w(t, i)$. That is, the sum of all weights on day t . Let $l(t, i)$ be the loss incurred by expert i on day t . Using update (19), we know $w(t, i) = (1 - \eta l(t, i)) w(t-1, i)$. We can then calculate the expected loss on day t as:

$$\begin{aligned} \mathbf{E}[\text{loss of MWM on day } t] &= \sum_{i=1}^d \Pr[\text{we pick expert } i] l(t, i) \\ &= \sum_{i=1}^d \frac{w(t-1, i)}{\sum_{j=1}^d w(t-1, j)} l(t, i) \\ &= \frac{1}{\phi(t-1)} \sum_{i=1}^d w(t-1, i) l(t, i). \end{aligned}$$

This is the expected loss incurred by MWM on day t . Denote this as $l_{our}(t)$. We have the following claim connecting $\phi(t)$ and $l_{our}(t)$.

Claim 7.2. *For each day t , the following inequality holds*

$$\phi(t) = (1 - \eta l_{our}(t)) \phi(t-1) \leq e^{-\eta l_{our}(t)} \cdot \phi(t-1).$$

Proof. (of claim) Notice

$$\begin{aligned} \phi(t) &= \sum_{i=1}^d w(t, i) \\ &= \sum_{i=1}^d (1 - \eta l(t, i)) \cdot w(t-1, i) \\ &= \sum_{i=1}^d w(t-1, i) - \eta \sum_{i=1}^d w(t-1, i) l(t, i) \\ &= \phi(t-1) - \eta \cdot \phi(t-1) l_{our}(t) \\ &= (1 - \eta l_{our}(t)) \cdot \phi(t-1). \end{aligned}$$

The statement follows by using the fact that $1 - x \leq e^{-x}$ for all $x \geq 0$. □

Now back to the main proof, since all the weights are non-negative, we know

$$\phi(t) = \sum_{i=1}^d w(t, i) \geq \text{weight of the best expert} = (1 - \eta)^{L_*(t)} \quad (20)$$

since $w(t, i) = (1 - \eta)^{L_i(t)}$ for all i where $L_i(t)$ is the total loss incurred by expert i up to time t . Summing everything up, by (20) and Claim 7.2, we have

$$\begin{aligned} (1 - \eta)^{L_*(t)} &\leq \phi(t) \leq e^{-\eta l_{our}(t)} \cdot \phi(t-1) \\ &\leq e^{-\eta l_{our}(t)} \cdot e^{-\eta l_{our}(t-1)} \dots \phi(0) \\ &= e^{-\eta(l_{our}(1) + l_{our}(2) + \dots + l_{our}(t))} \cdot d \\ &= e^{-\eta L_{our}(t)} \cdot d, \end{aligned}$$

where $L_{our}(t)$ is the total expected loss incurred by MWM. Hence,

$$(1 - \eta)^{L_*(t)} \leq e^{-\eta L_{our}(t)} \cdot d.$$

Taking logarithms of both sides and rearranging, we have

$$L_{our}(T) \leq \frac{\ln\left(\frac{1}{1-\eta}\right)}{\eta} L_*(T) + \frac{1}{\eta} \ln d.$$

For $\eta \leq \frac{1}{2}$, one can show $\frac{\ln(\frac{1}{1-\eta})}{\eta} \leq 1 + \eta$. Using this fact, we arrive at our conclusion:

$$\mathbf{E}[L(T)] = L_{our}(T) \leq (1 + \eta) L_*(T) + \frac{\ln d}{\eta}.$$

□

Setting $\eta = \sqrt{\frac{\ln d}{T}}$, we obtain the following corollary.

Corollary 7.3. *Let $\eta = \sqrt{\frac{\ln d}{T}}$. Then we have*

$$L_{our}(T) \leq L_*(T) + 2\sqrt{T \ln d}.$$

Proof. By Theorem 7.1, and $\eta = \sqrt{\frac{\ln d}{T}}$, we know

$$\begin{aligned}
L_{our}(T) &\leq (1 + \sqrt{\frac{\ln d}{T}})L_*(T) + \frac{\ln d}{\sqrt{\frac{\ln d}{T}}} \\
&= L_*(T) + \sqrt{\frac{\ln d}{T}}L_*(T) + \sqrt{T \ln d} \\
&\leq L_*(T) + \sqrt{\frac{\ln d}{T}} \cdot T + \sqrt{T \ln d} \\
&= L_*(T) + 2\sqrt{T \ln d}.
\end{aligned}$$

The inequality holds since $L_*(T) = \sum_{t=1}^T l_*(t) \leq \sum_{t=1}^T 1 = T$ where $l_*(t)$ is the loss incurred by the best expert at time t . $\square$

By the Corollary, we thus know

$$\frac{L_{our}(T)}{T} \rightarrow 0$$

as $T \rightarrow \infty$ provided $L_*(T) = o(T)$. Thus, MWM is a no-regret algorithm in expectation!

In summary, Multiplicative Weights Method (MWM) is a very powerful method with nice theoretical guarantees and has a variety of applications. Within learning, it can be used in say, Portfolio management, Online Learning, Graphical Models and many other areas. The famous Boosting method is a great application of MWM that adjusts the weight of each sample in the learning setting.

In Optimization, the same idea is also widely used. We can use it for a version of "Multiplicative" gradient descent. We can also use it to solve Linear Programs, Semi-definite Programs and many other problems.

Lecture 8. Bandit Learning

Last time we talked about the Learning with Experts problem. In that setting, given d experts, we know exactly the losses / rewards of all the experts at any given round. This is a very strong assumption that does not hold in many real-life settings.

So instead of assuming we have complete / exact info about each expert's losses, we imagine the experts as "slot machines." In other words, at any time, we cannot see the information about all the experts. Instead, we can only see the outcome (i.e., loss / reward) for the action we took. For instance, if we choose expert i on day t , we only get to see his or her reward/loss on this day. If we choose to pull the trigger of the j th slot machine, we only get to see the reward of this particular slot machine. This setting is called Multi-Armed Bandits (MAB). In particular, we consider the following setting:

1. Stochastic Bandits: Each expert and choice has a reward distribution P_i .
2. On each day t , the reward of expert i is sampled from P_i independently.

	P_1	P_2	$\dots$	P_d
	choice 1	choice 2		choice d
Day 1	✓ 0.8	?	?	?
Day 2	?	✓ 10	?	?
Day 3		✓ 9		
		:		

Figure 19: Multi-armed Bandit setup: on each day you only get to see the reward of the expert / choice you make.

More concretely,

- Suppose we have d -armed bandits with reward distributions $P_1, \dots, P_d$ over $\mathbb{R}$ respectively. Let μ_i be the mean of distribution P_i .
- Denote our algorithm as an "agent."
- On day t :
 - The algorithm chooses an arm $I_t \in \{1, 2, \dots, d\}$.
 - The environment shows us a reward r_t sampled from distribution P_{I_t} .

Our goal is to minimize the gap to the best choice, i.e., minimize

$$R_T = T\mu_* - \mathbf{E}[r_{I_1} + r_{I_2} + \dots + r_{I_T}]$$

where $\mu_* = \max\{\mu_1, \dots, \mu_d\}$ is the mean of the best choice.

The problem we are facing here is really an "Exploration vs Exploitation Dilemma." We started out knowing nothing about the P_i 's and μ_i 's. To gain more information, we must try pulling different arms. However, at some point, you have some basic information of some of the arms you pulled. In this case, how do you gain more information by exploring other arms or do you pull the best arms given the available information which might be the best action in hindsight? Ideally, we would like a balance of the two. Perhaps we would like to favor more exploration in the beginning and prefer more exploitation when we have a lot more information.

This MAB model has a lot of practical applications in, for instance, dynamic traffic, online ad display, restaurant choices, adaptive clinical trials and many others.

For people familiar with Reinforcement Learning, this MAB setting is the simplest non-trivial Reinforcement Learning Problem:

- Agent: Bandit Algorithm
- Environment: System giving rewards
- Actions: choices
- "Policy": algorithm for choosing actions
- "State": No states in MAB.

We will discuss the UCB algorithm and the Thompson Sampling algorithm. To that end, we make the following simplifying assumption on the reward distributions: consider the Gaussian Bandit Problem in which we assume each reward $r_i \sim \mathcal{N}(\mu_i, 1)$, i.e., a Gaussian distribution with mean μ_i and variance 1. Recalling the definition of Gaussian, this is saying

$$\Pr[\text{reward}_i = r] = \frac{1}{\sqrt{2\pi}} e^{-\frac{(r-\mu_i)^2}{2}}.$$

Our goal is to minimize:

$$R_T = T\mu_* - \mathbf{E}[r_{I_1} + r_{I_2} + \dots + r_{I_T}]. \quad (21)$$

On one hand, we have the following lower bound proved by Lai and Robbins back in the 80s:

Lemma 8.1. (*MAB Lower Bound*) *One must have $R_T \geq \Omega(\log T)$. Specifically,*

$$R_T \geq \Omega(\log T) \cdot \sum_{i:\mu_i < \mu_*} \frac{1}{\mu_* - \mu_i}.$$

We would like an algorithm that whose R_T is as close to this lower bound as possible. One idea is to perhaps adapt MWM. For the chosen expert at each round, we update their weight properly. For the rest of the experts, we use the past history to give rewards and update weights. This leads to the Upper Confidence Bound (UCB) algorithm discovered in the 2000's.

8.1 Upper Confidence Bound (UCB) algorithm

The high-level idea of the UCB algorithm is to take an "optimist's" approach: at each round, we choose the arm with the highest possible "upper confidence bound" given the current data.

Recall the idea of a confidence interval: say we have a random variable $X \sim \mathcal{N}(\mu, 1)$, at what interval do I have great confidence that it will contain the true mean μ ? To be more exact, let's say we have an estimate $\hat{\mu}$ of the true mean μ . Then a confidence interval is given by some C_i such that $\Pr[\mu_i \in [\hat{\mu}_i - C_i, \hat{\mu}_i + C_i]]$ with high probability (e.g. with probability ≥ 0.9). See Figure 20 for an illustration.

To put it simply, the iteration steps of the UCB algorithm look like the following: for each round t :

- Compute $\hat{\mu}_i(t-1) :=$ Observed average of arm i so far.
- Compute the "upper confidence bound."
- Pick the choice with the highest "reasonable" value for the unknown mean.

We will specify more rigorously in a short bit. To make the steps more concrete, we need to figure out how to compute each of the quantities mentioned above.

Given samples from an unknown Gaussian distribution $\mathcal{N}(\mu, 1)$, what is our best guess of the true mean? It turns out the answer is quite straightforward: we can simply take our guess $\hat{\mu}$ to be the empirical average: let $x_1, x_2, \dots, x_N \sim \mathcal{N}(\mu, 1)$ be samples from the distribution. Then the empirical average

$$\hat{\mu} = \frac{x_1 + x_2 + \dots + x_N}{N}$$

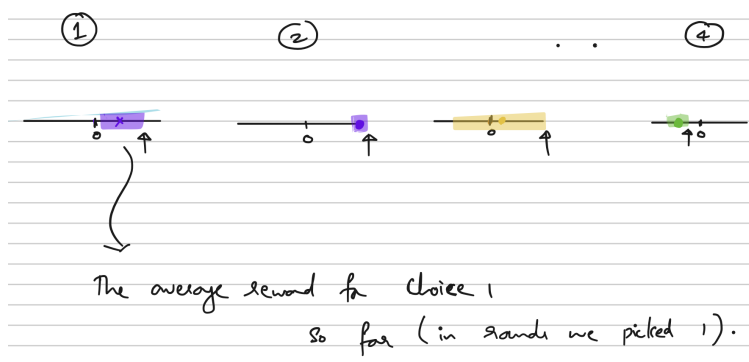


Figure 20: Illustration of upper confidence bound: here we have four arms. The average award for each arm so far are indicated by the dots. The shaded area indicated the computed confidence interval. The UCB algorithm will pick the arm with the highest upper confidence bound (the rightmost end of each shaded area).

is the "best" estimate of the true mean. (Note: there are exact rigorous notions of "best" in Statistics. We will take this as a given. If you are interested, you can refer to *Theoretical Statistics: Topics for a Core Course* by Keener for reference.)

Now that we have our estimate $\hat{\mu}$ of the true mean, we want to compute a value C such that

$$\Pr[\mu \notin [\hat{\mu} - C, \hat{\mu} + C]] \leq \delta.$$

To this end, we borrow the following fact about Gaussian distributions:

Claim 8.2. *If we see N i.i.d samples $x_1, \dots, x_N \sim \mathcal{N}(\mu, 1)$, then*

$$\Pr[\mu > \hat{\mu} + C] \leq \delta, \quad \text{for } C = \sqrt{\frac{2 \ln(1/\delta)}{N}}.$$

We now have our estimate $\hat{\mu}$ and a way of computing the confidence interval. The last step is to determine what confidence level, i.e., δ we need. Ideally, we want to be more accurate after many around. So we would like δ to get smaller. For instance, we can choose $\delta = \frac{1}{T}$. At last, we are ready to rigorously state the UCB algorithm.

UCB algorithm

- Input: number of arms d and time horizon T
- Initialization step:
 - For $i = 1, \dots, d$: pull arm i . Observe reward $r(0, i)$.
 - Set $\hat{\mu}_i(0) = r(0, i)$, i.e., the reward observed for pulling arm i .
 - Set $N_i(0) = 1$ for all i .
- // Let $\hat{\mu}_i(t)$ denote the empirical mean for rewards on arm i up to day t .
- // Let $N_i(t)$ denote the number of times arm i has been pulled in the first t days.
- Main iteration step:
 - For $t = 1, 2, \dots, T$:
 1. Compute the upper confidence bound for each arm:

$$B_i(t) = \hat{\mu}_i(t-1) + \sqrt{\frac{2 \ln t}{N_i(t-1)}}.$$

2. Select the arm with the highest $B_i(t)$, i.e.,

$$I_t = \arg \max_i B_i(t).$$

3. Observe reward for arm I_t then update $\hat{\mu}_{I_t}$ and $N_{I_t}(t) = N_{I_t}(t-1) + 1$.

Remark: recall our discussion earlier of the dilemma of choosing between "exploration" and "exploitation." We regard the component $\hat{\mu}_i(t-1)$ in the definition of $B_i(t)$ as the "exploitation" part as it contains information of which arm is the best so far. The $\sqrt{\frac{2 \ln t}{N_i(t-1)}}$ can be regarded as the "exploration" part which allows us for the opportunity to pull an arm that is different than the one having the highest empirical average.

We will state without proving the following guarantee of UCB algorithm.

Theorem 8.3. *The UCB algorithm gets regret*

$$R_T = T\mu_* - \mathbf{E}[r_{I_1} + r_{I_2} + \dots + r_{I_T}] = O(\log T).$$

In other words, the difference between the expected reward of UCB and the best arm is no larger than $O(\log T)$.

Comparing this with the lower bound in Lemma 8.1, we know UCB obtains the optimal bound.

8.2 Thompson Sampling

In UCB algorithm, we adopted an "optimist" approach by believing what the empirical data tell us and make decisions accordingly. In Statistics, this is typically called a "frequentist" approach.

Contrary to the frequentist approach is the Bayesian approach. In the Bayesian setting, instead of simply outputting the empirical average as our guess of the true mean, we maintain a posterior probability distribution for what rewards each arm has. Specifically, we make the following assumption.

Assumption 8.4. *There exists a prior distribution on the means $\mu_1, \mu_2, \dots, \mu_d$, i.e., for each $i = 1, 2, \dots, d$, we have $\mu_i \sim \mathcal{N}(\mu_0, \sigma_0^2)$ for some μ_0 and σ_0 . There is also a posterior update phase: after observing N_i rewards for arm i with empirical mean $\hat{\mu}_i$, we adjust our belief of the distribution of μ_i accordingly.*

In essence, the posterior for the distribution of μ_i is another Gaussian distribution $\mathcal{N}(\mu'_i, \sigma_i'^2)$ with mean $\mu'_i = \frac{N_i \hat{\mu}_i}{N_i + 1}$ and $\sigma_i'^2 = \frac{1}{N_i + 1}$. Based on this, we have the following Thompson Sampling algorithm (with the initialization step same as the UCB algorithm, omitted below).

Thompson Sampling

- For $i = 1, 2, \dots, T$:
 - Sample from current posterior distributions for each arm and obtain the posterior means $\hat{\mu}_1^{post}, \hat{\mu}_2^{post}, \dots, \hat{\mu}_d^{post}$
 - Pull the arm with the highest sample:

$$I_t = \arg \max_i \hat{\mu}_i^{post}$$

- We observe the reward $r(t, I_t)$
- Update the posterior distributions accordingly.

We have the following guarantee of Thompson Sampling.

Theorem 8.5. *Thompson Sampling has the regret bound*

$$R_T = O(\log T),$$

which is the same bound as UCB.

One might ask since the two algorithms have same guarantees, which algorithm should we use? If we have some domain knowledge or information that allows us to model the means of each arm, then Thompson Sampling may give us better results practically (e.g. the regret may be a constant factor smaller). If we do not have such information, then UCB generally works well.

In summary, in this module we discussed the problem of online learning. Unlike supervised learning, in the Online Learning setting, you need to make decisions as the data come in. We discussed the Perceptron algorithm for learning halfspaces and the learning experts problem with both deterministic (e.g FTWM) and randomized (e.g. MWM) algorithms. Lastly, we discussed the Bandit Learning problem and its connection to Reinforcement Learning.

Part III

Principal Component Analysis

Lecture 9. Best-fit Subspace

We now enter a new module. In this module, we are going to discuss Principal Component Analysis (PCA) and related problems. To start, let's consider the problem of determining the best-fit-line: given inputs $x^1, x^2, \dots, x^n \in \mathbb{R}^d$, we want to output the line that is closest to the dataset.

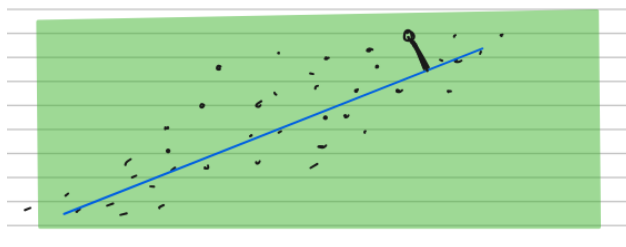


Figure 21: Example of what best-fit line looks like.

To obtain such a line, the idea is to minimize the sum of the distances of the points to the line. To this end, we define the projection of $x \in \mathbb{R}^d$ onto a line as the following.

Definition 9.1. Let $x \in \mathbb{R}^d$ and L be a line in $\mathbb{R}^d$. Then define

$$Proj_L(x) := \arg \min_{y \in L} \|x - y\|_2,$$

in other words, $Proj_L(x)$ is the point y on the line L that is closest to x .

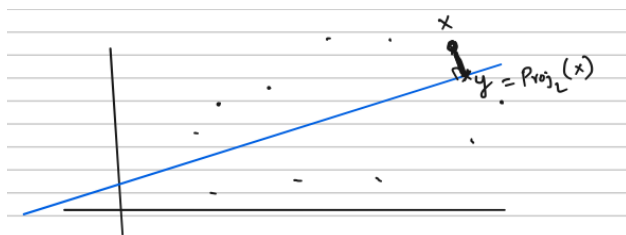


Figure 22: Illustration of $Proj_L(x)$.

If we have a dataset $X = \{x_1, x_2, \dots, x_n\} \subseteq \mathbb{R}^d$, let the error of a line L on X to be

$$ERROR(L, X) = \sum_{i=1}^n \|x_i - Proj_L(x_i)\|^2.$$

With this, we can formalize the Best-fit line problem.

Problem 9.2. (*Best-fit Line Problem*) Let $X = \{x_1, x_2, \dots, x_n\} \subseteq \mathbb{R}^d$ be the input dataset. We want to output a line L that minimizes

$$ERR(L, X) = \sum_{i=1}^n \|x_i - Proj_L(x_i)\|^2.$$

Recall from linear algebra: a line (through the origin) is a 1-dimensional subspace. Hence, as a natural generalization of the best-fit line problem, we can ask the question of what is the best-fit subspace of the dataset X .

Problem 9.3. (*Best-fit subspace Problem*) Let $X = \{x_1, x_2, \dots, x_n\} \subseteq \mathbb{R}^d$ be the input dataset. We want to output a k -dimensional subspace S that minimizes

$$ERR(S, X) = \sum_{i=1}^n \|x_i - Proj_S(x_i)\|^2,$$

where $Proj_S(x_i) = \arg \min_{y \in S} \|y - x\|_2$.

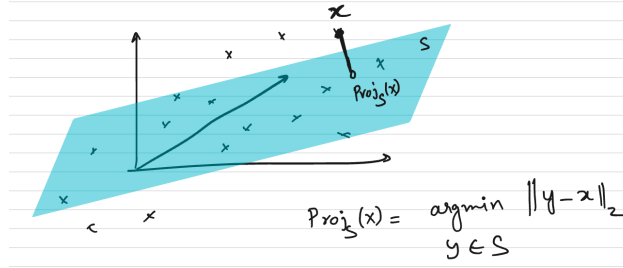


Figure 23: Projection of x onto a subspace S .

We want to answer the following questions: (1) how do we solve the best-fit line problem, i.e., when $k = 1$? (2) If there's a solution, can we extend this to higher k ? We will tackle these questions using Linear Algebra. So before we proceed, let's review some important concepts from Linear Algebra that we are going to use.

9.1 Quick Linear Algebra Review

We will be using the following facts from Linear algebra.

Fact 9.4. 1. The angle between $x - Proj_S(x)$ and $Proj_S(x)$ is 90 degrees. In other words,

$$\langle x - Proj_S(x), Proj_S(x) \rangle = 0.$$

2. The norm of x can be decomposed as

$$\|x\|^2 = \|Proj_S(x)\|^2 + \|x - Proj_S(x)\|^2.$$

3. For all $u \in S$, we have $x - Proj_S(x)$ is perpendicular to u , i.e.,

$$\langle u, x - Proj_S(x) \rangle = 0.$$

9.2 Principal Components

Using the linear algebra facts above, we can rewrite $ERR(L, X)$ as follows;

$$\begin{aligned} ERR(L, X) &= \sum_{i=1}^n \|x_i - Proj_L(x_i)\|^2 \\ &= \sum_{i=1}^n (\|x_i\|^2 - \|Proj_L(x_i)\|^2) \\ &= \sum_{i=1}^n \|x_i\|^2 - \sum_{i=1}^n \|Proj_L(x_i)\|^2 \end{aligned}$$

where the second of the linear algebra facts above. Since the x_i 's are given to us as the dataset, we can compute the first term exactly. Therefore, finding a line that minimizes $ERR(L, X)$ is the same as finding a line L that maximizes

$$\sum_{i=1}^n \|Proj_L(x_i)\|^2.$$

Denote this quantity as $Var(L, X)$.

We can analogously define the quantity $ERR(S, X)$ and from the same derivation as above, we know that **finding the subspace S that minimizes the error $ERR(S, X)$ is equivalent to finding the subspace S that maximizes $Var(S, X)$** .

Recall that for any finite dimensional space S , we can find an orthogonal set of vectors that completely determine S . This set of vectors is called an orthogonal basis for S . Starting with a line, i.e., an one-dimensional vector space, we characterize via the direction at which it is pointing. In other words,

$$L = Span(\{v\}) = \{x \in \mathbb{R}^d : x = \lambda \cdot v, \lambda \in \mathbb{R}\}.$$

Knowing this, the projection of x on to L can be computed as

$$Proj_L(x) = \langle x, \frac{v}{\|v\|_2} \rangle \cdot \frac{v}{\|v\|_2}.$$

This implies

$$\begin{aligned} \|Proj_L(x)\| &= \left\| \langle x, \frac{v}{\|v\|_2} \rangle \cdot \frac{v}{\|v\|_2} \right\|_2 \\ &= \left| \langle x, \frac{v}{\|v\|_2} \rangle \right| \cdot \left\| \frac{v}{\|v\|_2} \right\|_2 \\ &= \frac{|\langle x, v \rangle|}{\|v\|_2}. \end{aligned}$$

We denote $Var(L, X)$ to be the sum of the squared length of the projections. In other words,

$$Var(L, X) = \sum_{i=1}^n \|Proj_L(x_i)\|^2 = \sum_{i=1}^n \frac{\langle x_i, v \rangle^2}{\|v\|_2^2} = \frac{\|Xv\|^2}{\|v\|^2}$$

where X is an $n \times d$ matrix with rows being the data points $x_1, x_2, \dots, x_n$. See Figure 24.

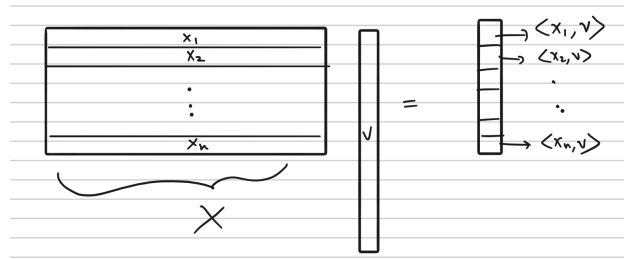


Figure 24: Writing $Var(L, X)$ in terms of matrix vector products

Hence, the Best-fit Line Problem can be formulated as:

- Input: A matrix $X \in \mathbb{R}^{n \times d}$.
- Output: A vector v that maximizes $\frac{\|Xv\|^2}{\|v\|^2}$.

A unit vector (i.e., norm being 1) that achieves this maximum is called the first Right Singular Vector or the first Principal Component of X . Intuitively, this unit vector is one which gets "stretched out" the most by X among all unit vectors. See the following for a few examples.

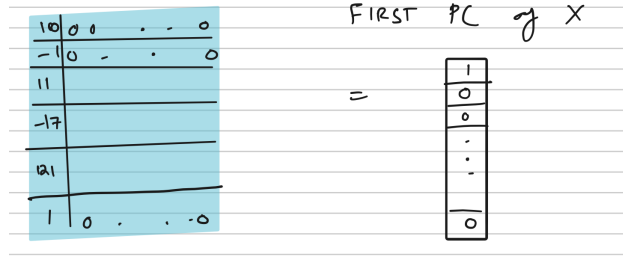


Figure 25: Given the data matrix X (on the left), the first Principal Component of X is the standard bases vector $e_1 = (1, 0, \dots, 0)^T$. To see this, note that only the first column of X contains non-zero values. Performing matrix-vector multiplication, only the first coordinate of v is involved. Hence, to maximize the stretch, we want the first coordinate to be as large as possible.



Figure 26: Here each data point x^i is a multiple of u . Hence, for each inner product $\langle x_i, v \rangle$, $\frac{\|x_i v\|^2}{\|v\|^2}$ is maximized when v is aligned with u . Since this holds for all i , we can just take v to be $\frac{u}{\|u\|_2}$.

Now we know how to conceptually solve the best-fit-line problem (i.e., $k = 1$), what can we do when we to solve the best-fit-subspace problem for $k = 2$ and larger? Recall the best-fit-subspace problem can be written as

$$\min \text{ERR}(S, X) = \sum_{i=1}^n \|x_i - \text{Proj}_S(x_i)\|^2$$

and equivalently,

$$\max \text{Var}(S, X) = \sum_{i=1}^n \|\text{Proj}_S(x_i)\|^2.$$

We are going to solve this using a "Greedy approach." Specifically, for $k = 2$, we can first find the first Principal Component v_1 . Then take v_1 "out" and repeat the process. Concretely, we will

1. Find the first Principal Component v_1 .
2. Let $\tilde{x}_i = x_i - \langle x_i, v_1 \rangle v_1$.
3. Find the first Principal Component v_2 of the "new" dataset

$$\tilde{X} = \{\tilde{x}_1, \tilde{x}_2, \dots, \tilde{x}_n\}.$$

Another idea is to do the following:

1. Find the first Principal Component v_1
2. Find the vector v that is orthogonal to v_1 and maximizes $\frac{\|Xv\|^2}{\|v\|^2}$.

It turns out these two ideas are equivalent! They both define the top principal components of X . The second Principal Component or the second Right Singular Vector of X is defined as:

$$\max_{v: \langle v, v_1 \rangle = 0} \frac{\|Xv\|^2}{\|v\|^2}.$$

We can naturally extend this definition to k larger than 2 as the following.

Definition 9.5. (the i th Principal Component / Right Singular vector) If $v_1, v_2, \dots, v_{i-1}$ are the first, second, ..., $(i-1)$ -th right singular vectors of X , then the i th right singular vector is any vector that

$$\max_{v: \langle v, v_1 \rangle = 0, \dots, \langle v, v_{i-1} \rangle = 0} \frac{\|Xv\|^2}{\|v\|^2},$$

i.e., the vector that maximizes $\frac{\|Xv\|^2}{\|v\|^2}$ among the vectors orthogonal to the first $(i-1)$ right singular vectors.

Remark: If we consider the best-fit subspace problem using the original definition, i.e., finding the subspace S that minimizes

$$ERR(S, X) = \sum_{i=1}^n \|x_i - Proj_S(x_i)\|_2,$$

then the greedy algorithm will fail. The resulting problem is thus much harder.

9.3 Top Principal Components and best-fit subspace

We are now ready to prove that the top Principal Components (Right Singular vectors) solves the best-fit subspace problem. We will prove the case for $k = 2$. One can easily adapt the proof to larger k 's.

Theorem 9.6. The span of the first two Principal Components solve the Best-fit Subspace problem for $k = 2$.

Proof. Let v_1 be the first Principal Component of X , i.e., v_1 maximizes $\frac{\|Xv\|^2}{\|v\|^2}$. Let v_2 be the second Principal Component of X , i.e., v_2 maximizes $\frac{\|Xv\|^2}{\|v\|^2}$ subject to the constraint that $\langle v, v_1 \rangle = 0$. Let $S^{PC} = \text{Span}(\{v_1, v_2\})$. We want to show that S^{PC} maximizes $Var(S, X)$ among all 2-dimensional subspaces where

$$Var(S, X) = \sum_{i=1}^n \|Proj_S(x_i)\|^2.$$

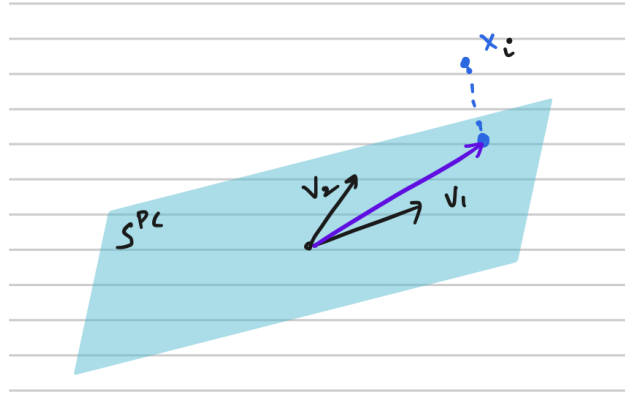


Figure 27: The subspace spanned by the first two Principal Components.

We will use the following fact from Linear Algebra.

Fact 9.7. Let w_1, w_2 be two orthonormal vectors and $T = \text{Span}\{w_1, w_2\}$. Then we have

$$\|Proj_T(x)\|^2 = \langle x, w_1 \rangle^2 + \langle x, w_2 \rangle^2.$$

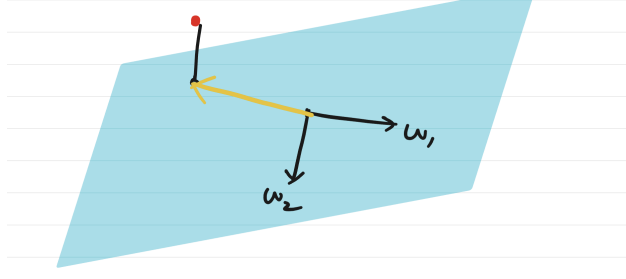


Figure 28: Illustration of Fact 9.7.

Using this fact, we can rewrite $\text{Var}(S^{PC}, X)$ as

$$\begin{aligned} \text{Var}(S^{PC}, X) &= \sum_{i=1}^n \|\text{Proj}_{S^{PC}}(x_i)\|^2 \\ &= \sum_{i=1}^n \langle x_i, v_1 \rangle^2 + \langle x_i, v_2 \rangle^2 \\ &= \sum_{i=1}^n \langle x_i, v_1 \rangle^2 + \sum_{i=1}^n \langle x_i, v_2 \rangle^2 \\ &= \|Xv_1\|^2 + \|Xv_2\|^2. \end{aligned}$$

Now, let T be an arbitrary 2-dimensional subspace spanned by two orthonormal vectors w_1, w_2 , i.e., $T = \text{Span}(\{w_1, w_2\})$. Then by the above, $\text{Var}(T) = \|Xw_1\|^2 + \|Xw_2\|^2$. We want to show $\text{Var}(S^{PC}, X) \geq \text{Var}(T)$.

On one hand, recall that v_1 maximizes $\frac{\|Xv\|^2}{\|v\|^2}$. Therefore, we know

$$\|Xw_1\|^2 \leq \|Xv_1\|^2,$$

which holds since $\|w_1\| = \|v_1\| = 1$ by assumption. In addition, we also have $\|Xw_2\|^2 \leq \|Xv_1\|^2$. However, using this, we cannot say $\|Xw_2\|^2 \leq \|Xv_2\|^2$ in general since v_1 may not be necessarily orthogonal to w_2 . To resolve this, notice that instead of picking an arbitrary orthogonal basis for T , we can always pick an orthogonal basis $\{w_1, w_2\}$ such that w_2 is orthogonal to v_1 (yet another linear algebra fact). See Figure 29 for an illustration.

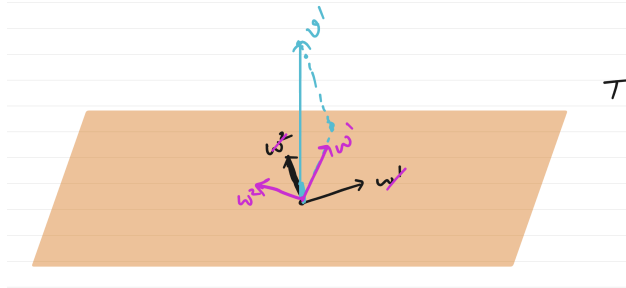


Figure 29: A basis for T such that w_2 is orthogonal to v_1 .

Using this new basis for T , i.e., the one such that w_2 is orthogonal to v_1 , we thus have

$$\|Xw_2\|^2 \leq \|Xv_2\|^2$$

which holds since by definition, v_2 is the second Principal Component of X . Therefore,

$$\begin{aligned} \text{Var}(T, x) &= \|Xw_1\|^2 + \|Xw_2\|^2 \\ &\leq \|Xv_1\|^2 + \|Xv_2\|^2 \\ &= \text{Var}(S^{PC}, X), \end{aligned}$$

as desired. □

In summary, we have thus shown that finding the first k Principal Components of X will solve the Best-fit Subspace problem for dimension k . We will discuss how to find the principal components via Singular Value Decomposition (SVD) and the Netflix Challenge (low-rank approximation) next time.

Lecture 10. Low-rank Approximation

In this lecture, we are going to discuss how to approximate a data matrix X with a "simple" version while maintaining most of the relevant information. Before we actually solve this problem, we need to be more precise about a couple terms in the question: what do "approximate" and "simple" mean for us?

We will address the second question first. For us, "simple" means low rank. Recall from Linear Algebra that the rank of a matrix X is defined as

$$\text{rank}(X) = \text{dimension}(\text{span of rows of } X) = \text{dimension}(\text{span of columns of } X).$$

We say a matrix X has rank $\leq r$ if it can be written as $X = AB$ where A is an n -by- r matrix and B is an r -by- d matrix.

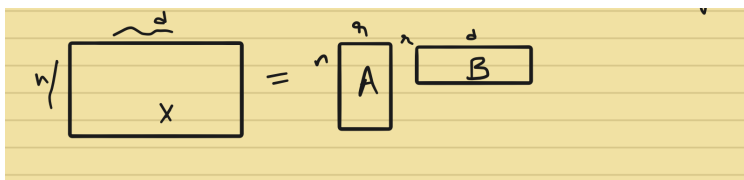


Figure 30: A matrix X of rank $\leq r$.

Simplicity: One might ask why do we study rank as a notion of "simplicity". To see the benefit of low-rank matrices, first notice that to store an $n \times d$ matrix X , in general we need nd memory. However, if X is low-rank, i.e., X admits a decomposition as shown in Figure 30, then we need only $nr + rd = (n + d)r$ memory. If the rank r is small compared to n, d , e.g., $n = 10,000, d = 10,000, r = 2$, then we can store the data much more efficiently.

In addition to storage issues, one can perform matrix-related efficiently if our matrix X is low-rank. For instance, let's consider Matrix-vector Multiplication. Let X be an $n \times d$ matrix and v an d -dimensional vector. Then multiplying Xv naively requires $O(nd)$ many operations. However, if X is low-rank, then X can be written as the product of A, B where A is an $n \times r$ matrix and B is an $r \times d$ matrix. Then Xv can be written as

$$Xv = (AB)v = A(Bv).$$

Computing Bv requires rd many operations. Given the computed Bv , computing $A(Bv)$ requires nr additional operations. So the overall number of operations is

$$rd + nr = (n + d)r$$

which again, can be much more efficient than computing the matrix vector multiplication naively.

Approximation: Now we need to discuss what do we mean by "approximating." One thing we can do is to essentially treat matrices as long vectors and compute the l_2 distance between them. In other words, let X be an $n \times d$ matrix and $\tilde{X}$ be an $n \times d$ matrix that is its approximation. We can compute the squared error by treating them as vectors as:

$$\sum_{i=1}^n \sum_{j=1}^d (X_{ij} - \tilde{X}_{ij})^2.$$

The root mean squared error of X and $\tilde{X}$, which we will denote $\|X - \tilde{X}\|_F$ is given by

$$\|X - \tilde{X}\|_F = \left(\sum_{i=1}^n \sum_{j=1}^d (x_{ij} - \tilde{x}_{ij})^2 \right)^{1/2}.$$

We use the notation $\|\cdot\|_F$ to denote a norm (length) that we can define on matrices. The Frobenius norm of an $n \times d$ matrix A is defined as:

$$\|A\|_F = \left(\sum_{i=1}^n \sum_{j=1}^d A_{ij}^2 \right)^{1/2}.$$

One alternate norm we consider is the spectral norm. But from now on, we will be focused on using the Frobenius norm as our measure of distance in our approximation. Now we are ready to formally state the **Low-rank Approximation Problem**: given inputs $X \in \mathbb{R}^{n \times d}$ and a positive integer k , we want to output an $n \times d$ matrix $\tilde{X}$ of rank k that minimizes $\|X - \tilde{X}\|_F$.

10.1 Netflix Challenge

A few years ago, Netflix launched a challenge. They have a large dataset with movie preferences / ratings of 500k users that includes 20k many features. It takes roughly 8GB to store the entire dataset.

	MOVIES (17k)																
USERS	?	?	?	4	0	?	1										
	?	4	?	0	?	?	1										5
500k																	

8GB to store the matrix

Figure 31: Netflix's user / movies dataset for the challenge.

However, not the entire dataset is available to the contestant. In fact, only roughly 100 million entries are seen which is roughly 1% of all the entries. The contestants' goal is to guess the missing entries. Each contestant will be scored based on the average RMSE between their guesses and some "holdout" entries. The first team to do 10% better than Netflix's baseline model could get \$1 million dollars!

While the winning team did something that is more special to the specific dataset, the following method can already beat the baseline by at least 4%.

- Step 1: Preprocess the data to center the data by subtracting the row average from each entry. (One additional thing one can do is to "scale" the variance).
- Step 2: Put a 0 for every missing entry. Denote the resulting matrix as X .
- Step 3: Find the best low-rank approximation $\tilde{X}$ of X (for some choice of rank k).
- Step 4: Predict the missing entries according to $\tilde{X}$ and adjust the values by "undoing" the preprocessing.

As it turns out, if you run the above steps for $k = 30$, you can beat Netflix's baseline by 4%! It remaining mystery is how one can find the best low-rank approximation. To do this, we will be using the **Singular Value Decomposition**.

10.2 Singular Value Decomposition (SVD)

10.2.1 SVD definition and examples

Theorem 10.1. (Singular Value Decomposition) Any $n \times d$ matrix X can be written as $X = U\Sigma V^T$ where

- Σ is an $r \times r$. diagonal matrix with non-negative entries
- U is an $n \times r$ matrix such that the columns of U are orthonormal, i.e., $U^T U = I_r$.
- V is an $r \times d$ matrix such that the columns of V are orthonormal, i.e., $V^T V = I_r$.
- The diagonal entries of Σ satisfies: $\Sigma_{11} \geq \Sigma_{22} \geq \dots \geq \Sigma_{rr}$.

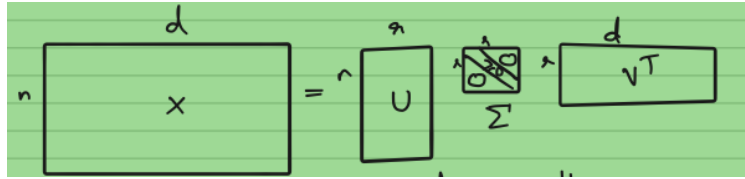


Figure 32: Singular Value Decomposition of an $n \times d$ matrix X into the product of U , Σ and V^T where U, V are orthonormal and Σ is diagonal with non-negative entries.

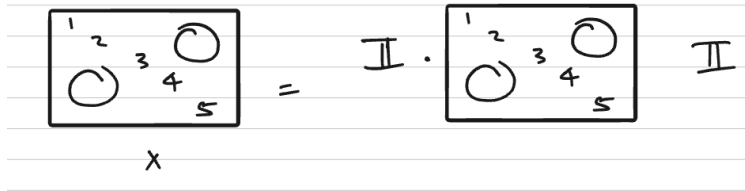


Figure 33: Example of SVD. To check this is a valid SVD, note that the identity matrix I is orthonormal and Σ is diagonal with non-negative entries.

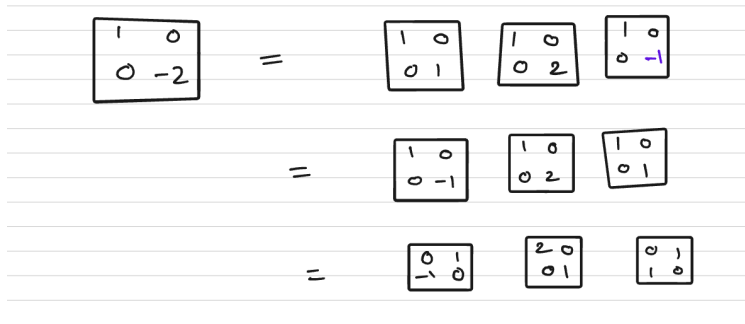


Figure 34: Another example of SVD. Different from the first example, here although X is diagonal, one of the entries on the diagonal is negative. So we cannot do the same decomposition as before by taking $U = V = I$. However, one can start with the same idea and then "push" the negative sign into entries of V or U . That's how we obtain the first line and the second line. Rearranging the columns and rows so that $\Sigma_{11} \geq \Sigma_{22}$, we obtain the last line as our SVD.

What is nice and perhaps striking about Singular Value Decompositions are the following properties of SVD.

Theorem 10.2. (SVD and Principal Components) Let X be an $n \times d$ matrix and let $X = U\Sigma V^T$ be its singular value decomposition. Then v_1 , i.e., the first column of V (the first row of V^T) is a first principal component of X . v_2 is a second principal component of X . In general, v_k is a k th principal component of X .

In addition, we also have the following nice connection between SVD and low-rank approximation.

Theorem 10.3. (SVD and low-rank approximation) Let X be an $n \times d$ matrix and let $X = U\Sigma V^T$ be its singular value decomposition. Then,

$$X_k = (u_1 \ u_2 \ \dots \ u_k) \begin{pmatrix} \Sigma_{11} & & & \\ & \Sigma_{22} & & \\ & & \dots & \\ & & \dots & \Sigma_k \end{pmatrix} \begin{pmatrix} v_1^T \\ v_2^T \\ \dots \\ v_k^T \end{pmatrix}$$

is the best rank- k approximation of X .

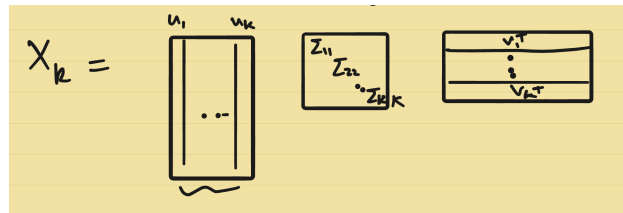


Figure 35: Best rank- k approximation from SVD

As a result, SVD is a very powerful tool that solves both PCA and low-rank approximation! The question now thus becomes how we can compute SVD efficiently.

10.2.2 Computing SVD

Recall the SVD of X is given as $X = U\Sigma V^T$. As shown in Figure 36, we can write the SVD as

$$X = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_r u_r v_r^T$$

where $\sigma_i = \Sigma_{ii}$, u_i is the i th left singular vector and v_i is the i th principal component which is also known as the first right singular vector. We call σ_i the i th singular value.

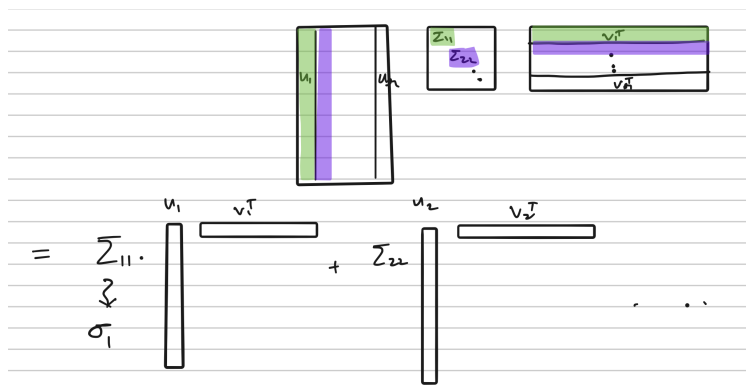


Figure 36: We can expand matrix multiplication as a summation of column row products.

We can prove a few useful facts using this summation form of SVD.

1. $Xv_i = \sigma_i u_i$ for all i . Notice that

$$\begin{aligned} Xv_i &= (\sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \dots + \sigma_r u_r v_r^T) v_i \\ &= \sigma_1 u_1 (v_1^T v_i) + \sigma_2 u_2 (v_2^T v_i) + \dots + \sigma_r u_r (v_r^T v_i) \\ &= \sigma_i u_i, \end{aligned} \tag{22}$$

where the last line holds since the v_i 's are orthonormal, i.e., $\langle v_i, v_j \rangle = 0$ if $i \neq j$ and $\|v_i\| = 1$ for all i .

2. Using the same idea, we can show

$$u_i^T X = \sigma_i v_i^T \quad (23)$$

for all i .

3. $\sigma_i = \|Xv_i\|$. This is true since

$$\|Xv_i\| = \|\sigma_i u_i\| = \sigma_i, \quad (24)$$

where we use 1 and the fact that u_i has unit norm.

As a result, given an arbitrary $n \times d$ matrix X , if we can compute $v_1, \dots, v_r$ in the SVD of X , we can then compute $\sigma_1, \dots, \sigma_r$ and $u_1, \dots, u_r$ as well. There are many methods to compute full SVD. In particular, we have the following theorem:

Theorem 10.4. *There exists an algorithm that can compute the full SVD in time $O(1) \cdot \min(dn^2, nd^2)$.*

However, in many cases, we really do not need to compute the full SVD. For instance, if we want to obtain a rank $k = 30$ approximation of X , we only need to compute the top k singular values and the associated singular vectors. So the question is: can we compute the first 30 singular vectors easily?

10.3 Power Iteration

Observe the following property of SVD. Let $X = U\Sigma V^T$ be the singular value decomposition of X . Then consider $Y = X^T X$. We have

$$\begin{aligned} Y = X^T X &= (U\Sigma V^T)^T \cdot (U\Sigma V^T) \\ &= V\Sigma(U^T U)\Sigma V^T \\ &= V\Sigma^2 V^T \end{aligned}$$

where the last line holds since U is orthonormal (thus $U^T U = I$).

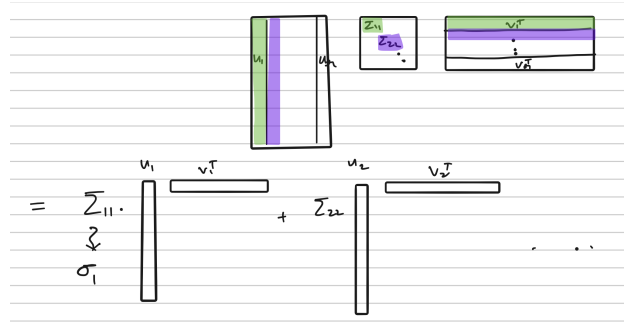


Figure 37: The matrix $Y = X^T X$ given the singular value decomposition $X = U\Sigma V^T$.

We can write in the summation form as:

$$Y = \sigma_1^2 v_1 v_1^T + \sigma_2^2 v_2 v_2^T + \dots \sigma_r^2 v_r v_r^T.$$

Similarly,

$$\begin{aligned} Y^2 &= (V\Sigma^2 V^T)(V\Sigma^2 V^T) \\ &= V\Sigma^2 (V^T V) \Sigma^2 V^T \\ &= V\Sigma^4 V^T \\ &= \sigma_1^4 v_1 v_1^T + \sigma_2^4 v_2 v_2^T + \dots \sigma_r^4 v_r v_r^T \end{aligned}$$

and

$$Y^3 = V\Sigma^6 V^T = \sigma_1^6 v_1 v_1^T + \sigma_2^6 v_2 v_2^T + \dots \sigma_r^6 v_r v_r^T.$$

In general, we have

$$Y^l = \sigma_1^{2l} v_1 v_1^T + \sigma_2^{2l} v_2 v_2^T + \dots \sigma_r^{2l} v_r v_r^T.$$

Therefore, if there is a "gap" between σ_1 and the rest of the singular values, then this gap will be amplified as l becomes large. For example, if $\sigma_1 = 1, \sigma_2 = 0.8$ ($\sigma_i \leq \sigma_2$ for $i \geq 3$ by the definition of SVD), then

$$Y^l = 1^{2l} \cdot v_1 v_1^T + (0.8)^{2l} v_2 v_2^T + (\sigma^3)^{2l} v_3 v_3^T + \dots$$

Hence, as $l \rightarrow \infty$, $Y^l \rightarrow v_1 v_1^T$. In particular, if we can look at any column of Y^l and normalize (scale) it, we should obtain a vector that converges to v_1 (since each column of $v_1 v_1^T$ is some constant multiple of v_1). In summary, to extract v_1 from X , we can use the following idea:

1. Compute Y^l for some large l ($l = 100$).
2. Extract the first column of Y^l by letting

$$\bar{v} = Y^l e_1$$

where e_1 is the vector with a 1 in the first entry and 0's in all the other entries. This extracts the first column of Y^l .

3. Output $\frac{\bar{v}}{\|\bar{v}\|}$.

This has a potential issue as computing matrix-matrix products can be really expensive. In the implementation above, we first perform l matrix multiplications then multiply by v . We can save a lot of running time by changing the order of computation: we can compute $Y^l v$ as $Y^l v = Y^{l-1}(Yv) = Y^{l-2}(Y(Yv)) = \dots$ See Figure 38. By doing this, we are now performing l matrix-vector multiplications instead of l matrix-matrix multiplications.

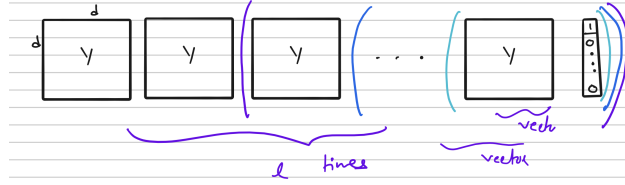


Figure 38: The matrix multiplication $Y^l v$.

Combining these ideas, we have our first attempt at the **Power Iteration** algorithm.

Power Iteration (naive):

1. Compute $Y = X^T X$. Let $v_0 = e_1$.
2. For $t = 1, 2, \dots, l$:

$$v_t = Y v_{t-1}$$

3. Output $\frac{v_l}{\|v_l\|}$.

However, we might run into the issue that the first column of Y is always 0, i.e., Y may look like

$$\begin{pmatrix} 0 & 0 & 0 & 0 & \dots & 0 \\ 0 & * & * & * & \dots & * \\ 0 & * & * & * & \dots & * \\ \dots & & & & & \\ 0 & * & * & * & \dots & * \end{pmatrix}.$$

This can happen even if our first singular vector is non-zero. To circumvent this problem, instead of relying on our choice of e_1 , we will start with a random unit vector in $\mathbb{R}^d$. By doing this, we can avoid the adversarial 0-first-column issue with overwhelming probability. With this issue fixed, now we are ready to state

the actual Power Iteration algorithm.

Power Iteration

- Input: $X \in \mathbb{R}^{n \times d}$
 - Output: A vector v that is close to the first right singular vector.
1. Let $\bar{v}_0$ be a random unit vector in $\mathbb{R}^d$.
 2. For $t = 1, 2, \dots, l$:
 - Set $Z_t = X^T(X\bar{v}_{t-1})$
 - Let $\bar{v}_t = \frac{Z_t}{\|Z_t\|}$.
 3. Output $\bar{v}_l$.

We have the following guarantee for the Power Iteration algorithm.

Theorem 10.5. (Convergence of Power Iteration) Power Iteration will converge to a top right singular vector after $L = O(\frac{\log d}{\epsilon})$ iterations. More specifically, we will have $\|X\bar{v}_L\| \geq (1 - \epsilon)\sigma_1$ with high probability.

Recall the v_1 is the first principal component which by definition, maximizes $\frac{\|Xv\|}{\|v\|}$ and $\max_v \frac{\|Xv\|}{\|v\|} = \sigma_1$. Now let $\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r$ be the singular values of X such that $\sigma_m \geq (1 - \epsilon)\sigma_1$ and $\sigma_{m+1} < (1 - \epsilon)\sigma_1$ for some $m \in \{1, 2, \dots, r\}$. Let $S_m = \text{span}\{v_1, v_2, \dots, v_m\}$, i.e., the span of the first m singular vectors. Then the following theorem holds.

Theorem 10.6. After L iterations of Power Iterations where $L = O(\frac{\log(1/\delta)}{\epsilon})$, $\bar{v}_L$ is almost entirely contained in S_m , i.e.,

$$\|\bar{v}_L - \text{Proj}_{S_m}(\bar{v}_L)\| \leq \delta.$$

One thing that is particularly nice about Power Iterations is that you do not really need to perform any matrix multiplications at all. For instance, suppose we are given three matrices X_1, X_2, X_3 and we are interested in computing the first principal component of $X = X_1X_2X_3$. If X_1, X_2, X_3 are of large dimensions, say $100k \times 100k$, then it would be incredibly inefficient if we have to carry out matrix multiplications. However, in Power Iteration, we can get around this as

$$\begin{aligned} X^T(X\bar{v}_{t-1}) &= (X_1X_2X_3)^T((X_1X_2X_3)\bar{v}_{t-1}) \\ &= (X_3^T(X_2^T(X_1^T(X_1(X_2(X_3\bar{v}_{t-1})))))). \end{aligned}$$

Consequently, to run Power Iteration on X , we only need a way to multiply vectors with X .

10.3.1 Variants of Power Iterations

Just like the NAGD method we learned in our Optimization module, we can use the same idea to accelerate Power Iterations.

Power Iteration + Momentum ("Lanczos Iteration")

1. Let v_0 be a random unit vector
2. For $t = 1, 2, \dots, l$:
 - Set $\bar{z}_t = X^T X \bar{v}_{t-1} - \beta_t \cdot \bar{v}_{t-2}$
 - Let $\bar{v}_t = \frac{\bar{z}_t}{\|\bar{z}_t\|}$.

We know from the above that Power Iterations converge in $\frac{1}{\epsilon}$ rounds. In comparison, Power Iteration with Momentum converges in $\frac{1}{\sqrt{\epsilon}}$ rounds.

In other cases, we are not just interested in the first principal component. We may also be interested in the top k singular vectors. In this case, we can do the following:

Subspace Iteration

1. Let $v_1^0, v_2^0, \dots, v_k^0$ be a set of orthonormal vectors.
2. For $t = 1, 2, \dots, l$:
 - Let $z_i^t = X^T(Xv_i^{t-1})$ for $i = 1, 2, \dots, k$.
 - Set $v_1^t = \frac{z_1^t}{\|z_1^t\|}, \dots, v_k^t = \frac{z_k^t}{\|z_k^t\|}$.

Lastly, instead of computing the first PC, what if we want to compute the last non-zero Principal Component? We would like some way of "reversing" the magnitude of the singular values and then run Power Iterations. Since the singular values are now in "reverse order," Power Iteration should give me the PC associated with the largest singular value, which in this case, is the last non-zero PC since the magnitudes have been "reversed." To achieve this, we know

$$Y = X^T X = \sigma_1^2 v_1 v_1^T + \sigma_2^2 v_2 v_2^T + \dots + \sigma_r^2 v_r v_r^T.$$

Then,

- Run Power Iteration to compute σ_1 .
- Set $\tilde{Y} = 2\sigma_1^2 I_V - Y$ where $I_V = v_1 v_1^T + \dots + v_r v_r^T$. We can compute this in some way but for now, for simplicity, let's assume X is full rank. Then I_V would just be equal to I . We thus find

$$\begin{aligned} \tilde{Y} &= 2\sigma_1^2(v_1 v_1^T + \dots + v_r v_r^T) - (\sigma_1^2 v_1 v_1^T + \dots + \sigma_r^2 v_r v_r^T) \\ &= (2\sigma_1^2 - \sigma_1^2)v_1 v_1^T + (2\sigma_1^2 - \sigma_2^2)v_2 v_2^T + \dots + (2\sigma_1^2 - \sigma_r^2)v_r v_r^T. \end{aligned}$$

- Run Power Iteration on $\tilde{Y}$.

Notice that $\tilde{Y}$ has the same principal components as Y but the singular values associated with $v_r v_r^T$ is $(2\sigma_1^2 - \sigma_r^2)$ which is the largest singular value of $\tilde{Y}$. As a result, running Power Iteration on $\tilde{Y}$ recovers the first PC of $\tilde{Y}$ which gives us v_r , the last non-zero PC of Y .

10.4 Matrix Completion

Now going back to the Netflix Challenge. We can beat the benchmark by at least 4% by first do some preprocessing of the data, then perform low-rank approximation, and lastly do some postprocessing before outputting the final result.

Another way of seeing the problem is to view it as the following optimization problem:

$$\min \sum_{(i,j) \text{ seen}} (X_{ij} - \tilde{X}_{ij})^2 \text{ such that } \text{rank}(\tilde{X}) \leq 30.$$

This is a constrained optimization problem so we would naturally like to run Projected Gradient Descent on the problem. However, the bad news is that the rank constraint is not convex. But as it turns out, in this case, although the constraint is non-convex, we can still perform the projection computation efficiently and carry out with the Projected Gradient Descent method. This is known as the "Singular Value Projection."

To put the algorithm into context, recall the setup of the Netflix Challenge. We are given a large $n \times d$ matrix that contains the reviews of n users on d different movies. Let $\Theta \subseteq [n] \times [d]$ be the set of seen entries. For the Netflix Challenge, very few entries are seen, for instance, $\frac{|\Theta|}{nd} \approx 0.1$. We can phrase the challenge as the *Matrix Completion Problem*:

- Input: Entries of X in Θ
- Output: Find $\bar{X}$ of rank $\leq k$ that minimizes

$$\sum_{i,j \in \Theta} (x_{ij} - \hat{x}_{ij})^2.$$

If we define $L(Y) = \sum_{(i,j) \in \Theta} (X_{ij} - Y_{ij})^2$, i.e., the Frobenius norm of $X - Y$, then we can write the Matrix Completion Problem simply as:

$$\min_{Y: \text{rank}(Y) \leq k} L(Y). \quad (25)$$

One thing we can do is to view $\|X - Y\|_F$ as a way of measuring the distance between X and Y in the space of matrices. By doing so, equation (25) is equivalent to finding the matrix Y that is the "closest" to X in the space of matrices of rank $\leq k$. Borrowing terminologies from earlier, we are trying to find the projection of X onto the space of rank $\leq k$ matrices. Define

$$\text{Proj}(Y) = \arg \min_{Z: \text{rank}(Z) \leq k} \|Z - Y\|_F^2.$$

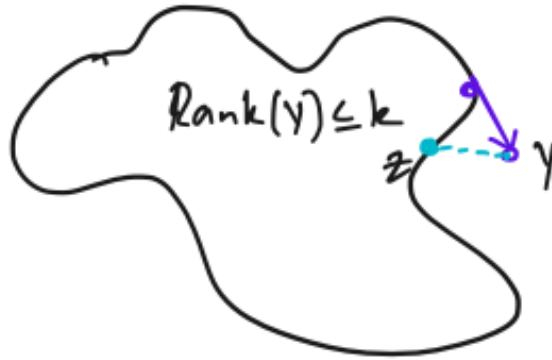


Figure 39: Projection of Y onto the space of matrices of rank $\leq k$.

As it turns out, we can solve this problem by combining SVD with gradient descent.

Singular Value Decomposition (JMD09)

- Preprocess X
- Set Y_0 to be the SVD of X
- For $t = 1, \dots, t$:
 1. Let $Y'_t = Y_{t-1} - \eta \nabla L(Y_{t-1})$
 2. Set Y_t to be the best rank- k approximation to Y'_t by SVD.

This algorithm works well for random matrices and randomly chosen $\Theta \subseteq [n] \times [d]$ even we only have $\approx (n+d)k \log(nd)$ seen entries. One concern one might possibly have is the memory needed to keep track of the matrix Y'_t at each stage. As mentioned earlier, it would be too memory inefficient if we need to record the whole $n \times d$ matrix at all time. But fortunately, we do not need to worry about that problem for this algorithm.

On one hand, the matrix $\nabla L(Y_{t-1})$ is a "sparse" matrix since ∇L is zero for entries not in Θ so we only need to keep track of the non-zero entries which are not too many by our assumption. On the other hand,

Y_t is low-rank by definition. Thus, we can just store Y_t in its low-rank decomposition (SVD) form which is very memory-efficient.

Combining these two facts, indeed it does not take too much memory to store Y'_t which is the sum of a low-rank matrix and a sparse matrix.

Part IV

Privacy

Lecture 11. Privacy in Machine Learning

11.1 Introduction

Starting this lecture, we will be discussing privacy-related problems in Machine Learning. In the last part of the class, we talked about the Netflix Challenge where the challenge is to complete the matrix with access only to a small number of entries. Not too long after the first challenge, they launched Netflix Challenge 2.0 with the similar setup and a 10 million dollars reward to the teams that can beat the benchmark by 10%. While many teams were tempted by the prize, the challenge was soon canceled. As it turns out, by releasing dataset, there are serious privacy-violation issues.

One might ask: why can there be privacy concerns when the usernames and ids are hidden in the public dataset? As a study from 2008 by Narayanan and Shmatikov shows, one can identify the exact user with the help of other publicly available datasets.

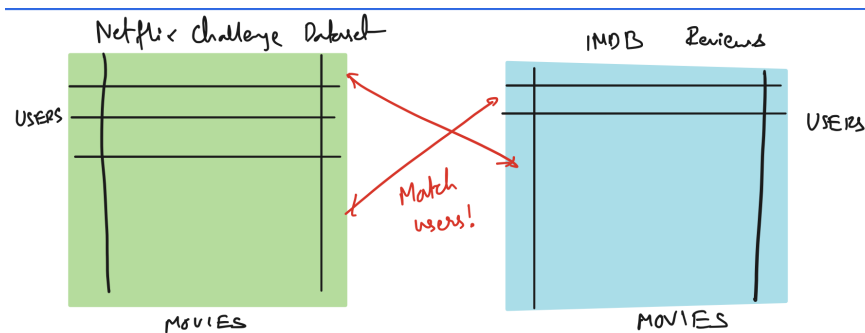


Figure 40: One can cross-reference the Netflix Challenge dataset and the publicly available IMDB reviews dataset to identify the users.

As another example of privacy concerns arising in Machine Learning, suppose we have a dataset that contains identifying information of an individual such as address, SSN, DOB, etc along with the relevant medical information, e.g., height, weight, diabetic or not... This is an important dataset for medical research but as it contains all the identifying information, it obviously have some serious privacy-violating issues. So one might ask, what if we just remove all the identifying information and permute the rows? It seems plausible that this can resolve the privacy issues. However, not long after the "cleaned" dataset was released, an email was sent to the governor containing all the correct medical records of the governor deduced from the dataset. So even if we hide the identifying information, one may still be able to recover the exact user / data correspondence with the help of other publicly available data. This is known as the "reconstruction attack."

Name	Email	Address	DOB	SSN	Hair	Weight	Diabetes	-	-	-

. William Weld's data was recovered & mailed to the governor.

Figure 41: Reconstructing governor’s medical records despite hidden identifying information.

As the examples above shown, it is difficult to say anything is 100% private against all empirical attacks. Hence, it is important to have a model based on which we discuss the notion of privacy. There are so many datasets that are sensitive in real life and people build large-scale machine learning models on datasets such as GPS data, Financial datasets, app usage time, medical data, genetic data, facial recognition data etc. We will formally define the notion of privacy used in Machine Learning shortly. Before that, let’s consider one simple example in machine learning: mean estimation.

11.2 Mean Estimation

Consider the problem of computing averages. In the simplest setting, suppose there are n people in a group and each possesses a binary value, i.e., the i th person possesses some $x_i \in \{0, 1\}$. Our goal is to estimate / calculate the average

$$P = \frac{x_1 + x_2 + \dots + x_n}{n}.$$

Obviously, if we ask each individual to tell us the exact number they have in hand, then we can compute P exactly. However, the issue is that the algorithm would know the information of all the individuals and thus could be a major violation of privacy. So the question is, is there a way of getting a good estimate of the true mean without conveying the exact information each individual has?

Back in 1965, Warner came up with a simple idea of using *Randomized Response*. For each user i , they will send to us a value $\bar{x}_i$ such that

$$\bar{x}_i = \begin{cases} x_i & \text{with prob } \frac{1}{2} + \delta \\ 1 - x_i & \text{with prob } \frac{1}{2} - \delta. \end{cases}$$

With this randomization step, the algorithm has no way of knowing the exact x_i that user i has.

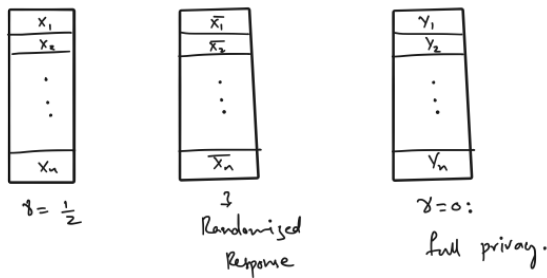


Figure 42: Randomized response: δ is a parameter you can choose. When $\delta = 1/2$, each individual sends the message that have. When $\delta = 0$, each individual sends a completely random message which thus guarantees "full privacy,"

So it seems we can guarantee certain level of privacy by choosing the appropriate δ . But can we still get a good estimate of the true mean in this case? Notice we have

$$\mathbf{E}[\bar{x}_i] = \left(\frac{1}{2} + \delta\right)x_i + \left(\frac{1}{2} - \delta\right)(1 - x_i) = \left(\frac{1}{2} - \delta\right) + 2\delta x_i.$$

In other words,

$$x_i = \frac{\mathbf{E}[\bar{x}_i] - (\frac{1}{2} - \delta)}{2\delta}.$$

This implies

$$\frac{x_1 + \dots + x_n}{n} = \mathbf{E} \left[\underbrace{\frac{\bar{x}_1 + \dots + \bar{x}_n}{n} - (\frac{1}{2} - \delta)}_{\bar{P}} \right].$$

This suggests that on average, $\bar{P}$ is the right estimate for the empirical average. By an standard application of Chebyshev's inequality, one can show:

Claim 11.1. (*Randomized Response for Mean Estimation*)

$$\Pr[|P - \bar{P}| > \epsilon] \leq \frac{1}{4\epsilon\delta\sqrt{n}}.$$

By the claim, suppose we want ϵ -accuracy with at least 75% chance, we would need $\frac{1}{\delta\epsilon\sqrt{n}} \leq 1$ which is equivalent to saying $n \geq \frac{1}{\delta^2\epsilon^2}$. From the example, we can see that we can achieve certain level of privacy while satisfying some accuracy guarantees. There are two questions we need to address here: (1) We need to find a way to formally define accuracy. (2) How does privacy requirements affect the accuracy of our predictors?

11.3 Differential Privacy

Our notion of quantifying privacy will be *Differential Privacy*. This is also called "Central Differential Privacy" or "Trusted Curator Model" as illustrated in the figure below.

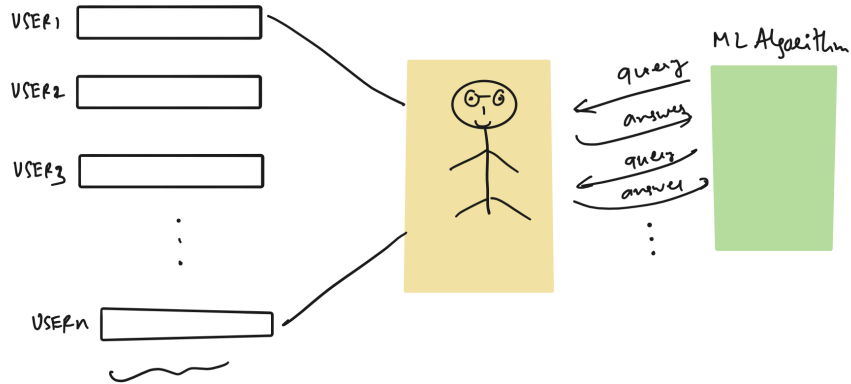


Figure 43: The Differential Privacy model: we have a database containing all the users data. The ML algorithm interacts with the dataset via a "middle" curator that answers each query the ML algorithm makes with processed information (for private reasons) from the dataset.

Formally speaking, given a database $D \in \mathcal{X}^n$ that contains information about the n users, there is a *mechanism* $\mathcal{M} : \mathcal{X}^n \rightarrow Y$ that takes in as input the dataset D and outputs answers for a specific query asked by the ML algorithm. We want a notion privacy such that the mechanism will output similar answers for any "neighboring" datasets.

Definition 11.2. (*Neighboring datasets*) Two datasets D, D' are neighbors if they differ in exactly one row.

Now we can define the notion of *differential privacy*.

Definition 11.3. (ϵ -differentially private) We say a mechanism $\mathcal{M} : \mathcal{X}^n \rightarrow Y$ is ϵ -differentially private if for all neighboring databases D, D' and $y \in Y$, we have

$$e^{-\epsilon} \Pr[\mathcal{M}(D') = y] \leq \Pr[\mathcal{M}(D) = y] \leq e^{\epsilon} \Pr[\mathcal{M}(D') = y].$$

This notion of privacy is widely adopted in practice. For instance, it is the standard of privacy that US Census Bureau 2020 used. It was also adopted by tech companies like Apple, Google, Microsoft and many others. A few remarks about the definition:

- Smaller ϵ means better privacy.
- ϵ is pretty small, for instance, $\epsilon \approx 0.01$.
- For the definition to hold, it has to hold in the worst case for all databases.

Now we examine this notion of privacy for the task of mean estimation with randomized response that we discussed earlier. Recall that to estimate the average $P := (x_1 + \dots + x_n)/n$, we use the estimator

$$M(D) = \frac{\frac{\bar{x}_1 + \dots + \bar{x}_n}{n} - (\frac{1}{2} - \delta)}{2\delta}$$

Now let D, D' be two neighboring databases. Let $a \in Y$ be an arbitrary outcome. Notice that for $\mathcal{M}(D) = \mathcal{M}(D')$ to happen, one of the entries of one dataset has to be flipped from the true value while the same entry is not flipped in the other dataset. This suggests,

$$\frac{\Pr[\mathcal{M}(D) = a]}{\Pr[\mathcal{M}(D') = a]} \leq \frac{\frac{1}{2} + \delta}{\frac{1}{2} - \delta} = \frac{1 + 2\delta}{1 - 2\delta} = 1 + \frac{4\delta}{1 - 2\delta} \approx e^{O(\delta)}$$

for some constant in the exponent. This suggests that Randomized response is in fact, $O(\delta)$ -differentially private. On the other hand, by Claim 11.1, we know

$$\Pr \left[\left| \mathcal{M}(x) - \frac{x_1 + \dots + x_n}{n} \right| \geq \frac{1}{\delta\sqrt{n}} \right] \leq \frac{1}{4}.$$

Let α be the error parameter. Then setting $\alpha = \frac{1}{\delta\sqrt{n}}$ in the above inequality, we thus know that to achieve ϵ -differential privacy and error $\leq \alpha$, we need

$$n \geq \frac{c}{\epsilon^2 \alpha^2}$$

for some constant c .

Lecture 12. The Laplacian Mechanism

Last time we introduced the notion of ϵ -differential privacy and showed that randomized response for mean computation is indeed differentially-private. Recall that in randomized response, instead of letting user report faithfully their true value $x_i \in \{0,1\}$, we ask the users to send $\bar{x}_i$ to the trusted curator such that $\bar{x}_i = x_i$ with probability $(1/2) - \delta$ and $\bar{x}_i = 1 - x_i$ otherwise. The curator then sends the $\bar{x}_i$'s to the algorithm. In other words, we are "perturbing" each user's input to maintain privacy.

But we do not have to do it this way. One alternative way is to perturb the output. Instead of sending noisy version of the inputs, the trusted curator can first compute the true quantity of interest using the faithful data, add some noise to the computed value, then send it to the algorithm.

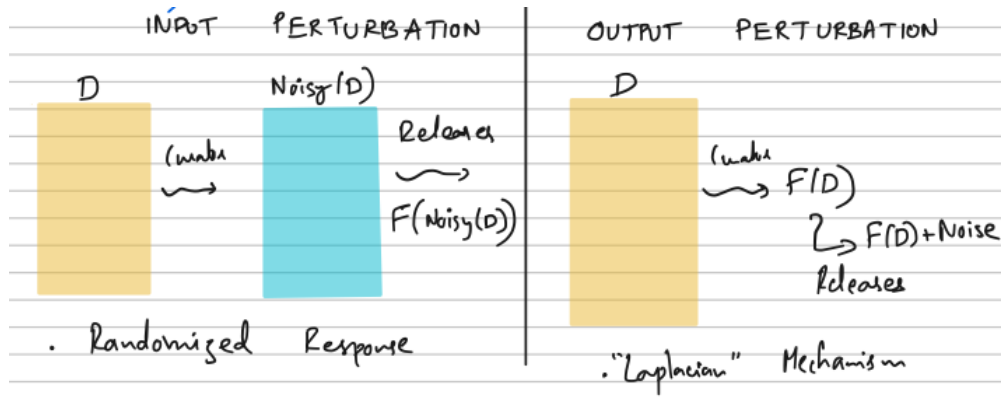


Figure 44: In an input perturbation mechanism (on the left), noise added to the true dataset D to obtain $Noisy(D)$. We then compute the function on this dataset and obtain $F(Noisy(D))$ which is then sent to our algorithm. Meanwhile in an output perturbation mechanism (on the right), we first compute F using the true data and obtain $F(D)$. We then add noise to it to get $F(D) + noise$ which we send to our algorithm.

As an example, in the problem of image generation, you are given a large dataset of images. If you feed in the images directly to your model (algorithm), then one can use the model to generate images and use them to trace the "source" in the dataset which inevitably has privacy concerns. To address this, it was shown that one can add a lot of noise to each pixel of the dataset. The resulting dataset looks like pure noise to human eyes but when it is fed to the model (algorithm), the performance of the resulting model is almost as good as the original one.

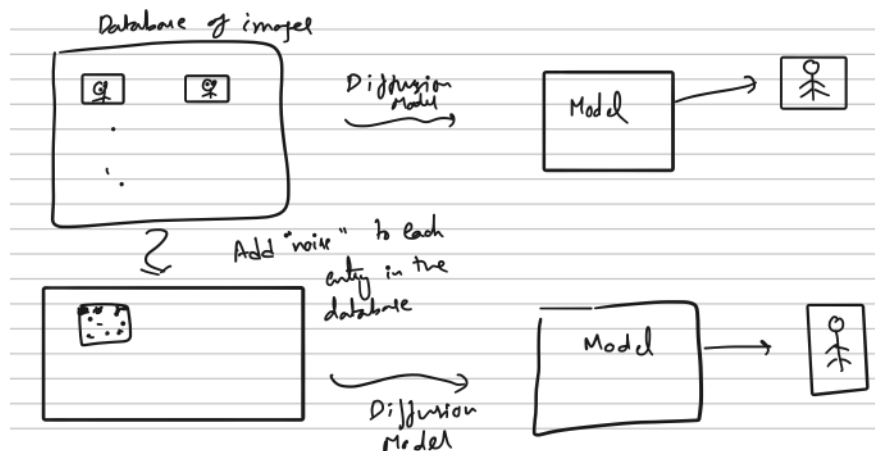


Figure 45: Example of privacy-related concerns in machine learning

12.1 Laplacian Mechanism

We will begin our discussion of the Laplacian Mechanism by again considering the problem of Mean Estimation. Again, in this setting, the input is given as $X = (x_1, \dots, x_n)$ where each $x_i \in \{0, 1\}$. The *Laplacian Mechanism* for computing the mean (an output perturbation mechanism) is as follows:

1. Compute $f(x) = \frac{x_1 + x_2 + \dots + x_n}{n}$
2. Release $f(x) + Z$

where Z is drawn from the Laplacian distribution.

Definition 12.1. (*Laplacian Distribution*) A Laplacian distribution with mean 0 and variance b is the distribution where the probability density function is given by

$$p(z) = \frac{1}{2b} \exp\left(-\frac{|z|}{b}\right).$$

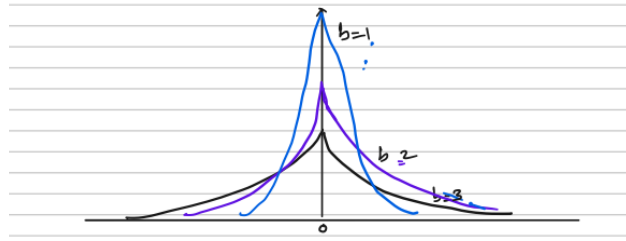


Figure 46: Laplacian distribution for different values of the parameter b .

We now prove that the Laplacian Mechanism for computing the mean is indeed differentially private.

Claim 12.2. The Laplacian Mechanism with $b = 1/(\epsilon n)$ is ϵ -differentially private for computing the mean.

Proof. Let X, X' be neighboring datasets, i.e., datasets that differ in exactly one row. The Laplacian Mechanism computes the estimated mean as $M(X) = f(X) + Z$ and $M(X') = f(X') + Z'$ where Z, Z' are sampled independently from a Laplacian distribution with mean 0 and variance b . Therefore, we know that for any $a \in \mathbb{R}$,

$$\begin{aligned} \Pr[M(X) = a] &= \Pr[f(X) + Z = a] \\ &= \Pr[Z = a - f(X)] \\ &= \frac{1}{2b} \exp\left(-\frac{|a - f(X)|}{b}\right). \end{aligned}$$

Similarly, we know

$$\Pr[M(X') = a] = \frac{1}{2b} \exp\left(-\frac{|a - f(X')|}{b}\right).$$

This suggests

$$\begin{aligned} \frac{\Pr[M(X) = a]}{\Pr[M(X') = a]} &= \exp\left(-\frac{(|a - f(X)| - |a - f(X')|)}{b}\right) \\ &\leq \exp\left(\frac{|f(X) - f(X')|}{b}\right) \\ &\leq \exp\left(\frac{1}{nb}\right) \\ &= \exp(\epsilon), \end{aligned}$$

where the second inequality holds since X, X' are neighboring datasets and each entry takes values in $\{0, 1\}$ (thus the mean of neighboring databases can differ by at most $1/n$). The last equality holds since $b = 1/(\epsilon n)$. $\square$

We have thus shown that the Laplacian Mechanism is indeed ϵ -differentially private. But how about its accuracy? We are releasing our mean estimate as $f(X) + Z$ where $Z \sim \text{Laplacian}(\frac{1}{\epsilon n})$. From standard concentration of measure results, we know for a zero-mean Laplacian distribution with parameter b , the following holds:

$$\Pr[|Z| > tb] \leq \exp(-t)$$

which suggests

$$\Pr[|Z| > 2b] \leq \exp(-2) \leq 1/4.$$

Combining this accuracy guarantee with the privacy guarantee earlier, we thus obtain

Claim 12.3. *Laplacian Mechanism with $b = 1/\epsilon n$ is ϵ -differentially private and*

$$\Pr[|M(X) - f(X)| > \frac{2}{\epsilon n}] \leq \frac{1}{4}.$$

From the above, if we let $\alpha = 2/(\epsilon n)$, then the Laplacian Mechanism for computing the mean ϵ -differentially private and α -accurate with probability at least $3/4$ for $n \geq 2/(\epsilon \alpha)$.

By now we have only considered the problem of mean estimation. In fact, we can make the above argument above more general and show that the Laplacian Mechanism can achieve good privacy and accuracy for a variety of tasks. To this end, let $f : \mathcal{X}^n \rightarrow \mathbb{R}$ be a function that takes in as input a dataset and outputs a real value. Then the sensitivity of f is given by

$$S_1(f) = \max_{X, X' \text{ are neighboring databases}} |f(X) - f(X')|.$$

For instance, for $Mean : \mathcal{X} \rightarrow \mathbb{R}$ which outputs the empirical mean, we have

$$S_1(Mean) = \frac{1}{n}.$$

Using the same proof as above, one can establish the following result.

Theorem 12.4. *(Laplacian Mechanism one-dimensional guarantee) If $f : \mathcal{X} \rightarrow \mathbb{R}$ has sensitivity $S_1(f) \leq s$, then the Laplacian Mechanism with noise level $b = s/\epsilon$ achieves ϵ -differential privacy.*

Note: we cannot get an accuracy guarantee in general as it depends heavily on the exact function f we are computing.

We can further extend this notion of sensitivity for vector-valued functions. Let $f : \mathcal{X}^n \rightarrow \mathbb{R}^k$. The sensitivity of the vector-valued f is defined as

$$S_1(f) = \max_{X, X' \text{ are neighboring datasets}} \sum_{i=1}^k |f(X)_i - f(X')_i|.$$

Hence, to compute $f(X)$ while maintaining some privacy, we can do the similar steps as before:

- Input: $(x_1, \dots, x_n) \in \mathcal{X}^n$
- Output:
 1. Compute $f(X)$
 2. Sample $Z_1, \dots, Z_k \sim \text{Laplacian}(b)$
 3. Output $f(X) + (Z_1, \dots, Z_k)$.

We have the following privacy guarantee:

Theorem 12.5. *(Laplacian Mechanism high-dimensional guarantees) If $f : \mathcal{X}^n \rightarrow \mathbb{R}^k$ has sensitivity $\leq s$, then the Laplacian Mechanism with noise $b \geq s/\epsilon$ achieves ϵ -differential privacy.*

12.2 Properties of Differential Privacy

12.2.1 Meta Processing

In most practical applications, computing a single function M is not the end of the story. Most likely, we are going to use the computed value M and use it to compute other quantities of interest.

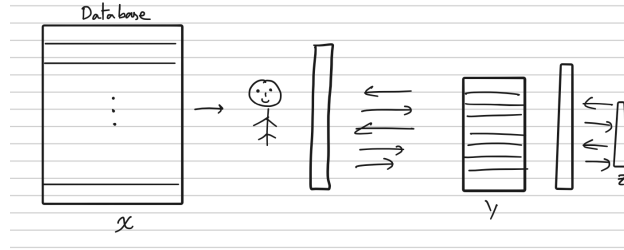


Figure 47: After Y receives the computed $M(X)$ from the trusted curator, it may use the resulting $M(X)$ and compute some other values and send them to Z .

One may wonder, does differential privacy still holds in this case? In other words, is it privacy preserved under function compositions? Fortunately, the answer is yes.

Theorem 12.6. (*DP is preserved under composition*) Let $M : \mathcal{X}^n \rightarrow \mathcal{Y}$ be ϵ -differentially private. Then, for any $F : \mathcal{Y} \rightarrow \mathcal{Z}$, the composition $F \circ M : \mathcal{X}^n \rightarrow \mathcal{Z}$ is also ϵ -differentially private.

Proof. Let $\mathcal{X}, \mathcal{X}'$ be neighboring datasets. Then for all $z \in \mathcal{Z}$, we have

$$\begin{aligned} \Pr[F \circ M(X) = Z] &= \sum_{Y: F(Y)=Z} \Pr[M(X) = Y] \\ &\leq \sum_{Y: F(Y)=Z} e^\epsilon \Pr[M(X') = Y] \\ &= e^\epsilon \sum_{Y: F(Y)=Z} \Pr[M(X') = Y] \\ &= e^\epsilon \Pr[F \circ M(X') = Z], \end{aligned}$$

where the inequality holds since M is ϵ -differentially private. $\square$

12.2.2 Group Privacy

What if now instead, we have two databases X, X' that differ in k entries instead of only one entries, what can we say about the differential privacy in this case? The answer is we still have DP-guarantees and in fact, if $M : \mathcal{X}^n \rightarrow \mathcal{Y}$ is ϵ -DP, then

$$\Pr[M(X) = Y] \leq e^{k\epsilon} \Pr[M(X') = Y].$$

This can be proof via a "Hybrid Method." See the figure below.

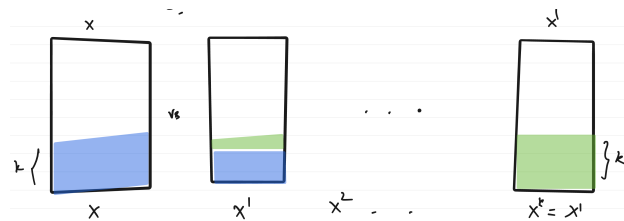


Figure 48: Create "intermediate" databases $X_0, X_1, \dots, X_{k-1}, X_k$ such that $X_0 = X, X_k = X'$ and X_i, X_{i+1} differ in exactly one entry. The result then follows by applying ϵ -differential privacy to X_i, X_{i+1} .

12.2.3 Some example of Queries

There are a few queries that are heavily used in practice.

- **Counting Queries:** this query asks about the number of users or records that have certain properties. It is easy to see that

$$S_1(\text{counting query}) \leq 1.$$

As a result, we can add $Lap(1/\epsilon)$ noise to obtain ϵ -differential privacy.

- **Histogram Queries:** suppose each user falls into a category, then this queries ask about what categories each user fall in (here each user can only fall in one category). We can view a histogram query as $f : \mathcal{X}^n \rightarrow \mathbb{R}^k$ which maps a dataset to a vector of length k such that the i th entry records the number of users that fall in the i th category. Since the total number of users is fixed, neighboring databases can have a change in two coordinates and thus,

$$S_1(\text{histogram query}) \leq 2.$$

See Figure 49.

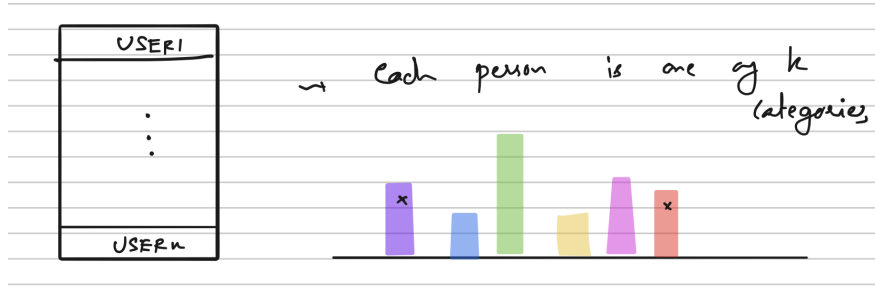


Figure 49: Histogram Query

From the above, we see that although a histogram query has k coordinates in the output, its sensitivity is very small. Thus, histograms are very nice high-dimensional queries.

- **Maximum queries:** Now consider a slightly different scenario. Suppose we have a database of medical records. We want to know what is the most common form of series health hazard out of the k options. In this case, our histogram query can be viewed as $f : \mathcal{X}^n \rightarrow \mathbb{R}^k$ where the i th coordinate counts how many of the users are of type i . To guarantee differential privacy, one thing we can do is to use the following algorithm:

1. Release $f(X)$ in a ϵ -differentially private manner.
2. Then compute which i has the highest count.

Notice that the final output will still be differentially private by the post-processing property of differential private. On the other hand, since in this case, each user does not necessarily fall in only one category (a user can have multiple health hazards [sad]), in this case, changing the data of one user can possibly leads to a change in all of the coordinates. This suggests

$$S_1(f) \leq k.$$

Hence, we need the noise level to be $Lap(k/\epsilon)$ to get ϵ -differential privacy.

As argued above, this algorithm is certainly differentially-private. However, in this case, we are releasing a differentially-private version of $f(X)$ which is a lot of data to send. Since we are only asking for one single value $g(X) = \arg \max_i f(X)_i$, is it possible to output only one value while still maintaining the differential privacy guarantee? The answer is yes. Consider the following "Noisy Max" algorithm:

1. Compute $f(X)$.
2. Add noise $Lap(c/\epsilon)$ to each coordinate of $f(X)$.
3. Compute the max of these noisy counts and release the id of the disease with the noisy max.

One can show that the above is indeed ϵ -differentially private for computing $g(X)$. We are not going to prove this. Interested readers can refer to Claim 3.9 in <https://www.cis.upenn.edu/~aaroht/Papers/privacybook.pdf> for more details.

Lecture 13. The Exponential Mechanism and other topics in DP

Last time we discussed the Laplacian Mechanism and some related examples. Today we are going to discuss another mechanism for a slightly different setting.

13.1 The Exponential Mechanism

Suppose someone is holding a digital auction for a digital painting somewhere online. Since this is a digital auction, the auctioned item does not only go to the one who has the highest bid. Instead, any participant who is willing to pay the set price will receive copies of the item and they participants who get the copies pay the amount that is equal to the set price.

For instance, suppose we have four users bidding for one item. User 1 bids (is willing to pay) 1 dollars. User 2 bids 2.5 dollars. User 3 bids 1 dollar. User 4 bids 3 dollars. In this case, if the auctioneer sets the item price to be 1 dollars, then he would collect $4 \times 1 = 4$ dollars since every bidder is willing to pay at least 1 dollar for the item. Similarly, if the auctioneer sets the price to be 2.5 dollars, then he would collect $2 \times 2.5 = 5$ dollars since only User 2 and User 4 are willing to pay more than the the price.

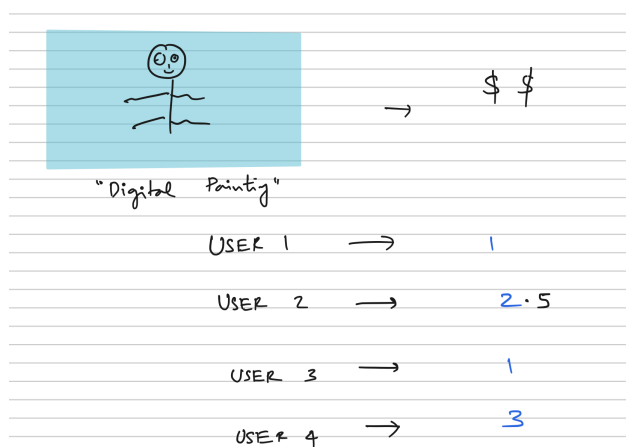


Figure 50: Digital good auction: users' bidding prices

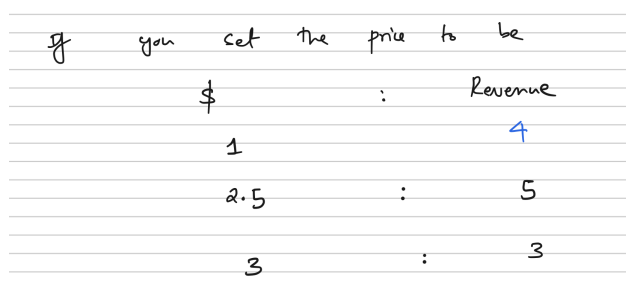


Figure 51: Digital good auction: auctioneer's set prices and the corresponding revenues.

Let $Revenue(p)$ denote how much revenue the auctioneer would collect at price p . Then we would like to know what is the price at which revenue is maximized, i.e., what is $\arg \max_P Revenue(P)$. To state this more formally and more generally, let P be the range of prices. Let $U : \mathcal{X}^n \times P \rightarrow \mathbb{R}$ be a "utility function" such that $U(X, p)$ denotes the utility / revenue the auctioneer collects at price p . Our goal is to compute and release $\arg \max_p U(X, p)$.

Many real life scenarios fall into this general problem framework.

Example 13.1. • *Digital goods auction is a special case of this problem. In this case,*

$$U(X, p) = \text{Revenue generated at price } p = p \cdot (\# \text{ of users whose bid is } \geq p)$$

where X is a vector of the bids of all the users.

- *Computing the most frequent item can also be formulated in the same manner. Let $U : \mathcal{X}^n \times \{1, 2, \dots, k\} \rightarrow \mathbb{R}$ such that*

$$U(X, i) = \# \text{ of people who have type } i.$$

The most frequent item is thus $\arg \max_i U(X, i)$.

Our goal now is to compute / release $U(X, p)$ privately so as not to reveal any information about each user's bid. We will use The Exponential Mechanism for this task.

Exponential Mechanism: Given $U : \mathcal{X}^n \times P \rightarrow \mathbb{R}$, $M_E(X)$ selects and outputs $p \in P$ with probability proportional to $\exp(\lambda U(X, p))$. In other words,

$$\Pr[M_E(X) = p] = \frac{\exp(\lambda U(X, p))}{\sum_{q \in P} \exp(\lambda U(X, q))}.$$

Here $\lambda = \frac{\epsilon}{2s(u)}$ where $s(u) = \max_p \max_{X, X' \text{ neighboring datasets}} |U(X, p) - U(X', p)|$.

Recall the definition of ϵ -differential privacy: $M : \mathcal{X} \rightarrow \mathcal{Y}$ is ϵ -differentially private if for all neighboring databases X, X' and $y \in \mathcal{Y}$, we have

$$\Pr[M(X) = y] \leq e^\epsilon \Pr[M(X') = y].$$

Theorem 13.2. *The Exponential Mechanism M_E for outputting $\arg \max_p U(X, p)$ is ϵ -differentially private.*

Proof. Let X, X' be two neighboring databases and $p \in P$. We want to compare the quantity

$$\Pr[M_E(X) = p] = \frac{\exp(\lambda U(X, p))}{\sum_{q \in P} \exp(\lambda U(X, q))}$$

with

$$\Pr[M_E(X') = p] = \frac{\exp(\lambda U(X', p))}{\sum_{q \in P} \exp(\lambda U(X', q))}.$$

First, we compare the numerators:

$$\frac{\exp(\lambda U(X, p))}{\exp(\lambda U(X', p))} = \exp(\lambda(U(X, p) - U(X', p))) \leq \exp(\lambda s(u)) = \exp\left(\frac{\epsilon}{2}\right),$$

where the inequality holds by the definition of $s(u)$. Now for the denominators, by the above inequality, we have

$$\sum_{q \in P} \exp(\lambda U(X, q)) \geq \sum_{q \in P} e^{-\epsilon/2} \exp(\lambda U(X', q)) = e^{-\epsilon/2} \sum_{q \in P} \exp(\lambda U(X', q)).$$

Combining the two inequalities, we thus know

$$\Pr[M_E(X) = p] = \frac{\exp(\lambda U(X, p))}{\sum_{q \in P} \exp(\lambda U(X, q))} \leq e^\epsilon \frac{\exp(\lambda U(X', p))}{\sum_{q \in P} \exp(\lambda U(X', q))} = \Pr[M_E(X') = p]$$

as desired. □

We have thus proved the privacy guarantee of the Exponential Mechanism. However, the mechanism would be useless if its output is very far from the true value. In other words, we also want it to satisfy a good accuracy guarantee. Fortunately, this is indeed the case. Recall we want to compute $\max_p U(X, p) = \text{OPT}_U(x)$. We have the following guarantee.

Theorem 13.3. (*Accuracy of the Exponential Mechanism*) Let $M_E(X)$ be the price outputted by the Exponential Mechanism. Then we have,

$$\Pr \left[U(X, M_E(X)) \geq \text{OPT}_U(X) - \frac{s(u)}{\epsilon} [\ln(|P|) + t] \right] \geq 1 - e^{-t},$$

where $|P|$ is the number of prices / categories in consideration.

Here is a quick summary of what we have covered so far:

- We defined the notion of ϵ -differential privacy. As we will see a bit later, this is a form of "Pure DP."
- We talked about the Laplacian Mechanism which provides nice privacy guarantees depending on the sensitivity $s_1(f)$ of the function.
- We also discussed the Exponential Mechanism. This gives us the desired the privacy when we are maximizing certain functions.

13.2 Approximate Differential Privacy

Recall we say M is ϵ -differentially private if for any two neighboring databases X, X' , we have $\Pr[M(X) = y] \leq e^\epsilon \Pr[M(X') = y]$. This is a nice property to have in many many cases. But this requirement maybe too restrictive in some cases. For instance, it requires the privacy guarantee to hold in a very strong multiplicative sense even when the probability of a value, i.e., $\Pr[M(X) = y]$ is very small. In many practical applications, we do not really want to pay too much attention for situations that are extremely unlikely to occur while sacrificing some accuracy guarantees. One way to resolve this issue is to relax the differential constraint a bit.

Definition 13.4. ((ϵ, δ) -differentially private) We say a mechanism $M : \mathcal{X}^n \rightarrow Y$ is (ϵ, δ) -differentially private if for any two neighboring databases X, X' and any $y \in Y$, we have

$$\Pr[M(X) = y] \leq e^\epsilon \cdot \Pr[M(X') = y] + \delta.$$

Because of this relaxed constraint, this notion of privacy is thus much easier to be satisfied by, for instance, the following trivial mechanism.

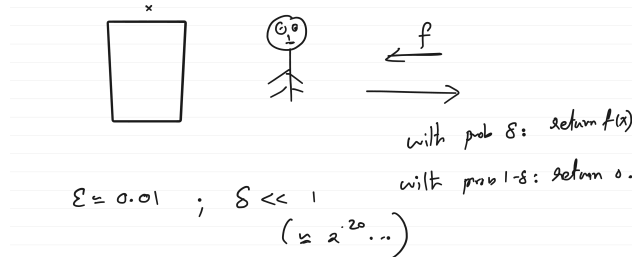


Figure 52: A trivial mechanism that satisfies the (ϵ, δ) -privacy constraint. Since we allow us to violate the privacy with probability at least δ , we can just sent $f(X)$ with δ and 0 otherwise. In practice, we often send ϵ to be say 0.01 and δ to be much smaller, say 2^{-10} .

This trivial mechanism certainly have bad accuracy guarantees. But since the approximate / relaxed notion of privacy is easier to satisfy, does it mean that we have more "wiggle room" to obtain better accuracy? In other words, suppose for a mechanism M achieves ϵ -DP and α -accuracy for $n \geq N$ and another mechanism M' can achieve (ϵ, δ) -DP and α -accuracy with $n \geq N'$ samples. How does N compare to N' ?

To address this question, let's first recall from earlier the guarantees of the Laplacian Mechanism. Given $f : \mathcal{X}^n \rightarrow \mathbb{R}^k$, recall the L_1 sensitivity $S_1(f)$ is defined as $S_1(f) := \max_{X, X'} \sum_{i=1}^k |f(X)_i - f(X')_i|$ where X, X' are neighboring databases. Then we know the Laplacian Mechanism with $\text{Lap}(S_1(f)/\epsilon)$ noise is ϵ -differentially private. Similar to this notion sensitivity, we will define a related but different notion of sensitivity as follows.

Definition 13.5. (*L₂-sensitivity*) For a function $f : \mathcal{X}^n \rightarrow \mathbb{R}^k$, we define the *L₂-sensitivity* of f , denoted $S_2(f)$ as

$$S_2(f) = \max_{X, X'} \|f(X) - f(X')\|_2 = \max_{X, X'} \left(\sum_{i=1}^k |f(X)_i - f(X')_i|^2 \right)^{1/2},$$

where X, X' are neighboring databases.

Example 13.6. Consider the example of computing the mean again. Let $X = \{x^1, \dots, x^n\}$ such that $x^i \in \{0, 1\}^d$ and $f : (\{0, 1\}^d)^n \rightarrow \{0, 1\}^d$ is the mean $f(X) = \frac{1}{n} \sum_{i=1}^n x^i \in \mathbb{R}^d$. In this case, if we let X, X' be two neighboring databases, then

$$S_1(f) = \max_{X, X'} \sum_{i=1}^d |f(X)_i - f(X')_i| = \max_{X, X'} \frac{1}{n} \sum_{i=1}^d \left| \sum_{j=1}^n x_i^j - x_i'^j \right| \leq \frac{1}{n} \sum_{i=1}^d 1 = \frac{d}{n}.$$

On the other hand,

$$S_2(f) = \max_{X, X'} \left(\sum_{i=1}^d |f(X)_i - f(X')_i|^2 \right)^{1/2} = \max_{X, X'} \left(\sum_{i=1}^d \left| \frac{\sum_{j=1}^n x_i^j - x_i'^j}{n} \right|^2 \right)^{1/2} \leq \left(\frac{1}{n^2} \sum_{j=1}^d 1 \right)^{1/2} = \frac{\sqrt{d}}{n}.$$

Instead of adding Laplacian noise as in the case of pure DP, we consider the following mechanism in which we add Gaussian noise.

Gaussian Mechanism: given a function $f : \mathcal{X}^n \rightarrow \mathbb{R}^k$,

- Sample $(Z_1, \dots, Z_k)$ from i.i.d $\mathcal{N}(0, \sigma^2)$.
- Output $f(X) + (Z_1, \dots, Z_k)$.

The Gaussian Mechanism has the following guarantee.

Theorem 13.7. (*Guarantees of the Gaussian Mechanism*) Let $f : \mathcal{X}^n \rightarrow \mathbb{R}^k$. Then Gaussian Mechanism with

$$\sigma \geq \sqrt{2 \ln \left(\frac{1.25}{\delta} \right)} \cdot \frac{S_2(f)}{\epsilon}$$

is (ϵ, δ) -DP.

Now let's say we have a d -dimensional dataset. To achieve Pure DP, Laplacian Mechanism needs noise $\approx d/(\epsilon n)$. To achieve approximate DP, Gaussian Mechanism needs noise $\approx \sqrt{d}/(\epsilon n)$. Hence, if we want to compute mean to accuracy α in each coordinate, for ϵ -DP, we need

$$n \geq \frac{d}{\epsilon \alpha},$$

while for (ϵ, δ) -DP, we need

$$n \geq \frac{C \sqrt{d \log(1/\delta)}}{\epsilon \alpha}.$$

In other words, we need fewer data to obtain approximate DP than pure DP.

13.2.1 Properties of Approximate-DP

As with Pure-DP, we also have some nice meta-properties of Approximate-DP.

Lemma 13.8. (*Group Privacy of Approximate-DP*) Suppose X, X' are two databases that differ in k rows and $M : \mathcal{X}^n \rightarrow \mathcal{Y}$ is (ϵ, δ) -differentially private, then

$$\Pr[M(X) = y] \leq e^{k\epsilon} \Pr[M(X') = y] + k \cdot e^{k\epsilon} \cdot \delta.$$

Compare this with the group privacy for Pure DP: recall for ϵ -DP, we have $\Pr[M(X) = y] \leq e^{k\epsilon} \Pr[M(X') = y]$. Another nice property of pure-DP is the composition theorem. Recall if $M_1, \dots, M_k$ where $M_1 : \mathcal{X}^n \rightarrow \mathcal{Y}_1$, $M_2 : \mathcal{X}^n \rightarrow \mathcal{Y}_2, \dots, M_k : \mathcal{Y}_{k-1} \rightarrow \mathcal{Y}_k$ are ϵ -DP, then $M_1 \circ M_2 \circ \dots \circ M_k$ is $k\epsilon$ -DP. Fortunately, the composition theorem also holds for approximate-DP.

Lemma 13.9. (*Composition Theorem for approximate-DP*) If $M_1, \dots, M_k$ are (ϵ, δ) -DP, each with the approximate domain and codomain, then

$$M_1 \circ M_2 \circ \dots \circ M_k \text{ is } (k\epsilon, k\delta) - \text{DP}.$$

In fact, one can show a stronger version of the Composition theorem holds for approximate-DP.

Theorem 13.10. (*Advanced Composition theorem*) If $M_1, M_2, \dots, M_k$ are (ϵ, δ) -DP, then their composition is

$$\left(\sqrt{2k \ln \left(\frac{1}{\delta} \epsilon + k\epsilon^2 \right)}, \delta \right) - \text{DP},$$

which is roughly $(C\epsilon\sqrt{k \ln \frac{1}{\delta}}, \delta)$ -DP. In other words, this is saying we can even have better privacy by composing unless $k \approx 1/\epsilon^2$.

13.3 Some Practical Scenarios for DP

13.3.1 Adaptive Data Analysis

Consider the following scenario: we have a database containing information of certain individuals. Although we do not directly have access to the dataset, we can ask questions (queries) about the dataset and the dataset would respond with answers.

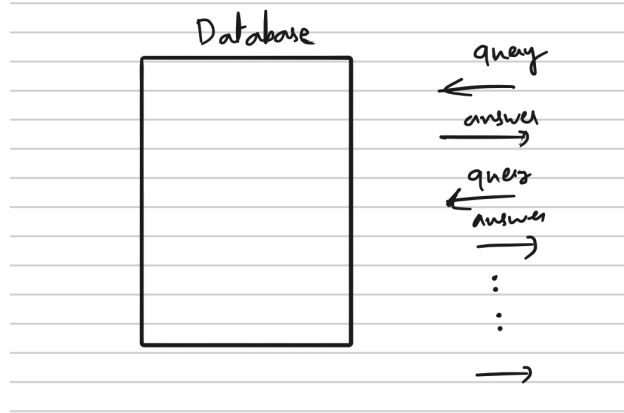


Figure 53: Adaptive Data Analysis: we can query the database and the database would respond with answers to our queries.

There are a couple of questions we would like to address here.

1. How accurate are the answers to each query? Denote the accuracy as α .
2. How many questions (queries) are we asking? Denote the number of queries to be k .

In summary, we want to know: if we want all our k questions to have "good" answers (to accuracy α), how many data points do I need? We say the queries are adaptive if the next question you ask depends on the answer to the previous questions. For instance, suppose you obtain a sample of 100 people's data in Los Angeles. If you first ask how tall the people are, you might get answer like "most people are over 6'5." Knowing this answer, you then adaptively ask "how many of your play basketball/volleyball?" Then you would likely get obtain answers that tell you many of them play these sports. If you use this as a basis for

your conclusion that many many people in LA plays basketball / volleyball, then this could be problematic. The issue with this is you decided what questions to ask depending on the previous answers. This can lead to very flawed data analysis.

So we need to be extra careful when our queries are adaptive. This can be seen in the following results:

- If the queries are non-adaptive, then the size of the samples you need is roughly $\approx \frac{\log k}{\alpha^2}$
- If the queries are adaptive, then the size of the samples you need is $\approx \frac{k}{\alpha^2}$.

Notice that the above issue with adaptive queries occur when each individual report faithfully their answers. One idea to circumvent this major problem is to use differential privacy. This allows us to not reveal too much true information while giving a pretty accurate answer. More concretely, an article from *Science* showed that if we use (ϵ, δ) -differential privacy, then we can support k adaptive queries with sample size $\approx \frac{\sqrt{k}}{\alpha^2}$.

13.3.2 Kaggle / ML Competitions

In a Kaggle / ML Competition, you are given a training dataset. You will train your model on the training set and submit your model which will then be evaluated using a hidden Evaluation Dataset. Your model's performance on the hidden Evaluation Dataset will be released as your score on a public leaderboard.

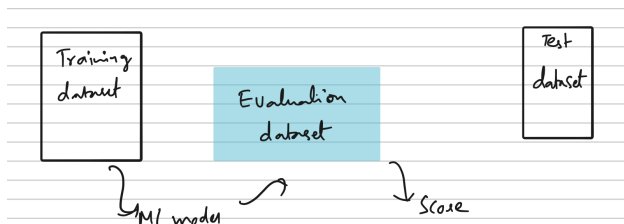


Figure 54: Kaggle Competition setup

It turns out one can achieve 100% accuracy (top of the leaderboard) without even looking at the training data. One can do the following: submit a complete bogus model with very bad accuracy. Then based on the score on the leaderboard, one can change bits of the model and submit again. If one does this many times, then one can eventually get a very high score and even top of the leaderboard without even looking at the training data. This certainly is not desired. To fix this, we can use differential privacy. Instead of releasing the actual score of the submitted model, we can release the score with some "noise." The released the leaderboard can thus serve as a good reference for the model performance while maintaining some privacy.

13.4 Private Empirical Risk Minimization

Let's now go back and consider a more abstract and general problem: private empirical risk minimization. In this scenario, we have labeled dataset. There's a trusted curator who can interact with the database. They train a machine learning model on the datasets and then release the model / the weights.

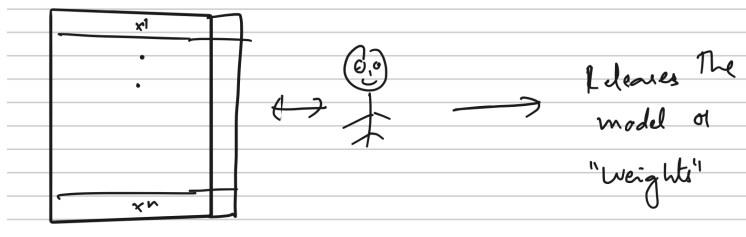


Figure 55: Private Empirical Risk Minimization.

Suppose we have a family of functions $h_\theta : \mathcal{X} \rightarrow \mathcal{Y}$ parametrized by θ as candidate predictors. Recall in Empirical Risk Minimization, given a loss function l , our goal is to minimize the empirical risk:

$$L_{\bar{x}}(\theta) = \frac{1}{n} \sum_{i=1}^n l(h_\theta(x_i), y_i).$$

Ideally, we would want to compute and release

$$\theta^* = \arg \min L_{\bar{x}}(\theta).$$

However, it turns out that if we compute the optimum and release it faithfully the computed minimizer can raise privacy concerns. To address this, one method is to add some noise in the optimization step so that the learned weights may not necessarily be the exact optimum. This leads us to differentially-private SGD.

Differentially-private SGD

- Pick a starting point θ_0 .
- For $t = 1, 2, \dots, T$:
 - Pick a random index i .
 - $\theta_{t+1} = \theta_t - \eta_t \nabla_\theta (l(h_\theta(x^i), y^i)) + Z_t$ where $Z_t \sim \mathcal{N}(0, \sigma_t^2)$ is some Gaussian noise.

There are several different ways to do private ERM. These include:

1. Output perturbation: compute the optimizer θ and then output $\theta + \text{noise}$.
2. Input perturbation: perturb the dataset.
3. Objective perturbation: instead of minimizing $L(\theta)$, we can minimize

$$L'(\theta) = L(\theta) + \lambda \langle v, \theta \rangle$$

where v is a random vector.

Privacy is a broad topic in modern Machine Learning. There are of course other notions of privacy that we have not discussed here, for instance, Local Differential Privacy, Label Differential Privacy etc. Interested readers can refer to this link for a more detailed exposition.

Part V

Distribution Learning

Lecture 14. Bayesian Network

Starting this lecture, we will talk about a new topic namely, Distribution Learning. Suppose there is some true distribution $\mathcal{D}^*$ over $\mathbb{R}^d$. We observe samples $x^1, x^2, \dots, x^n$ independently drawn from $\mathcal{D}^*$. Our goal is to use the samples to understand something about the underlying distribution $\mathcal{D}^*$. Some goals include:

- estimating some statistics about $\mathcal{D}^*$
- Clustering / grouping
- Generate new samples from the distribution
- Compression
- "Outlier" detection etc.

In the following lectures, we will discuss miscellaneous topics like Bayesian Networks, Chow-Liu Algorithm for trees, "Undirected" Markov Random Fields and Diffusion Models (for generating new samples).

14.1 Bayesian Networks

In probability theory, we have this notion of independence. Let's recall what it is. Let (X, Y) be a pair of random variables. We say X, Y are independent, denoted $X \perp Y$ if the following equivalent conditions hold:

$$\Pr[X = x | Y = y] = \Pr[X = x], \text{ or equivalently}$$

$$\Pr[X = x, Y = y] = \Pr[X = x] \cdot \Pr[Y = y].$$

In many real life problems, after gathering the data, we want to make some deductions / inference about the underlying distribution. However, we need to be extra careful when it comes to making conclusions as it's sometimes easy to arrive at the absurdly wrong ones. See Figure 56 for an example.

	Heart Problem	Wake-up time	Age
P_1	NO	10 AM	19
P_2	YES	5 AM	55
P_3	NO	2 PM	17
P_4	YES	6:30 AM	50
P_5	YES	7:00 AM	40
P_6	NO	9 AM	21
P_7	NO	11 AM	22

Figure 56: Just by looking at the column for Heart Problems and the column for Wake-up times, one may be tempted to arrive at the conclusion that waking up early is bad. However, if we include the age column, combining with our biology knowledge, we know the misleadingly high correlation between heart problem and waking up early is due to the elder ages of the individuals. In other words, having a heart problem and waking-up early are independent if we condition on the age variable.

This example leads us to the notion of conditional independence.

Definition 14.1. Let X, Y, Z be random variables with a joint distribution. Then we say X, Y are conditionally independent given Z , denoted $X \perp Y | Z$ if for all x, y, z , we have

$$\Pr[X = x, Y = y | Z = z] = \Pr[X = x | Z = z] \cdot \Pr[Y = y | Z = z].$$

As we will see later, conditional independence is an important concept that is prevalent in data modeling. To help us understand this concept, we use a kind of model called *graphical models*. More specifically, graphical models are distributions with "constrained" conditional independence relations. Broadly speaking, there are two kinds of graphical models: directed graphical models (including Bayesian Networks, Causal Networks) and Undirected graphical models (aka Markov Random Fields).

Example 14.2. Let's consider the perhaps familiar example of a Markov Chain. The following is an example of a Markov Chain.



Figure 57: Example of a Markov Chain.

For any three variables x_i, x_j, x_k such that $i < j < k$ in the above Markov Chain, they satisfy the conditional independence relation: $X_i \perp X_k | X_j$. For instance, the following relations hold: $X_3 \perp X_1 | X_2, X_4 \perp X_2 | X_3, X_4 \perp X_1 | X_3$.

Definition 14.3. (Bayesian Networks) Let G be a directed acyclic graph (DAG) G on d vertices. Associated each vertex i with a random variable X_i that takes value in Σ . Then a distribution D on the variables $(X_1, \dots, X_d)$ is a Bayesian Network (Bayes Net) with graph G if

$$\Pr[(X_1, \dots, X_d) = (x_1, \dots, x_d)] = \prod_{i=1}^d \Pr[X_i = x_i | \text{Parents of } i \text{ in } G],$$

where $\text{Parents}(i)$ is the set of nodes with edges to node i .

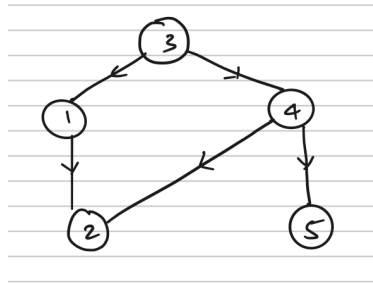


Figure 58: Example of a directed graph. Here $\text{Parents}(3) = \emptyset, \text{Parents}(1) = \{3\}, \text{Parents}(2) = \{1, 4\}, \text{Parents}(4) = \{3\}, \text{Parents}(5) = \{4\}$.

The Markov Chain in Example 14.2 is a Bayesian Network since we have

$$\Pr[X = (x_1, \dots, x_n)] = \Pr[X_1 = x_1] \cdot \Pr[X_2 = x_2 | X_1 = x_1] \cdot \Pr[X_3 = x_3 | X_2 = x_2] \dots \Pr[X_n = x_n | X_{n-1} = x_{n-1}].$$

Bayesian networks can be extremely helpful when analyzing real life situations. For instance, one can model smoking and related factors via the following Bayesian network.

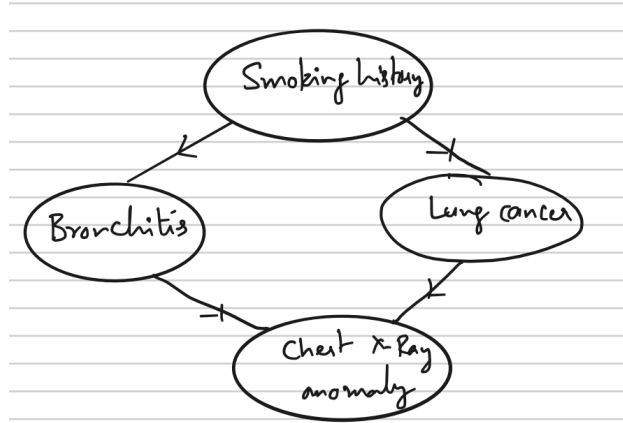


Figure 59: Bayesian network involving Smoking and Chest X-Ray anomalies.

Let $\mathcal{D}$ be a Bayesian Network with graph G . Suppose we observe n samples $x^1, x^2, \dots, x^n \in \Sigma^d$. We would like to use these samples to deduce

1. The underlying directed dependency graph G and
2. The right conditional distributions.

In order to answer these questions (especially the first one), we want to put some restrictions on the type of graphs in consideration. For instance, let's again consider the Markov Chain in Example 14.2. If we obtain samples $x^1, \dots, x^n$ from the distribution, then the following graph also can be the correct underlying graph as it's more general.

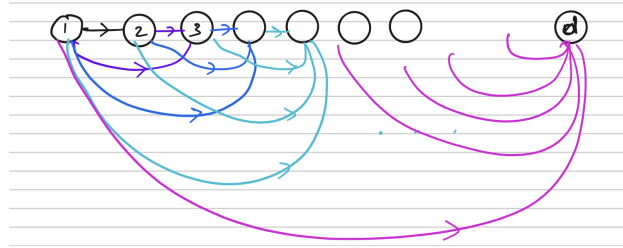


Figure 60: A much more complicated underlying graph for the same distribution.

Based on this graph, we thus have

$$\Pr[X = (x_1, \dots, x_d)] = \Pr[X_1 = x_1] \Pr[X_2 = x_2 | X_1 = x_1] \dots \Pr[X_d = x_d | X_1 = x_1, \dots, X_{d-1} = x_{d-1}].$$

This is of course a correct graph. However, it is inefficient, much more complicated than what is necessary and difficult to model the dependencies between variables, making it hard to obtain any meaningful deductions and inference from the dataset. As a result, we will make some restrictions / assumptions about the underlying DAG.

We may assume:

- The in-degrees of each vertex is bounded.
- The graph possesses some structural properties, e.g., contains certain paths, simple tree-like structure and so on.

14.2 Tree Learning

Let's first consider one simple kind of Bayesian Network with its underlying DAG being a *tree*. Recall from Graph Theory that a DAG is a *tree* if the corresponding undirected graph does not contain any cycle. In this case, the joint distribution can be decomposed quite nicely as illustrated in the following example.

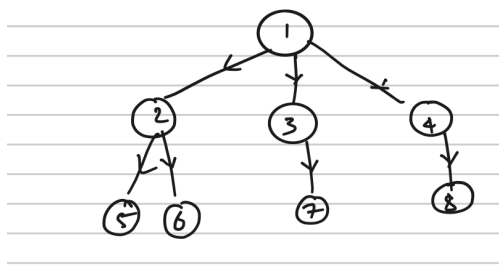


Figure 61: A Bayesian network with the underlying DAG being a tree.

For this particular Bayesian network, the joint distribution can be written as

$$\Pr[X = (x_1, \dots, x_d)] = \Pr[X_1 = x_1] \Pr[X_2 = x_2 | X_1 = x_1] \Pr[X_3 = x_3 | X_1 = x_1] \Pr[X_4 = x_4 | X_1 = x_1] \\ \cdot \Pr[X_5 = x_5 | X_2 = x_2] \Pr[X_6 = x_6 | X_2 = x_2] \Pr[X_7 = x_7 | X_3 = x_3] \Pr[X_8 = x_8 | X_4 = x_4].$$

Now we can formulate our tree-learning problem as the following: given as inputs $x^1, x^2, \dots, x^n \in \Sigma^d$ from a distribution $\mathcal{D}^*$ that is a Bayesian network with the underlying graph being a tree. Our goal is to learn the distribution $\mathcal{D}^*$. In the case of directed graphs, G is a tree if every node has in-degree ≤ 1 .

To be more specific, there are a few objectives we hope to learn from the samples:

- Output the underlying tree G . This corresponds to "Structure Learning."
- Output the right conditional distribution. This corresponds to "Parameter Learning."
- Output $\hat{\mathcal{D}}$ that is "close" to $\mathcal{D}^*$.

We saw how we can quantify the distance between parameters in $\mathbb{R}^d$ in Supervised Learning. However, here we are learning a distribution. How do we define the distance between two distributions? To this end, we define the KL-divergence between two distributions.

Definition 14.4. (*KL-Divergence*) Given two distributions p, q , the KL-Divergence between p and q is defined as

$$KL(p||q) = \sum_a p(a) \log_2 \frac{p(a)}{q(a)}$$

where $p(a)$ denotes the probability of a under distribution p and $q(a)$ denotes the probability of a under q .

KL-Divergence satisfies the following properties.

Lemma 14.5. (*Properties of KL-Divergence*)

- $KL(p||p) = 0$
- $KL(p||q) \geq 0$
- $KL(p||q) = 0 \Leftrightarrow p = q$.

From the above, we can see KL-Divergence satisfies many properties we would want from a distance function. For our purpose, we can treat KL-Divergence as the "distance" between two distributions though KL-Divergence is not rigorously a metric (the formal definition of distance in mathematics) since it does not satisfy the symmetric property, i.e., $KL(p||q) \neq KL(q||p)$.

On the other hand, we can rewrite the definition of KL-divergence as follows:

$$\begin{aligned}
KL(p||q) &= \sum_a p(a) \log_2 \left(\frac{p(a)}{q(a)} \right) \\
&= \sum_a p(a) [\log_2 p(a) - \log_2 q(a)] \\
&= \sum_a p(a) \log_2 p(a) - \sum_a p(a) \log_2 q(a)
\end{aligned}$$

where the first term $\sum_a p(a) \log_2 p(a)$ is the entropy of p and the second term $\sum_a p(a) \log_2 q(a)$ is the cross-entropy between p and q .

As an example, suppose the domain of both p and q is $Z = \{0, 1\}$. If $p(0) = p(1) = 1/2$ and $q(0) = 1/3, q(1) = 2/3$, then we would have $KL(p||q) = \frac{1}{2} \log_2(9/8)$. Now that we have our notion of distance, we can rephrase our distribution learning problem as following:

- **Input:** samples $x^1, \dots, x^n \sim \Sigma^d$ from a Bayes net $\mathcal{D}^*$ with the underlying graph being T^* which is a tree.
- **Output:** we want to output a tree T and a distribution $\mathcal{D}_T$ whose Bayes Net (underlying graph) is T and satisfies

$$KL(\mathcal{D}^*||\mathcal{D}_T) \leq \epsilon.$$

For instance, suppose we have a distribution $\mathcal{D}_{T^*}$ with the following tree T^* as the Bayes net:

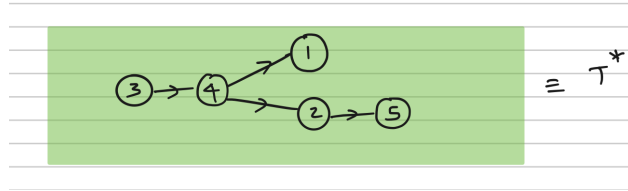


Figure 62: An example of a true underlying graph T^*

we would want to output a $\mathcal{D}_T$ such that $\mathcal{D}_T$ is within ϵ -distance to the minimizer

$$\arg \min_T \min_{\mathcal{D}_T} KL(\mathcal{D}^*||\mathcal{D}_T).$$

14.2.1 Finding the best $\mathcal{D}_T$ given T

Let's first suppose we have a candidate tree structure for the Bayes Net so that we first fix a tree T to work with. Our goal is to find the best conditional distribution $\mathcal{D}_T$ that minimizes $KL(\mathcal{D}^*||\mathcal{D}_T)$. More explicitly, the problem can be stated as follows:

- **Input:** samples $x^1, \dots, x^n \sim \Sigma^d$ (from $\mathcal{D}^*$ with a tree T^*) and another tree T
- **Output:** a distribution $\mathcal{D}_T$ whose Bayes Net is T that minimizes $\min KL(\mathcal{D}^*||\mathcal{D}_T)$.

For instance, if we are given as input the tree T^* from Figure 62, we know the distribution on this graph T must be decomposed as

$$\begin{aligned}
\Pr[X = (x_1, x_2, x_3, x_4, x_5)] &= \Pr[X_3 = x_3] \cdot \Pr[X_1 = x_1 | X_3 = x_3] \cdot \Pr[X_4 = x_4 | X_3 = x_3] \\
&\quad \cdot \Pr[X_2 = x_2 | X_4 = x_4] \cdot \Pr[X_5 = x_5 | X_2 = x_2].
\end{aligned}$$

Simplifying notations, we can write this as

$$\Pr[X = (x_1, x_2, x_3, x_4, x_5)] = P_{T,3}(x_3) \cdot P_{T,1}(x_1|x_3) \cdot P_{T,4}(x_4|x_3) \cdot P_{T,2}(x_2|x_4) \cdot P_{T,5}(x_5|x_2).$$

To estimate these quantities, one natural idea we can try is to use the estimates from the samples. By the Bayes rule, for all edge (i, j) in T and $a, b \in \Sigma$, we have

$$\Pr[X_j = b | X_i = a] = \frac{\Pr[X_j = b \cap X_i = a]}{\Pr[X_i = a]}.$$

We can use the samples to estimate (1) how many times $X_j = b, X_i = a$ and (2) how many times $X_i = a$ and then use the quotient of these two quantities as our estimate $\hat{P}(X_j | X_i)$ for the true conditional distribution $\Pr[X_j | X_i]$. Using this, we then output our estimate $\hat{\mathcal{D}}_T(x)$ as

$$\hat{\mathcal{D}}_T(x) = \prod_{e=(i,j) \in T} \hat{P}(x_j | x_i).$$

We have the following guarantee.

Theorem 14.6. (estimating $\mathcal{D}_T$ given T) The empirical estimate $\hat{\mathcal{D}}_T$ computed as above minimizes $KL(\mathcal{D}^* || \mathcal{D}_T)$ over all distributions $\mathcal{D}$ with Bayes Net T and we can estimate the conditional probabilities to error ϵ with $\approx O(1/\epsilon^2)$ samples.

Now we know how to output the best distribution given a tree T . However, this only partly answers our question as we still do not know what tree we should use as our candidate for the Bayes Net. Now, we are ready to discuss how we can find a "good" tree using Chow-Liu Algorithm.

14.3 Chow-Liu Algorithm

Let $\mathcal{D}^*$ be the ground-truth Bayesian network with its true underlying tree being T^* . Let T be a fixed candidate tree and $\mathcal{D}_T$ be the best estimate of the distribution on T using the samples (from last lecture). Then, one can do some arithmetic and find that

$$KL(\mathcal{D}^* || \mathcal{D}_T) = J_* - \sum_{(i,j) \in T} I(X_i; X_j)$$

where J_* is some term that does not depend on the candidate tree T and $I(X_i; X_j)$ is the mutual information between X_i and X_j .

Definition 14.7. (Mutual Information) Let X, Y be two random variables. Then the mutual information between X and Y , denoted $I(X; Y)$ is defined as:

$$I(X; Y) = \sum_{a,b} \Pr[X = a, Y = b] \log \frac{\Pr[X = a, Y = b]}{\Pr[X = a] \cdot \Pr[Y = b]}.$$

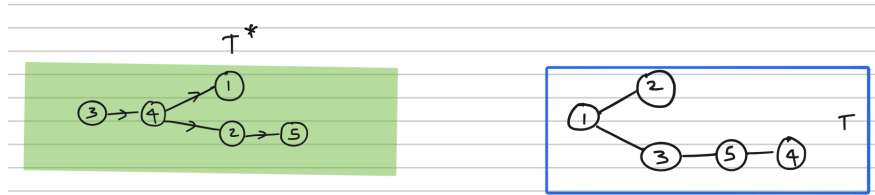


Figure 63: Left: the ground-truth underlying tree. Right: candidate tree.

For instance in Figure 63, for the candidate tree (on the right), we have

$$KL(\mathcal{D}^* || \mathcal{D}_T) = J_* - [I(X_1; X_2) + I(X_1; X_3) + I(X_3; X_5) + I(X_5; X_4)].$$

Therefore, to find the best candidate tree T , we would like to find the tree T that

$$\text{maximizes } \sum_{(i,j) \in T} I(X_i; X_j).$$

Using samples, we can compute / estimate the mutual information between all pairs of variables.

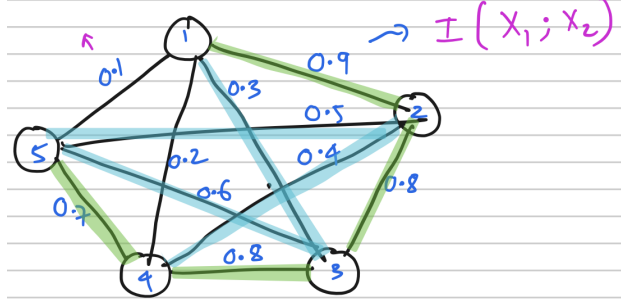


Figure 64: Calculate the mutual information between all pairs of variables using samples.

As a result, we can summarize our approach as the following:

1. Use samples to compute $I(X_i; X_j)$ for all pairs of variables.
2. Run an algorithm to find the "Maximum Spanning Tree" on the graph where the weight of edge (i, j) is $I(X_i; X_j)$.

It remains to show we can find the "Maximum Spanning Tree" efficiently.

Maximum Spanning Tree:

- Input: a graph G on d vertices with weights $w_{u,v}$ on edges.
- Output: a tree T that maximizes $\sum_{(u,v) \in T} w_{u,v}$.

This can be found efficiently using algorithms for Minimum Spanning Tree by making the edge weights negative (multiplying all the weights by -1). Combining this with the mutual information estimates, we obtain the Chow-Liu algorithm for determining the best tree structure.

Chow-Liu Algorithm (1960s)

- Use samples to estimate $I(X_i; X_j)$ for all pairs with error $\delta = \epsilon/d$.
- Form a weighted graph with the mutual information estimates as the weights.
- Compute the Maximum Spanning Tree T on the graph.
- Pick a node to be the root.
- Output $\mathcal{D}_T$ (recall from last time, this can be estimated using samples) as your approximate distribution.

In short, we thus have an algorithm that finds a $\mathcal{D}_T$ such that

$$KL(\mathcal{D}^* || \mathcal{D}_T) \leq \epsilon$$

with roughly $O(d^2/\epsilon^2)$ many samples. Notice that up to this point, we have assumed the underlying directed graph is a tree. This is the simplest case one can think of. One ongoing research area is to find other classes of interesting Bayesian networks that we can learn efficiently.

Lecture 15. Undirected Graphical Models

Last time we introduced the notion of a Bayesian network. We discussed how we can find the best tree structure using Chow-Liu algorithm. Now let's move on and talk about *Undirected Graphical Models*, also known as *Markov Random Fields*.

15.1 Undirected Graphical Models

Let $\mathcal{D}$ be a distribution over d random variables $(X_1, \dots, X_d)$. Let G be a dependency graph for $\mathcal{D}$. The pair $(\mathcal{D}, G)$ is an undirected graphical model if it satisfies some conditional independence properties.

Definition 15.1. (*Pairwise Markov Property*) We say $\mathcal{D}$ satisfies Pairwise Markov Property with respect to G if for all i, j such that $\{i, j\}$ is not an edge in G , we have

$$X_i \perp X_j | X_{\text{remaining}},$$

i.e., X_i and X_j are conditionally independent conditioning on all the other variables.

For instance, suppose the following graph (Figure 65) is a dependency graph for $\mathcal{D}$. If $\mathcal{D}$ satisfies *Pairwise Markov Property* with respect to G , then the followings (a subset of many more such relations) would hold:

$$X_2 \perp X_4 | X_1, X_3, X_5, X_6,$$

$$X_1 \perp X_3 | X_2, X_4, X_5, X_6.$$

since $(2, 4), (1, 3)$ are not edges in the graph.

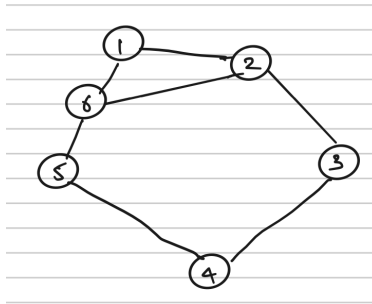


Figure 65: Underlying dependency graph G for $\mathcal{D}$.

Definition 15.2. (*Local Markov Property*) We say $\mathcal{D}$ satisfies Local Markov Property with respect to G if for all i, j such that (i, j) is not an edge in G , we have

$$X_i \perp X_j | X_{\text{neighbors of } i},$$

i.e., X_i and X_j are conditionally independent when conditioned on the neighbors of X_i or X_j .

For example, we again consider the dependency graph in Figure 65. If $\mathcal{D}$ satisfies *Local Markov Property* with respect to G , then we would have, for instance:

$$X_1 \perp X_3 | X_2, X_6$$

$$X_2 \perp X_4 | X_1, X_6, X_3$$

$$X_2 \perp X_4 | X_3, X_5.$$

Definition 15.3. (*Global Markov Property*) We say $\mathcal{D}$ satisfies Global Markov Property with respect to G if for all i, j such that (i, j) is not an edge in G , we have

$$X_i \perp X_j | X_{\text{any separating set}}.$$

Again consider the same example. If it satisfies the *Global Markov Property*, then we would have:

$$X_1 \perp X_4 | X_3, X_6$$

since $\{3, 6\}$ is a separating set for $\{1, 4\}$, i.e., every path from 1 to 4 has to go through either 3 or 6.

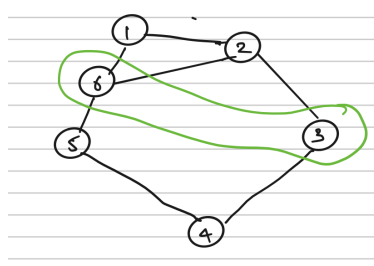


Figure 66: $\{3, 6\}$ is a separating set for 1 and 4.

Example 15.4. Consider the following graph G that is a single path.

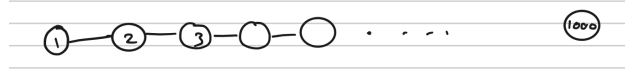


Figure 67: An undirected path

- If $\mathcal{D}$ with respect to G satisfies *Pairwise Markov Property*, then we would have, for instance,

$$X_1 \perp X_{1000} | X_2, X_3, \dots, X_{999}$$

- If $\mathcal{D}$ with respect to G satisfies *Local Markov Property*, then we would have,

$$X_1 \perp X_{1000} | X_2, X_1 \perp X_{1000} | X_{999}$$

- If $\mathcal{D}$ with respect to G satisfies *Global Markov Property*, then we would have,

$$X_1 \perp X_{1000} | X_{17}, X_1 \perp X_{1000} | X_{19}.$$

Remark: The difference between these definitions can be quite subtle at times. Fortunately, for most "reasonable" settings, all three properties are equivalent!

Now that we have discussed various conditional independence notions for undirected graphical models, we are ready to formulate the learning problem as follows.

Structure Learning:

- Input: $x^1, x^2, \dots, x^n$ from $\mathcal{D}^*$ on σ^d .
- Output: A dependency graph for $\mathcal{D}^*$ with the "fewest edges."

The reason we want the "fewest" edges is that we can always learn a dependency graph for $\mathcal{D}^*$ the is the complete graph K_d , i.e., the graph in which every vertex is connected to all the other vertices.

From now on, we will make the following assumptions on the underlying graph: each vertex in the underlying graph has at most k edges. Think of k as very small compared to d . For instance, we may have $d = 10^6$ variables but each vertex only has degree $k = 10$.

15.2 Boltzmann Machines / Ising Models

We will first consider a special undirected graphical model that plays an important role in Statistical Mechanics, namely, Boltzmann Machines which are also known as Ising Models and "Hopfield Networks." Let $\mathcal{D}^*$ be the joint distribution of d variables $(X_1, \dots, X_d)$ such that $X_i \in \{-1, 1\}$ for each $i \in [d]$. $\mathcal{D}^*$ is called an Ising model or a Boltzmann Machine if it $\mathcal{D}^*$ satisfies

$$\Pr[X = (x_1, \dots, x_d)] \propto \exp \left(\sum_{\{i,j\} \in G} w_{ij} x_i x_j + \sum_i \alpha_i x_i \right)$$

for a graph G . In this case, one can check that G is a dependency graph for $\mathcal{D}^*$.

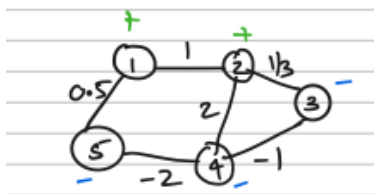


Figure 68: Dependency graph of an Ising model on 5 variables with edge weights. Since this is the dependency graph, we would have $X_1 \perp X_4 | X_2, X_5$.

Now we can formalize our structure learning problem as follows:

- Input: samples $x^1, x^2, \dots, x^n \sim \{1, -1\}^d$ from some Ising Model where the graph G has degree $\leq k$.
- Output: learn the dependency graph G .

In 2010, Guy Bresler gave a $(n \cdot d^2)$ -time algorithm for learning G which requires $n \gg 2^{2^k} \cdot \log d$ samples. In 2017, Adam Klivans and Raghu Meka gave an $O(nd^2)$ -runtime algorithm to learn G with $n \approx 2^{O(k)} \log d$. This is close to the best possible since [SW07] has a lower bound saying that one needs at least $2^k \log d$ many samples. Perhaps surprisingly, the algorithm is relatively straightforward and uses the Multiplicative Weights Method (MWM) that we discussed earlier in this class.

As stated as the way it is, we have in our hand an "unsupervised learning" problem in that we are not really predicting any label but rather "uncovering" the underlying structure of the distribution. To solve this problem, one thing we can try is to convert this "unsupervised learning" problem into a "supervised learning" problem, use the samples to solve this supervised learning problem and then use the information we learned to deduce the structure of the underlying graph.

More concretely, we will use the idea of an "auto-regressive model." Specifically, we will treat some coordinate as the "label" and we will use the other coordinates to predict this "label." Suppose we start with a dataset containing n samples. See the figure below.

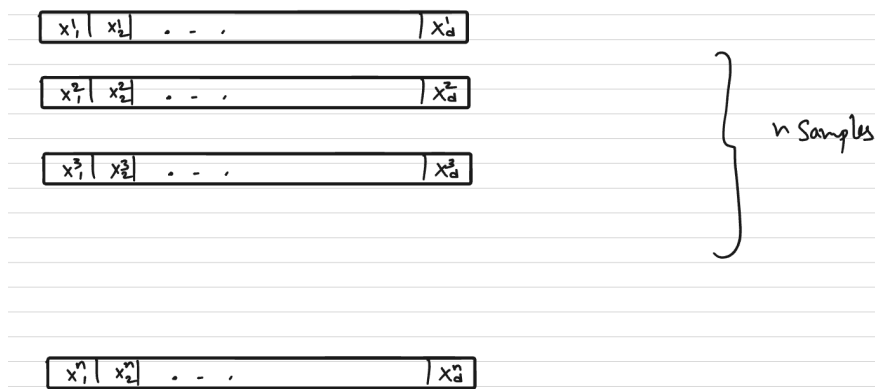


Figure 69: A dataset containing n samples $x^1, \dots, x^n \in \{\pm 1\}^d$.

We can "treat" the first coordinate as the "label" and try to use the other coordinates to predict this "label." In some Deep Learning literature, this step is called "masking." Having done so, we can thus set up a Supervised Learning problem.

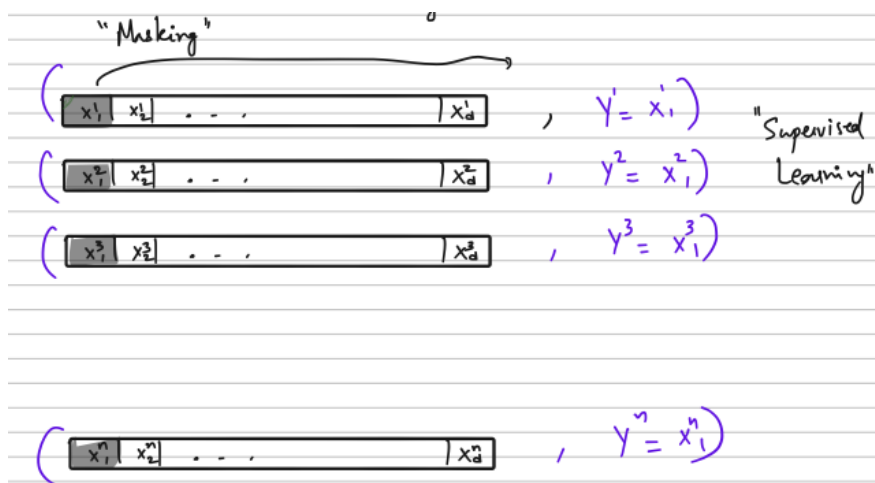


Figure 70: Treat the first coordinate as labels (denote them as the y 's). Use the other $d - 1$ coordinates to predict this label.

Similarly, we can set up more such supervised learning problems by treating the other coordinates as the labels.



Figure 71: Set up a second Supervised Learning problem by treating the second coordinate as labels. We can analogously use the 3rd, 4th,...,dth coordinate as labels.

This technique of converting "unsupervised learning" problems to "supervised learning" problems is widely used in modern Machine Learning. Common models / architectures such as the Transformer architecture, BERT, GPT, etc are all trained on a similar principle).

Now that we know we can in some sense convert unsupervised to supervised learning, there are still two major problems remaining:

- What is a good parametric hypothesis class?
- What loss function should we use?

To address these questions, let's first unpack the Ising model a little bit. Suppose $x^1, \dots, x^n \in \{1, -1\}^d$. Let's begin by treating the last coordinate as the label. For $x = (x_1, \dots, x_d) \in \{1, -1\}^d$, denote $y = x_d$ and $x_{-d} = (x_1, \dots, x_{d-1})$. For simplicity, let's consider the Ising Model of the form

$$\Pr[X = (x_1, \dots, x_d)] \propto \exp\left(\sum_{i,j} w_{ij} x_i x_j\right),$$

i.e., setting the α_i 's in the original definition to be 0. More specifically,

$$\Pr[X = (x_1, \dots, x_d)] = \frac{\exp\left(\sum_{i,j} w_{ij} x_i x_j\right)}{Z}$$

where $Z = \sum_{a \in \{1, -1\}^d} \exp\left(\sum_{i,j} w_{ij} a_i a_j\right)$ is the "normalizing constant." Z is notoriously hard compute in general, but fortunately, we do not need to worry about this for our purpose we will see they are canceled out. Now by Bayes rule, we have

$$\begin{aligned} \Pr[X_d = 1 | X_{-d} = x_{-d}] &= \frac{\Pr[X_d = 1, X_{-d} = x_{-d}]}{\Pr[X_{-d} = x_{-d}]} \\ &= \frac{\Pr[X_{-d} = x_{-d}, X_d = 1]}{\Pr[X_{-d} = x_{-d}, X_d = 1] + \Pr[X_{-d} = x_{-d}, X_d = -1]}. \end{aligned}$$

The normalization constant Z is thus canceled out as they appear in both the numerator and the denominator!

Now we can expand out the terms and obtain

$$\begin{aligned} \Pr[X_d = 1 | X_{-d} = x_{-d}] &= \frac{\exp\left(\sum_{i,j=1}^{d-1} w_{ij} x_i x_j + \sum_{i=1}^{d-1} w_{id} x_i \cdot 1\right)}{\exp\left(\sum_{i,j=1}^{d-1} w_{ij} x_i x_j + \sum_{i=1}^{d-1} w_{id} x_i \cdot 1\right) + \exp\left(\sum_{i,j=1}^{d-1} w_{ij} x_i x_j + \sum_{i=1}^{d-1} w_{id} x_i \cdot (-1)\right)} \\ &= \frac{\exp\left(\sum_{i,j=1}^{d-1} w_{ij} x_i x_j\right) \cdot \exp\left(\sum_{i=1}^{d-1} w_{id} x_i \cdot 1\right)}{\exp\left(\sum_{i,j=1}^{d-1} w_{ij} x_i x_j\right) \cdot \exp\left(\sum_{i=1}^{d-1} w_{id} x_i \cdot 1\right) + \exp\left(\sum_{i,j=1}^{d-1} w_{ij} x_i x_j\right) \cdot \exp\left(\sum_{i=1}^{d-1} w_{id} x_i \cdot (-1)\right)} \\ &= \frac{\exp\left(\sum_{i=1}^{d-1} w_{id} x_i\right)}{\exp\left(\sum_{i=1}^{d-1} w_{id} x_i\right) + \exp\left(-\sum_{i=1}^{d-1} w_{id} x_i\right)} \\ &= \frac{1}{1 + \exp\left(-2 \sum_{i=1}^{d-1} w_{id} x_i\right)}. \end{aligned}$$

Consider the function $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ defined as $\sigma(z) = \frac{1}{1 + e^{-z}}$. Then the calculations above tell us

$$\Pr[X_d = 1 | X_{-d} = x_{-d}] = \sigma\left(\sum_{i=1}^{d-1} (2w_{id}) \cdot x_i\right).$$

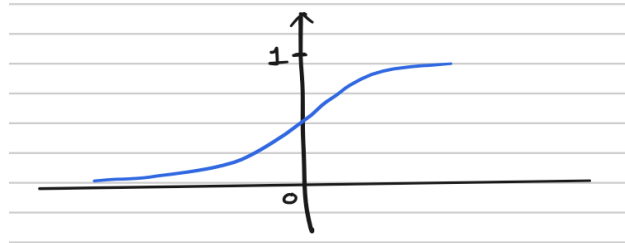


Figure 72: Plot of $\sigma(z)$. This function is also known as the *sigmoid function*.

If we further let $w^* \in \mathbb{R}^{d-1}$ be such that $w_i^* = 2w_{id}$, then we have the following:

$$\Pr[X_d = 1 | X_{-d} = x_{-d}] = \sigma(\langle w^*, x_{-d} \rangle).$$

Thus, it is natural for us to consider the parameter family: $\sigma(\langle u, x_{-d} \rangle)$. From now on, we will for simplicity write x_- instead of x_{-d} . We can then formulate the supervised learning problem as:

- **Input:** $(x_-^1, y^1), \dots, (x_-^n, y^n)$ where for each $i = 1, 2, \dots, n$ we have $y^i = x_-^i$, satisfying the condition:

$$\Pr[Y | X_-] = \sigma(\langle w^*, x_- \rangle)$$

for some $w^* \in \mathbb{R}^{d-1}$.

- **Output:** Find a predictor u such that

$$\mathbf{E} \left[(\sigma(\langle u, x_- \rangle) - \sigma(\langle w^*, x_- \rangle))^2 \right] \leq \epsilon$$

This problem is equivalent to

$$\text{minimize } \mathbf{E} \left[\left(\left(\frac{1+y}{2} \right) - \sigma(\langle u, x_- \rangle) \right)^2 \right]$$

and the corresponding empirical risk minimization problem:

$$\text{minimize } L(n) = \frac{1}{n} \sum_{i=1}^n \left[\left(\frac{1+y_i}{2} \right) - \sigma(\langle u, x_{-i} \rangle) \right]^2$$

In KKS10, the authors showed that we can solve the problem (achieving ϵ error) with $n \approx O(d/\epsilon^2)$ samples with a runtime of $O(dn)$. Klivans and Meka showed we can do better this when the true underlying w^* is sparse (many coordinates are 0).

15.3 Sparsitron

In particular, Klivans and Meka proved the following theorem.

Theorem 15.5. (*KM17 k-sparse*) *If w^* is k -sparse (at most k coordinates of w^* are non-zero), then there is an algorithm that obtains error $\leq \epsilon$ with $n = O(k^2 \log d/\epsilon^4)$ samples and linear runtime.*

We now illustrate the algorithm. Recall we showed $\Pr[X_d = 1 | X_{-d}] = \sigma(\langle w^*, x_{-d} \rangle)$. We will start by making the following simplifying assumptions (though later we will see these assumptions are in fact, very generous):

1. All weights are non-negative, i.e., $w_i^* \geq 0$ for all i .
2. All weights add up to 1.

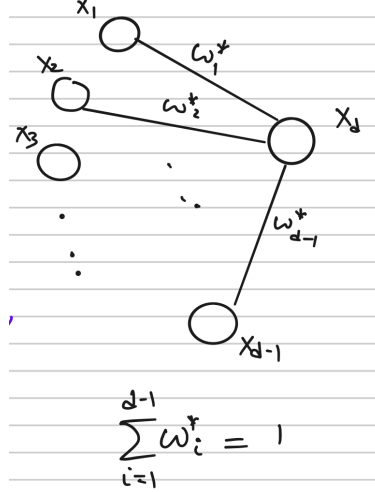


Figure 73: Simplifying assumption 2.

The *Sparsitron* algorithm is as follows:

Sparsitron

- Start by setting $w_i^0 = \frac{1}{d-1}$ for each i .
- For $t = 1, \dots, n$:
 1. Obtain a sample of the form $(x_1^t, x_2^t, \dots, x_{d-1}^t, y^t = x_d^t)$.
 2. If $x_i^t = y^t$, then set $w_i^t = w_i^{t-1}(1 + \eta \cdot b)$
 3. If $x_i^t \neq y^t$, then set $w_i^t = w_i^{t-1}(1 - \eta \cdot r)$

Here r, b are some numbers that depend on $\sigma(\langle w^{t-1}, x^t \rangle)$. The algorithm works regardless of the sparsity of w^* but it does give us a bigger advantage when w^* is sparse. The algorithm will zero-in on the true neighbors of vertex i very fast, using only $t = O(\log d)$ samples.

Notice this is extremely similar to the Multiplicative Weights Method we studied earlier, especially when we make the two simplifying assumptions above. In fact, we can show the convergence of this method using the MWM regret bounds.

Now that we have recover the weights with $\leq \epsilon$ error, we are ready to recover the structure. It turns out (not too difficult to show) that if we have small squared loss, then we can actually recover the dependency graph!

There are various notions of sparsity one can consider. In some cases, k -sparse is too strong a condition to assume. Fortunately, *Sparsitron* works for weaker notions of sparsity (say L_1 -sparsity) as well.

Theorem 15.6. (*KM17 L_1 -sparse*) *If w^* has $\|w^*\|_1 \leq \lambda$, then there is an algorithm that gets error $\leq \epsilon$ with $n = O(\lambda^2 \log d / \epsilon^4)$ samples and with linear runtime.*

We mentioned the simplifying assumptions we made are quite forgiving. Here is why. To deal with the first assumption (non-negativity), given an edge (i, d) with a possibly negative weight w_{id} , one thing we can do is to create two copies of the vertex i . Denote these two copies as i_- , i_+ such that i_- intuitively corresponds to the situation in which $w_{id} < 0$ and i_+ corresponds to the situation in which $w_{id} \geq 0$. Then we can assume w_{i_-d} and w_{i_+d} are both non-negative. In the end, we determine the magnitude and the sign of w_{id} using $w_{i_-d} - w_{i_+d}$. See the following Figure for an illustration.

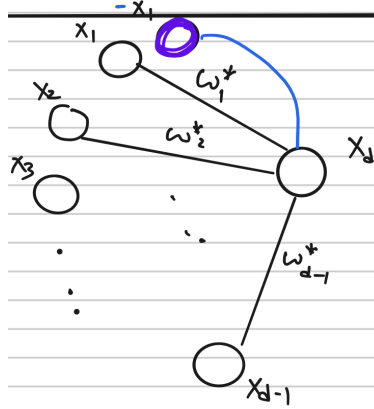


Figure 74: Create an additional $-x_1$ vertex to deal with the potentially negative weight.

The second assumption (true weights sum to 1) is again easy to handle. We can just rescale the weights to make the new sum equal 1.

In summary, given samples from a Restricted Boltzmann Machine (RBM) / Ising Model of degree $\leq k$, we can learn it in time $O(nd^2)$ where $n = 2^{O(k)} \log d$ (first do Sparsitron, then recover the underlying structure using the learned weights).

15.4 Generalized Linear Model

In the discussions above, we talked about a function of the form $y = \phi(\langle w^*, x \rangle)$. Functions of this form are called *Generalized Linear Models* and are very common in learning theory and related areas.

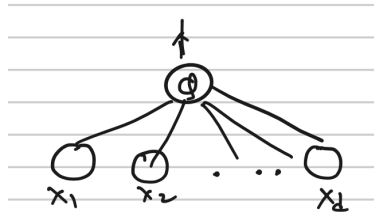


Figure 75: Generalized Linear Model (GLM). One can think of GLM as a way to process the inner product (similarity) between two vectors before outputting.

Example 15.7. • If we take $\phi(z) = z$, then the GLM corresponds to *Least Squares Regression*.

- If we take $\phi(z) = \sigma(z)$, then we obtain the problem we just discussed.
- If we set $\phi(z) = \text{ReLU}(z)$, then it corresponds to the problem of learning a single *ReLU* node.
- If we set $\phi(z) = z^2$, then this corresponds to "Phase Reconstruction."

All of the models in the example above are *single-index models* in that we are only taking one inner product as the argument. We make this more general and consider *multi-index models* (MIMs) which can take in multiple inner products as arguments. It has the form:

$$y = \Psi(\langle w_1^*, x \rangle, \langle w_2^*, x \rangle, \dots, \langle w_t^*, x \rangle).$$

Lecture 16. Diffusion Model

In this lecture, we are going to talk about *Diffusion Model* and more specifically, Denoising Diffusion Probabilistic Modeling (DDPM). Modern machine learning / AI development has lead to progresses in a wide variety of tasks. Among those tasks is the task of image generation. When given a prompt, AI models can generate an image based on the description. Many of the image generation models including DALL-E, IMAGEN, VEO, SORA and many others are all diffusion-based models. More recently, teams from Google also showed you can use text-diffusion models in Large-language Models (LLMs).

More formally speaking, this task of generating new images is what we called *sampling*. Concretely, suppose we have a target distribution q^* . We are given as input samples $x_1, x_2, \dots, x_n \sim q^*$. Our goal is to sample from distribution q^* , i.e., generate new samples from the target distribution.

A Brief History of Sampling: Sampling has been an important problem people study in Computer Science. As briefly explained above, the question can be framed as the following: if I give you "some" limited access to q^* , can you generate new samples from q^* ? Some known sampling techniques include the Metropolis-Hastings Algorithm, Rejection Sampling, Glauber Dynamics, etc. However, despite the existence of certain algorithms that can work on some of the practical tasks, sampling is in fact, computationally hard in general. For instance, even for the simple model like the Ising model we discussed last time, sampling is still hard.

Although we cannot hope to achieve our goal efficiently in general, there are certain cases / distributions that we can efficiently sample, especially with the help of models like Deep Neural Networks!

16.1 Langevin Dynamics

Before we dive directly into the algorithm, let's unpack some intuitions first. Suppose q^* is the target distribution and it looks like the following.

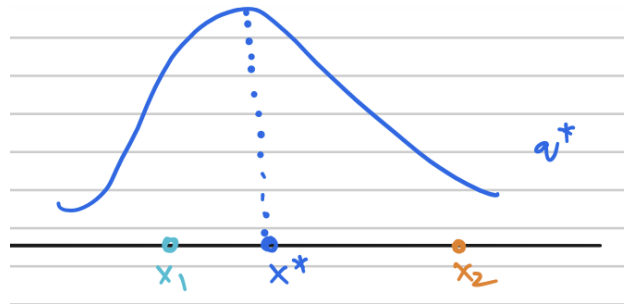


Figure 76: Distribution of q^* where $x^* = \arg \max_x q^*(x)$

Hence, if x_1, x_2 satisfy $|x_1 - x^*| < |x_2 - x^*|$, then intuitively we should output x_1 more "often" than we output x_2 (this saying is of course not rigorous as they both have measure 0). We can use this idea to guide our search for the point to output. Specifically, starting at an arbitrary point x_0 , we would like to move closer to the region with high densities. Recalling our discussion in the first part of the class, we can achieve this via gradient ascent. This leads to Idea 1.

Idea 1: Start with an arbitrary point x_0 and use gradient ascent (move in the direction of $\nabla q^*(x)$).

However, our target distribution may not look as nice as the one shown in the above Figure. In particular, it might have many local maxima. To prevent the Gradient Ascent Algorithm from being "stuck" on one of the local maxima, one thing we can do is to add some small noise in each iteration.

Idea 2: Spread away from the local maxima by adding some noise. For instance, we can do the following update:

$$x^{t+1} = x^t + \eta_t \nabla q^*(x^t) + \sigma_t \epsilon_t$$

where $\epsilon_t \sim \mathcal{N}(0, I_d)$ is some small independent noise.

So far it seems we are simply doing variants of gradient ascent and nothing seems special about the fact we are dealing with probability density functions. This is almost correct except for one additional thing that we need to address. Note that since we are dealing with probabilities, $q^*(x)$ can have very different magnitudes. For instance, we may have x_1, x_2 such that $q^*(x_1) = 0.001, q^*(x_2) = 0.002$ and x_3, x_4 such that $q(x_3) = 0.1, q(x_4) = 0.101$. To bring those numbers to a similar scale, we adapt the algorithm by taking the gradient of the logarithm.

Let q^* be the target probability density on $\mathbb{R}^d$. Since q^* is non-negative, we can write $q^*(x) = e^{-U^*(x)}$ for $U^*(x) = -\log q^*(x)$. To this end, we define the score function of $q^*(x)$ as

$$s^*(x) = \nabla \log q^*(x) = -\nabla U^*(x).$$

Idea 3: Use $\nabla \log q^*(x^t)$ in gradient ascent.

Combining these ideas gives us the Langevin Dynamics update:

$$x^{t+1} = x^t + \eta_t \nabla \log q^*(x^t) + \sqrt{2\delta_t} \epsilon_t$$

for some chosen step-size η_t and noise level δ_t . Here $\epsilon_t \sim \mathcal{N}(0, I)$. What's nice about Langevin Dynamics is that it is actually the discretized version of a Stochastic Differential Equation (SDE) known as the Euler-Maruyama SDE (not required to know this):

$$dY_t = \nabla \log q^*(Y_t) dt + \sqrt{2} dB_t$$

where B_t is the standard Brownian motion (one can view Brownian motion as a continuous version of Gaussian distribution). Since Langevin Dynamics is a "close relative" to the above SDE, we can analyze the corresponding SDE to deduce the convergence of Langevin Dynamics. In particular, one can show that under "nice" conditions on q^* , Langevin Dynamics actually converges to q^* . Similar to our discussion of gradient descent, Lipschitzness and convexity are nice conditions to work with. Instead of imposing these conditions on q^* , we need these conditions on the score function since our the gradient we use in each iteration is $\nabla \log q^*(x)$. We know the Langevin Dynamics converges to the target distribution q^* if the followings holds:

- Lipschitzness: $\|U^*(x_1) - U^*(x_2)\| \leq L \cdot \|x_1 - x_2\|_2$ for some $L > 0$.
- "Strong convexity": U^* is convex.

Example 16.1. (simple calculations) For a standard Gaussian distribution $\mathcal{N}(0, I_d)$ over $\mathbb{R}^d$, the density $q^*(x)$ is given by

$$q^*(x) \propto e^{-\frac{\|x\|^2}{2}}$$

where the symbol $\propto$ means "proportional to" so that we can hide the constants that do not depend on x . Using this, it is easy to see that

$$U^*(x) = \frac{\|x\|^2}{2} + (\text{some constant}).$$

Langevin Dynamics has been around for quite some time. It works really well if our functions satisfy these nice conditions. However, there are issues with the Langevin Dynamics if we want to deploy it in practice.

Problem 1: Lipschitzness is too strong in practice. In many practical situations, our distribution have certain hard constraints. For instance, it might be the case that the target distribution q^* is supported only one a low-dimensional manifold of the entire space, deeming it difficult to even talk about "Lipschitzness" since the any point x in the zero-probability region has $U^*(x)$ unbounded.

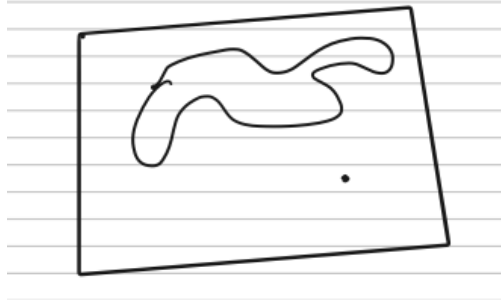


Figure 77: Distribution q^* is supported on a low-dimensional manifold instead of the entire space.

Problem 2: Another problem one may encounter in practice is that the target distribution may be multi-modal. Consider the following distribution that is a mixture of two Gaussians.

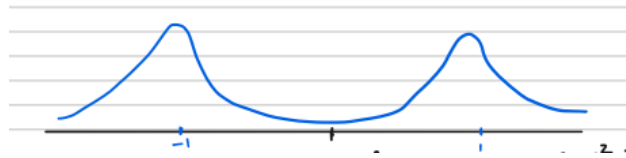


Figure 78: Distribution of a mixture of two one-dimensional Gaussians

This distribution has density

$$q^* \propto \frac{1}{2} \exp\left(-\frac{(x - (-1))^2}{2}\right) + \frac{1}{2} \exp\left(-\frac{(x - 1)^2}{2}\right).$$

If we run our Langevin Dynamics on this distribution, then we will run into troubles. For instance, if we initiate the algorithm with some x_0 that is to the right of the mode to the right. Then once we run the algorithm, we will end up in the right mode pretty fast. However, it will take a large amount of time for the algorithm to return any point in the range of the mode to the left. This implies we only can have samples from one subpart of the distribution.

To address these problems, one thing we can do is the following: starting with the target distribution, we gradually add noise to it to "smoothen" the distribution. This will solve both of the problems mentioned as it will both extend the support of the distribution to the entire space and "mix" the two modes gradually. See the figure below.

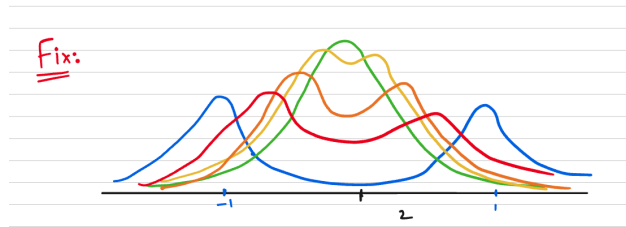


Figure 79: Blue indicates the target distribution. We gradually add noise to it to "mix" the two modes until we obtain Green, which is the standard normal distribution.

In order to make this fix work, we need some way of learning the "added noise." In other words, we need some way of remembering the path from "Blue" to "Green." Incorporating these ideas together, we have the so-called *diffusion model*.

We will be relying heavily on the score function defined earlier. One reason is that to use it, we do not need to worry about the "normalization" constant. More specifically, we know $q^*(x) = e^{-U^*(x)}/Z$ where Z

is a normalization constant that is notoriously hard to compute. The score function $s^*(x)$ is thus given by

$$s^*(x) = \nabla \log q^*(x) = -\nabla U^*(x)$$

which allows us to proceed without computing / estimating Z .

16.2 Diffusion model

Diffusion model was proposed in the 2020s (very recently). It has been used in most state-of-the-art generative models. Recall we said one of the fixes we suggested to the Langevin Dynamics is to add noise and "smoothen" the distribution.

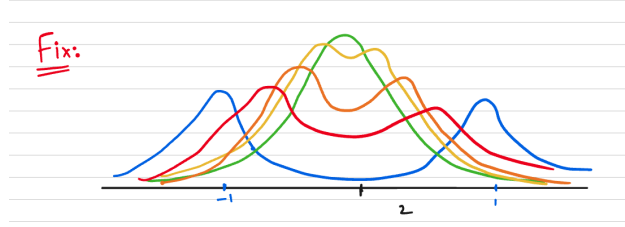


Figure 80: Forward process: adding noise to smoothen the distribution.

This step is called the *Forward Process* in a diffusion model. More concretely, we start with a point $x_0 \sim q^*$. Then for each iteration, we update the point as:

$$x_{t+1} = \sqrt{1 - \sigma_t^2} x_t + \sigma_t \cdot g_t$$

where $g_t \sim \mathcal{N}(0, I_d)$ is some Gaussian noise. $\sigma_t > 0$ is some parameter we can choose. By the properties of independent Gaussian distributions, we can rewrite the $(t + 1)$ -st iteration as

$$x_{t+1} = \exp(-t) \cdot x_0 + \sqrt{1 - \exp(-2t)} \cdot z_t. \quad (26)$$

Notice that as t becomes larger, $\exp(-t) \rightarrow 0$ and thus $x_{t+1} \rightarrow z_t$. In other words, if we run the Forward Process long enough, we will eventually obtain only the standard Gaussian noise:

$$X_0 \rightarrow X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_T$$

where $X_T \approx \mathcal{N}(0, I_d)$. Hence, via the forward process, we can gradually add noise to the initial distribution, obtain a smoother distribution at each step till we eventually reach the standard Gaussian. Recall our goal is to obtain new samples from the target distribution q^* . Now that we have a path from q^* to $\mathcal{N}(0, I_d)$, can we "reverse" the path so that we can obtain new samples from q^* by starting with an arbitrary point sampled from $\mathcal{N}(0, I_d)$? The answer is yes. This step is called the *Reverse Process*. We use $\leftarrow$ to indicate variables in the reverse process. For instance, $\overleftarrow{x}_0$ denote the initial point of the reverse process. Then the reverse process looks like the following:

$$\overleftarrow{x}_T \leftarrow \dots \leftarrow \overleftarrow{x}_2 \leftarrow \overleftarrow{x}_1 \leftarrow \overleftarrow{x}_0$$

where $\overleftarrow{x}_0 \sim \mathcal{N}(0, I_d)$ is the standard Gaussian. Then ideally, we would have $\overleftarrow{x}_T \sim q^*$.

Notice that the reverse step looks a lot like a Langevin Dynamics! Here we are also evolving a point from a standard gaussian over time to obtain a sample from q^* . However, instead of using the score function of q^* in the process, we can instead use the distribution of the intermediate variables as a "guide" to avoid the problems with the original Langevin Dynamics that we mentioned earlier.

Let q_t^* be the distribution X_t , i.e., the t th variable in the forward process. Then the reverse process generates new samples as the following:

$$\begin{aligned} \overleftarrow{x}_{t+1} &= \overleftarrow{x}_t + \overleftarrow{\eta}_t \nabla \log \overleftarrow{q}_t^*(\overleftarrow{x}_t) + \overleftarrow{\sigma}_t \epsilon_t \\ &= \overleftarrow{x}_t + \overleftarrow{\eta}_t \nabla \log q_{T-t}^*(\overleftarrow{x}_t) + \overleftarrow{\sigma}_t \epsilon_t. \end{aligned} \quad (27)$$

Here $\overleftarrow{\eta}_t$'s and $\overleftarrow{\sigma}_t$'s are parameters for the reverse process. $\overleftarrow{q}_t^*$ denotes the distribution of $\overleftarrow{X}_t$. Notice if we align the forward and reverse process, we can see that the t th step of the reverse process corresponds to the $(T-t)$ th step of the forward process. Therefore, q_{T-t}^* is the "desired" distribution of $\overleftarrow{X}_t$ if the reverse process actually does what we desire (reversing the forward process)! Indeed, we have the following theorem.

Theorem 16.2. (Informal) *If T is big enough and the score functions are Lipschitz, then*

$$\overleftarrow{X}_T \approx q^*.$$

More specifically, for an estimation error of ϵ , we need roughly $T \approx 1/\epsilon^2$.

In short, we have that $\overleftarrow{q}_t^*$ (the pdf of $\overleftarrow{X}_t$) is the pdf of X_{T-t} . Now we have a "reverse process" that can ideally generate new samples from q^* provided we know $\nabla \log q_t^*$ at each time t . However, we do not really have access this so we need to estimate these using the time-progressing samples generated via the forward process. Recall that at each time t of the forward process, we have $X_t = \sqrt{1 - \sigma_t^2} \cdot X_t + \sigma_t g_t$ where $g_t \sim \mathcal{N}(0, I_d)$.

Lemma 16.3. (Tweedie's formula) *Let X be some random variable and $\tilde{X} = \sqrt{1 - \sigma^2}X + \sigma g$ where $g \sim \mathcal{N}(0, I_d)$. Then we have*

$$\mathbf{E}[X|\tilde{X}] = \frac{1}{\sqrt{1 - \sigma^2}}\tilde{X} + \frac{\sigma^2}{1 - \sigma^2}\nabla \log \tilde{q}\left(\frac{\tilde{X}}{\sqrt{1 - \sigma^2}}\right)$$

where $\tilde{q}$ is the probability density function of $\tilde{X}$

Letting $a = \frac{1}{\sqrt{1 - \sigma^2}}$ and $b = \frac{\sigma^2}{1 - \sigma^2}$, then the above is saying

$$\mathbf{E}[X|\tilde{X}] = a \cdot \tilde{X} + b \cdot \tilde{S}\left(\frac{\tilde{X}}{\sqrt{1 - \sigma^2}}\right). \quad (28)$$

Rearranging, we have

$$\tilde{S}\left(\frac{\tilde{X}}{\sqrt{1 - \sigma^2}}\right) = \frac{1}{b}\left(\mathbf{E}[X|\tilde{X}] - a\tilde{X}\right) = \frac{1}{b}\left(\mathbf{E}[X - a\tilde{X}|\tilde{X}]\right).$$

Recall that $\tilde{X} = \sqrt{1 - \sigma^2} \cdot X + \sigma \cdot g$ and $a = 1/\sqrt{1 - \sigma^2}$. This suggests $X - a\tilde{X} = \frac{\sigma}{\sqrt{1 - \sigma^2}}g$. Thus,

$$\tilde{S}\left(\frac{\tilde{X}}{\sqrt{1 - \sigma^2}}\right) = \left(\frac{1}{b} \cdot \frac{\sigma}{\sqrt{1 - \sigma^2}}\right) \cdot g$$

where $g \sim \mathcal{N}(0, I_d)$ is some standard Gaussian noise. Therefore, to compute the score function $\tilde{S}$ at a point, we need to "denoise." In other words, given $\tilde{x}$ which is a random variable with added noise, we want to know what is the expected value of $x - a\tilde{x}$ (since $\tilde{x}$, this is asking how can we predict x). To this end, we set up the following optimization problem:

$$\min_{\psi} \mathbf{E} \left[\|x - \psi(\tilde{x})\|^2 \right] \quad (29)$$

where $x, \tilde{x}$ are such that $\tilde{x} = \sqrt{1 - \sigma^2}x + \sigma \cdot g$. On one hand, we have the following lemma.

Lemma 16.4. *Let (Y, Z) be two real-valued random variables. Then*

$$\mathbf{E}[Z|Y] = \min_{\text{all possible functions } \psi \text{ of } y} \mathbf{E} \left[\|Z - \psi(Y)\|^2 \right].$$

As a result, the optimal solution to (29) is always $\mathbf{E}[X|\tilde{X}]$ which is hard to compute. Hence, we can instead ask what is best estimator ψ for ψ that is in some function family. In other words, we can relax the above optimization problem to the following:

$$\min_{\psi \text{ from some function class}} \mathbf{E} \left[\|Z - \psi(Y)\|^2 \right].$$

Once we have a ψ that solves this relaxed version of optimization problem, we can then approximate the score function via equation (28) as:

$$\hat{s}\left(\frac{\tilde{x}}{\sqrt{1 - \sigma^2}}\right) = a\tilde{x} + \psi(\tilde{x}).$$

16.3 The Learning Problem

Recall that in order to run the reverse process to generate new samples, we need to good estimate of the score functions $\overleftarrow{s}_t$'s. To do this, we first generate noisy samples at different noise level (time) via the forward process.

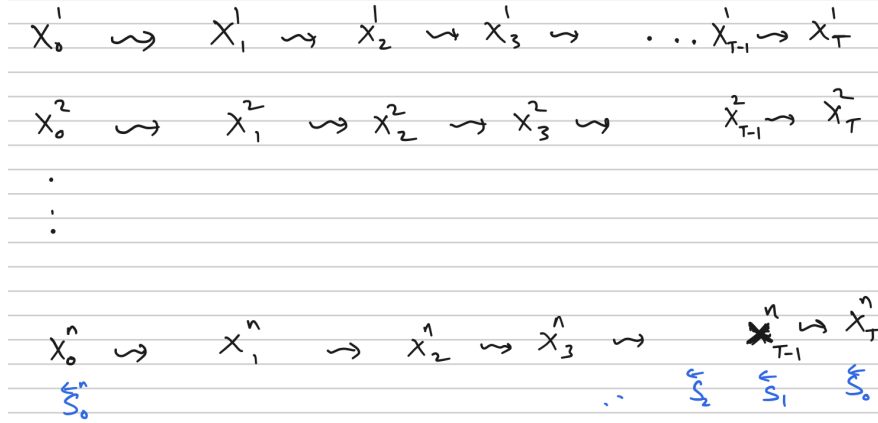


Figure 81: For each of the n samples $x_0^1, \dots, x_0^n$ drawn from the target distribution, evolve each of them from $t = 1$ to $t = T$.

We need to compute the score functions $\overleftarrow{s}_0, \overleftarrow{s}_1, \dots$. For each t , by our discussion in the last section, we can compute $\overleftarrow{s}_t$ by setting up the following supervised learning problem:

given as input $(x_{T-t-1}^1, x_{T-t}^1), (x_{T-t-1}^2, x_{T-t}^2), \dots, (x_{T-t-1}^n, x_{T-t}^n)$, we want to solve

$$\min_{\psi_t: \text{a neural network with a specific architecture}} \frac{1}{n} \sum_{i=1}^n \left\| x_{T-t-t}^i - \psi_t(x_{T-t}^i) \right\|^2.$$

Let ψ_t be the solution to the above minimization problem. We can then estimate the score function as

$$\overleftarrow{s}_t\left(\frac{\tilde{x}}{\sqrt{1-\sigma^2}}\right) = a\tilde{x} + \psi_t(\tilde{x}).$$

We can now summarize the diffusion model for generating new samples as the following.

1. Give a database of samples $x_0^1, x_0^2, \dots, x_0^n$, generate noisy versions

$$\begin{aligned} & x_0^1, x_1^1, \dots, x_T^1 \\ & x_0^2, x_1^2, \dots, x_T^2 \\ & \dots \\ & x_0^n, x_1^n, \dots, x_T^n. \end{aligned}$$

2. Set up T supervised learning problems as above to get approximate score functions

$$\overleftarrow{s}_{T-1}, \dots, \overleftarrow{s}_2, \overleftarrow{s}_1, \overleftarrow{s}_0.$$

3. To generate new samples, for $t = 1, \dots, T$, set

$$\hat{x}_{t+1} = \overleftarrow{x}_t + a_t \overleftarrow{s}_t(\hat{x}_t) + b_t \cdot g_t$$

where $g_t \sim \mathcal{N}(0, I_d)$.

The above steps allows us to generate new samples from q^* . In practice, for efficiency and clarity, we make two simplifying assumptions.

First simplifying assumption: instead of setting up T different supervised learning problems and train T different neural networks, we can solve one "large" problem:

$$\min_{\phi \text{ over all networks of a certain architecture}} \frac{1}{T \cdot n} \sum_{i=1}^T \sum_{t=1}^n \left\| x_{T-t-1}^i - \psi(x_{T-t}^i; t) \right\|^2.$$

Second simplifying assumption: instead of using variables at different time as the input variables, for each time t we can regress between

$$(x_0^1, x_t^1), \dots, (x_0^n, x_t^n)$$

instead. Notice we can do this since the iterative steps in the forward process suggests $X_t = a_t X_0 + b_t g$ for some a_t, b_t .

With these two simplifying assumptions being made, we can now generate data using the forward process and solve the following problem instead:

$$\min_{\phi \text{ over all networks of a certain architecture}} \frac{1}{T \cdot n} \sum_{i=1}^T \sum_{t=1}^n \left\| x_0^i - \psi(x_{T-t}^i; t) \right\|^2. \quad (30)$$

To generate new samples, we run the reverse process:

$$\overleftarrow{X}_{t+1} = \overleftarrow{X}_t + a_t \overleftarrow{S}_t(\overleftarrow{x}_t) + b_t \cdot g_t.$$

Even with these simplifying assumptions made, we can still obtain very nice convergence guarantees.

Theorem 16.5. (CCL+ 23) *If the following conditions hold: (1) the score function of q^* is Lipschitz and (2) the error we get in (30) is small, then we can get good convergence in KL-divergence.*

Lecture 17. Transformers

17.1 Introduction

It is a beautiful, efficient and also in some sense, a "ridiculous" idea to model human languages. Back in 1950, Shannon was interested in figuring out the entropy of human language. Let V be the set of words in a vocabulary. Then for a sentence consisting 10 words, we have at most $|V|^{10}$ possibilities in total. Of course, most of the sentences formed this way are nonsense. As an experiment to "measure" the entropy, he asked people to guess the next word based on the first couple of words. If human language has low entropy, then one should be able to deduce the next word based on this partial information.

This led to the most naive way of language generation, i.e, next-word / token prediction. For instance, if one is given the sentence: this is a wide BLANK. Can one predict the missing word? This task is also sometimes called "sequence completion." The similar idea can be used in translation and the corresponding task is "sequence-to-sequence translation." More formally speaking, we can consider an n -gram model in which we predict the next word using the previous n words. In other words, we use some function ψ to predict the next word as

$$\text{Next word / token} = \psi(\text{previous } n \text{ words/tokens}).$$

To formally model languages and predict using machine learning models we need to do Word Embeddings, i.e., we want some methods that treat tokens / words as vectors over $\mathbb{R}^d$ while maintaining some properties between the words. For instance, if we consider the pair of words (men, women), then the pairs (king, queen) and (prince, princess) should roughly follow the same relationship. Thus, the "distance" within these pairs should be similar in a word embedding. See the Figure below.

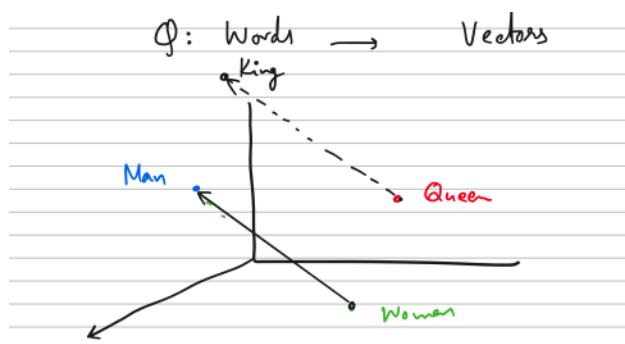


Figure 82: A Word Embedding that preserves certain properties / relationships of words.

In order to have a good language model, ideally we would want to somehow capture and preserve "all" of these relationships when we embed words in $\mathbb{R}^d$. This embedding step is an essential first step. Shortly, we will see how instead of words, we can embed tokens more generally.

17.2 Language Modeling

Suppose we have a finite set of "vocabulary" $V = \{1\text{st token}, 2\text{nd token}, \dots, N_v\text{th token}\}$. We are interested in the language formed using this vocabulary set, i.e., elements over V^* . For instance, our vocabulary set may look like

$$V = \{\text{a-z, A-Z, 0-9, punctuation symbols, common prefixes/suffixes/words}\}.$$

Example 17.1. Consider the sentence

This is a cat.

Suppose "this" and "is" are words included as common words in the vocabulary. Then we can encode this sentence, for example, as

$$[17] [25] [10] [25] [1] [25] [3][1][11] \in V^*,$$

where [25] is the number corresponding to the "space character" in the vocabulary. [17] corresponds to "this." [10] corresponds to "is" [1] corresponds to the letter [a] while [3] [11] corresponds to the letters "c" and "t" respectively.

As a result, natural languages over vocabulary V can be formulated mathematically as a distribution over V^* . To model natural languages, the simplest method we can use is sequence modeling: given as data $x_0, x_1, \dots \in V^*$ from some distribution p , we want to sample from the same distribution. Let $x \in V^*$. Then applying Bayes Rule repeatedly, we can write

$$\Pr(x[0], x[1], \dots, x[T-1]) = \Pr(x[0]) \cdot \Pr(x[1]|x[0]) \dots \Pr(x[T-1]|x[0] \dots x[T-1]).$$

This corresponds to the "next token prediction" problem. Hence, to estimate p , it suffices to use the input data to estimate each conditional distribution on the RHS of the equation.

A close relative of the sequence model above is sequence-to-sequence modeling. In this case, we are given dataset in pairs $(z_1; x_1), (z_2; x_2), \dots, (z_n; x_n)$. The best real-life analogy / application of this is the task of language translation. One can view the z_i 's as sentences in one language and the x_i 's as translated sentences in a different language. One can view the x 's and the z 's coming from the same large vocabulary that is the union of the two vocabularies. In this case, again via the Bayes rule, we can model the distribution as

$$\Pr[X|Z] = \Pr[X[0]|Z] \cdot \Pr[X[1]|X[0], Z] \dots \Pr[X[T-1]|X[0], \dots, X[T-2], Z].$$

From the above, we see how we can break down the problem into estimating distributions given the tokens. To formally model the problem, we still need a way to represent tokens as vectors in $\mathbb{R}^d$.

Token Embeddings: For a natural language, there are in fact many different tokenizations one can pick depending on the specific language and the task in hand. Now for each tokenization, we want to map each token to a vector. This is called token embeddings. To do this, we will need a big matrix $W_e \in \mathbb{R}^{d_e \times N_v}$, i.e., the embedding matrix. This matrix tells us for each token in a given tokenization, what vector in $\mathbb{R}^d$ it corresponds to.

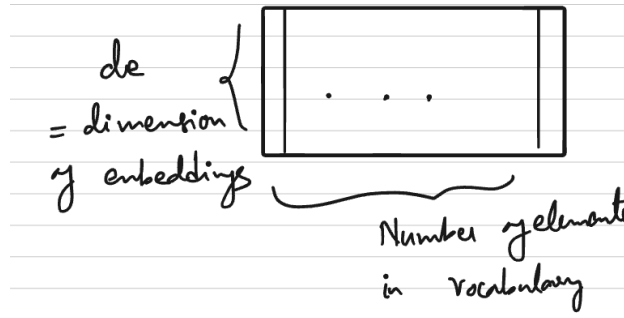


Figure 83: Example of a token embedding matrix. d_e is the dimension of embeddings. N_v is the number of elements in the vocabulary.

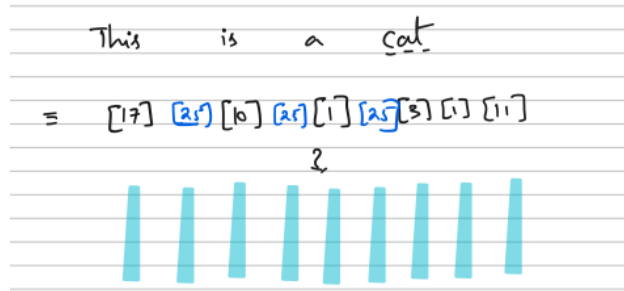


Figure 84: From a sentence, we first abstract it as positions in the vocabulary. We then map each position to the corresponding vector in the embedding matrix. The sentence "this is a cat" becomes vectors $W_e[:, 17]$, $W_e[:, 25]$, ..., $W_e[:, 11]$.

Position Embeddings: notice that token embeddings only tell us what the components of a sentence are. It does not tell us how these components are put together at all. So we also need information about the position of the tokens. This step is called *position embeddings*. A position embedding matrix is of the form $W_p \in \mathbb{R}^{d_e \times [\text{context-length}]}$. Now that we have both position embeddings and token embedding, this allows to regard, say the first token $x[0]$ of a sentence x as a d_e dimensional vectors $W_e[:, x[0]] + W_p[:, 0]$. By doing this, instead of considering an input x as a sentence, we can view it as a sequence of vectors $x_0, x_1, \dots, x_{T-1} \in \mathbb{R}^{d_e}$.

17.3 Attention

Now given the embeddings, we want to know what how we can use it to extract information and generate new words / sentences. Before we delve into how to accomplish this, there are some subtle language phenomena that we need to be aware of and address.

Example 17.2. Consider the following sentence: the cow did not cross the street because it was too [BLANK].

How should we or the model fill in the blank? Depending on the context, the word "it" may refer to different parts of the sentence. It may refer to "the street" or it may also refer to "the cow." So the two completions in the figure below both make sense.

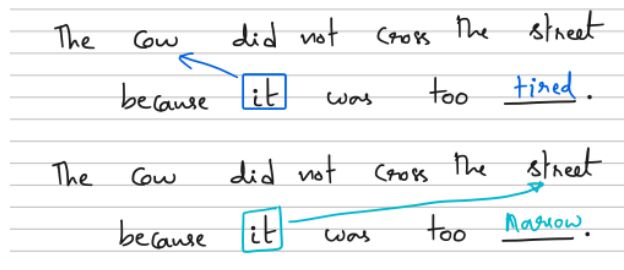


Figure 85: Two sentence completions that both make sense.

Hence, to figure out which completion to use, we cannot just focus locally on a single word or a single sentence. We need to consider the entire paragraph / article. In other words, we need a more global perspective when finding the connection between individual tokens and the entire context. To achieve this, we use *attention*.

Recall that we start with a token + position embedding of a sentence in which each token is discrete / separated and these vectors only encode local information of the sentence. Now, we apply an attention block, which is some matrix we use to aggregate the token / position information of different tokens. Given these aggregated / combined information, we then apply a Neural Network to each vector to "model" the sentence. These two steps combined together is called one layer. We can add layers on top of each other as a better model. See the Figure below for an illustration.

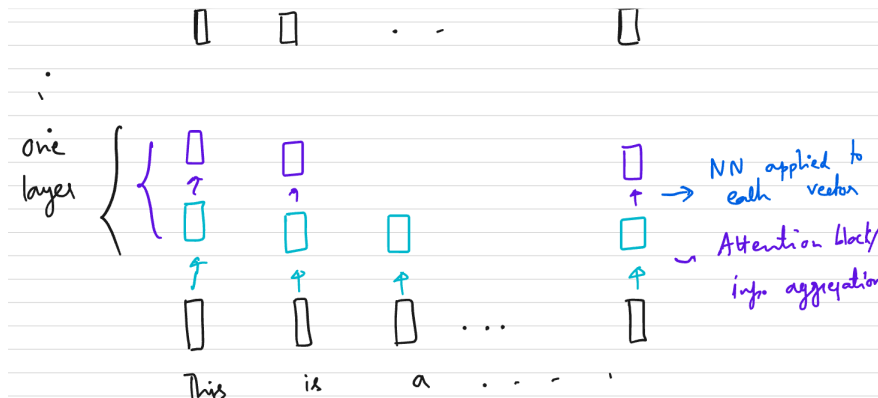


Figure 86: Illustration of an attention layer.

At the end of the day, you obtain some new set of vectors (the vectors at the top in the figure above). Now we need to "convert" this set of vectors to actual sentences. We do this via an "unembedding matrix."

Unembedding matrix: given a vector $v \in \mathbb{R}^{d_e}$, you want to give a probability distribution over V . The main idea is to use softmax. Let $W_u \in \mathbb{R}^{d_e \times N_V}$ be the "unembedding matrix." In practice W_u and W_e are separately trained so they are different in general.

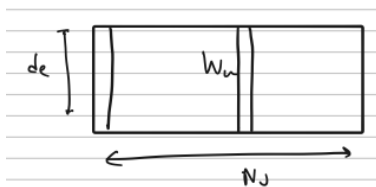


Figure 87: An unembedding matrix W_u .

Once we have such a matrix, we then output the i th token in the vocabulary with probability

$$\Pr[\text{output } i] \propto \exp(\langle v, W_u[:, i] \rangle).$$

Here the dot product tells us how aligned the tokens are to each other as vectors. Hence, implicitly in the embedding / unembedding step is the assumption that words with similar meanings should be more aligned with each other.

Now let's rewrite the model in terms of the parameters more formally. There will be three types of matrices of parameters we will be considering:

- "Query matrix" Q
- "Key matrix" K
- "Value matrix" V .

We now describe the attention layer in terms of these matrices which we will specify shortly.

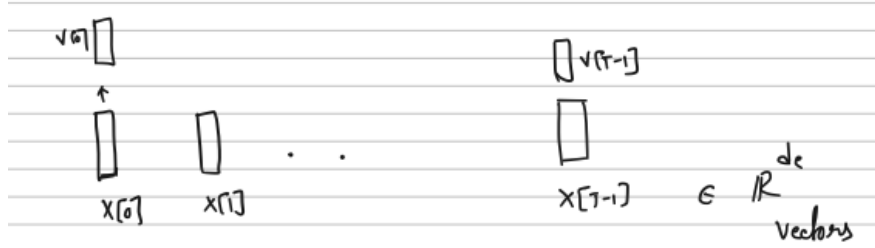


Figure 88: In one attention layer, we first apply a linear transformation to each embedding. We then aggregate the information across different vectors.

Attention layer:

- For $t = 0, 1, \dots, T - 1$:
 - Compute $v_t = V \cdot x[t]$.
- For $t = 0, \dots, T - 1$:
 - Replace

$$v_t \leftarrow \sum_{l=0}^{T-1} \alpha_{l,t} v_l + b_v$$

- $\alpha_{l,t}$ is the "similarity score" to be defined below.

Defining $\alpha_{l,t}$: Fix a $t \in \{0, 1, \dots, T - 1\}$:

- for $l \in \{0, 1, \dots, T - 1\}$:
 - Compute the "Key value": $k_l = K \cdot x[l] + b_k$.
- Compute a "query" value $q_t = Q \cdot x[t] + b_r$.
- Output

$$\alpha_{l,t} \propto \exp(\lambda \langle k_l, q_t \rangle) = \frac{\exp(\lambda \cdot x[l]^T K^T \cdot Q \cdot x[t])}{\sum_{l'} \exp(\lambda \cdot x[l']^T \cdot K^T \cdot Q \cdot x[t])}.$$

Intuitively, for each t , K is some matrix that "condense" every other vector $x[l]$, extracting only the "key" information about $x[l]$. Q is some matrix that reweigh the current vector x_t to make it ready as a "query" for comparison. Then we compute the similarity score using inner products between the computed k_l, q_t to tell us how "similar" $x[t]$ is to $x[l]$. Pictorially, it looks like the following:

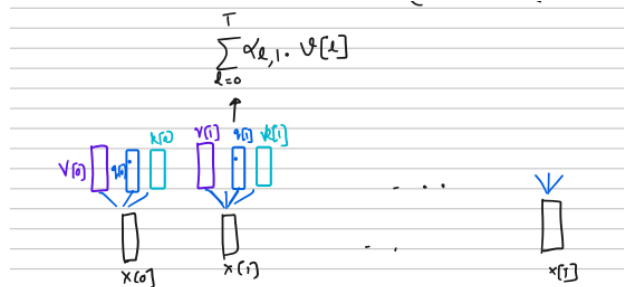


Figure 89: For each token vector, we are computing three associated vectors. For instance, for $x[0]$, we compute $v[0], q[0], k[0]$. We then use these computed values to compute $\alpha_{l,t}$.

As a result, for a single Head of Attention, we have parameters Q, K, V, b_Q, b_K, b_V . We then apply attention as above.

This is for one attention head. Now what if we have multiple attention heads?

Multihead Attention: Instead of one, we can have H layers $[Q^1, K^1, V^1], \dots, [Q^H, K^H, V^H]$. In this case, the inputs are still embeddings $x[0], \dots, x[T-1]$, the output will be $\bar{x}[0], \dots, \bar{x}[T-1]$ where

$$\bar{x}[i] = \sum_{l=1}^H \text{Attention}_{Q^l, K^l, V^l}(x[i]; x[0], x[1], \dots, x[T-1]).$$

As a result, to specify a transformer, we need to specify (1) How many layers of attentions do we have? (2) How many heads do we have per layer? (3) What kind of neural network are we using? The following figure illustrate these components.

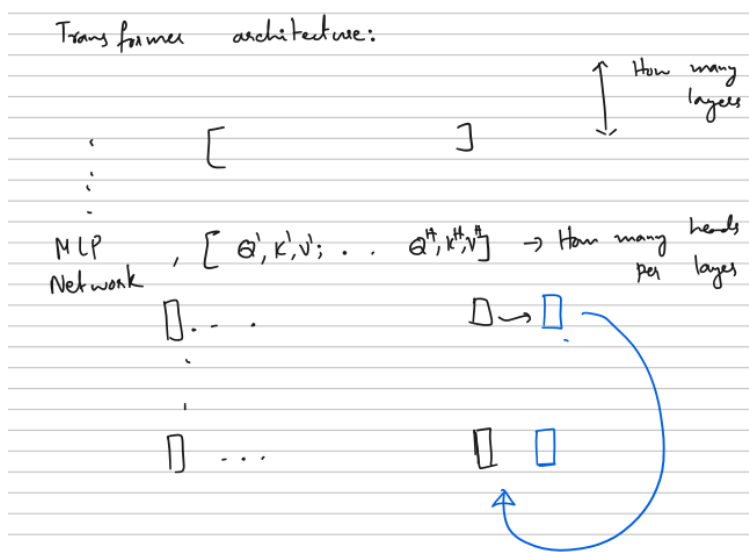


Figure 90: Illustration of a transformer architecture. To generate a sentence given prompt, you first generate the first word then feed the generated first word as part of the prompt and proceed again.

17.4 Runtime efficiency

Now that we have the above transformer architecture, can we make it more efficient? One potential of transformer is that its processing time depends on the context length. More specifically, the time to process a single layer of attention is quadratic: $O(T^2)$. A natural question to ask is: can we make it run faster? Here are a couple of ideas.

Idea 1: Locality Sensitive Hashing (LSH) There are two many vectors involved with each token. Can we be more efficient at this? We want to hash vectors such that "similar" vectors are much more likely to be hashed to the same bucket.

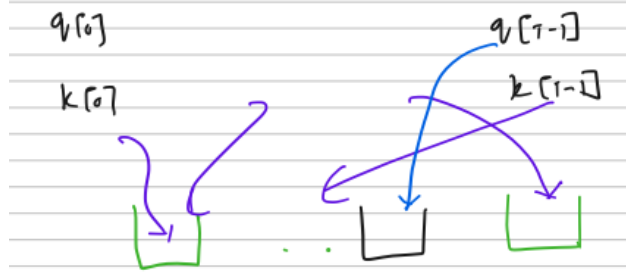


Figure 91: Illustration of Locality Sensitive Hashing.

A bit more formally, the idea is to (1) hash all keys and (2) for each t , hash $q[t]$ and only compute softmax over vectors in the same bucket.

Idea 2: Sparse Attention In the original attention layer, we take $v[t] = \sum_{l=0}^{T-1} a_{l,t} v_l$, which is a sum over a lot of terms. To improve computational efficiency, one thing we can do is to take partial sums. More specifically, we take

$$v[t] = \sum_{l: \text{the top } r \text{ inner products}} \bar{a}_{l,t} v[l].$$

To find the top r inner products, we can use algorithms like k -Nearest Neighbor Search.

Idea 3: Flash Attention Notice that one operation we perform many times in an attention layer is the operation of computing the value $v[t]$. Hence, one thing that can improve the runtime significantly is to implement some methods that can reuse/save some of the computations. One way to do such this is to use an online "softmax computation." The high-level idea is as follows: instead of computing all the terms in the sum, we can

- Keep track of the max inner product so far. Denote this as m
- Track $\exp(k_j^T v_j - m)$, i.e., the numerator.
- Track the running sum $\sum_{i \leq j} \exp(k_i^T q_j - m)$.

Part VI

Reinforcement Learning

Lecture. Markov Decision Process

Definition 18.1 (Markov Decision Process). A finite Markov Decision Process (MDP) is specified as a tuple $(S, \mathcal{A}, \mu, P, \delta)$ where the components are specified as

- S : contexts / states
- $\mathcal{A}$: actions
- μ : mean reward, i.e., $\mu(s, a) := \mathbf{E}[\text{reward after taking action } a \text{ from state } s]$.
- P : transition probability, i.e., $\mathbf{P}(s'|s, a)$ is the probability of ending in state s' when performing action a in state s .
- δ : the discount parameter ($\delta < 1$).

Given this definition, we can consider the problem of **planning**. Specifically, suppose we know μ and P , our goal is to find a policy $\pi : S \rightarrow \mathcal{A}$ maximizing $V^\pi(s_0)$ defined as:

$$V^\pi(s_0) = \mathbf{E} \left[\sum_{t=0}^{\infty} \gamma^t \cdot R_t \right]$$

where s_0 is the start state and R_t is the received reward in round t .

18.1 Policy evaluation

Before we try to find the optimal policy for planning, the first question we should ask is: *given a policy π , how do we determine how good it is?* This is the question of **policy evaluation**. To answer this, let's unpack the definition of V^π a bit. Notice

$$\begin{aligned} V^\pi(s) &= \mathbf{E}[R_0] + \mathbf{E} \left[\sum_{t=1}^{\infty} \gamma^t \cdot R_t \right] \\ &= \mu(s, \pi(s)) + \gamma \cdot \mathbf{E} \left[\sum_{t=0}^{\infty} \gamma^t \cdot R_{t+1} \right] \\ &= \mu(s, \pi(s)) + \gamma \cdot \sum_{s' \in S} P(s'|s, \pi(s)) \cdot V^\pi(s'). \end{aligned}$$

This is the famous **Bellman Expectation Equation**:

$$V^\pi(s) = \mu(s, \pi(s)) + \gamma \cdot \sum_{s' \in S} P(s'|s, \pi(s)) \cdot V^\pi(s'),$$

which is satisfied by the value function of any policy π . Now, observe that we can view V^π as a length- $|S|$ vector where $(V^\pi)_s = V^\pi(s)$ for all $s \in S$. Taking this perspective, if we define the operator $T^\pi : \mathbb{R}^{|S|} \rightarrow \mathbb{R}^{|S|}$ which takes in a length- $|S|$ vector v and outputs a length- $|S|$ vector whose s -th coordinate is defined as:

$$(T^\pi(v))_s = \mu(s, \pi(s)) + \gamma \cdot \sum_{s' \in S} P(s'|s, \pi(s)) \cdot v(s').$$

Then based on the Bellman Expectation Equation, finding the value V^π of a given policy π is equivalent to solving for the *fixed point* of T^π , i.e., a vector v such that

$$T^\pi(v) = v.$$

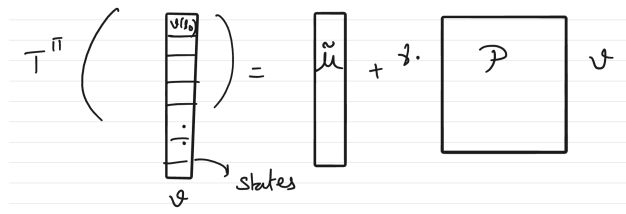


Figure 92: Matrix view of the operator T^π . Here, $v, \tilde{\mu}$ are vectors of length $|S|$. $\gamma \in \mathbb{R}$ is a constant. P is a $|S| \times |S|$ matrix.

As a result, computationally the fixed point satisfies:

$$T^\pi(v) = v \iff \tilde{\mu} + \gamma \cdot P \cdot v = v.$$

Rearranging, we have

$$(\mathbf{I} - \gamma P)v = \tilde{\mu} \implies v = (\mathbf{I} - \gamma P)^{-1} \cdot \tilde{\mu}.$$

Consequently, for any given policy π , we can evaluate its value exactly! Algorithmically, we can get away with performing the costly matrix inversion via the following **value iteration**:

1. Start with $v_0 = \bar{0}$.
2. For $k = 1, 2, \dots$:

$$v_{k+1} = T^\pi(v_k).$$

As we will see later, this iterative approach converges.

18.2 Finding the optimal policy for Planning